Operating Systems and Networks SoSe 25 Notes

Igor Dimitrov

2024-12-18

Table of contents

Preface **8**

I Exam Notes **9**

1 Assignment Sheets Overview **10**
1.1 Assignment Sheet 1 . . . 10
1.2 Sheet 2 . . . 11
1.3 Sheet 3 . . . 12
1.4 Sheet 4 . . . 13
Classification of Übungsblatt 4 . . . 13
Topics . . . 21
1.5 Sheet 5 . . . 21
Main syllabus-level topics . . . 21
Analysis of Übungsblatt 5 . . . 22
Why Aufgabe 2 and Aufgabe 6 stand out . . . 27
Aufgabe 1: important concept, less important packaging . . . 28
Aufgabe 3: TLB capacity + locality . . . 29
Aufgabe 4: TLB and context switches . . . 29
Aufgabe 5: useful, but supplementary . . . 29
Aufgabe 7: very useful calculation pattern . . . 30
Aufgabe 8: multilevel tables . . . 30
My exam-preparation grouping . . . 31
1.6 Sheet 6 . . . 32
Main syllabus-level topics . . . 32
Detailed subtopics . . . 32
Assignment-by-assignment classification . . . 32
Recommended study priority . . . 36
1.7 Sheet 7 . . . 37
Main syllabus-level topics . . . 37
Detailed subtopics . . . 38
Analysis of the individual assignments . . . 38
Important distinction for Aufgabe 2 . . . 42
Recommended exam-preparation priority . . . 42

1.8 Sheet 8 . . . 43
Main syllabus-level topics . . . 43
Detailed subtopics . . . 43
Analysis of the individual assignments . . . 44
Why these tasks are strongly exam-relevant . . . 48
Recommended preparation priority . . . 49
1.9 Sheet 9 . . . 49
Detailed subtopics . . . 50
Exercise-by-exercise analysis . . . 50
Recommended exam-preparation priority . . . 55
1.10 Sheet 10 . . . 56
Main syllabus-level topics . . . 56
Detailed subtopics . . . 57
Classification of the individual exercises . . . 57
Evidence from the reference exams . . . 60
Recommended study priority . . . 61
1.11 Overview . . . 61
Strongest sheets overall . . . 64

2 Plan **65**
2.1 Priority hierarchy . . . 65
Priority 1: direct reference-exam matches . . . 65
Priority 2: likely supporting material . . . 65
Priority 3: postpone unless time remains . . . 66
2.2 Ten-day core plan . . . 66
2.3 If you have 11–13 days . . . 68
2.4 How to structure each study day . . . 68
2.5 Build an error log, not another long summary . . . 69
2.6 Learn reusable solution templates . . . 69
2.7 The most important rule . . . 70

3 Processes and `fork()` **71**
3.1 Unit 1: `fork()`, process trees, and process states . . . 71
1. What is a process? . . . 71
3.2 2. What does `fork()` do? . . . 72
Return value . . . 72
Crucial distinction . . . 73
3.3 3. Counting processes . . . 73
3.4 4. The safe process-tree method . . . 74
Example . . . 74
3.5 5. Process hierarchy . . . 75
3.6 6. The five basic process states . . . 75
State transitions . . . 76

Where does `fork()` fit? . . . 76
3.7 7. Common exam traps . . . 76

4 Exam-like exercise **77**
Part A: `fork()` and the process tree . . . 77
Part B: Process states . . . 78
4.1 Answers . . . 79
Part A . . . 79
Part B . . . 80

5 Race conditions, semaphores and synchronization **81**
5.1 Why synchronization is necessary . . . 81
5.2 Race conditions . . . 82
Critical region . . . 82
Requirements of a good mutual-exclusion solution . . . 83
5.3 Atomicity . . . 83
5.4 Semaphores . . . 83
`wait(S)` . . . 84
`signal(S)` . . . 84
Binary and counting semaphores . . . 84
5.5 Mutual exclusion with a semaphore . . . 85
Mutex versus binary semaphore . . . 86
5.6 Enforcing an execution order . . . 86
5.7 Blocking versus busy waiting . . . 87
Busy waiting . . . 87
Blocking . . . 87
5.8 Condition variables . . . 87
Why `while`, not `if`? . . . 88
`signal` versus `broadcast` . . . 89
Condition variables do not remember notifications . . . 89
Why update and signal under the mutex? . . . 89
Example: producer-consumer queue . . . 90
Complete producer-consumer program . . . 91
Bounded-buffer producer-consumer problem . . . 92
5.9 Producer–consumer pattern . . . 95
5.10 Simplified read–write lock . . . 96
Naive Solution . . . 97
Solving fairness Problem . . . 98
5.11 Common exam traps . . . 100
5.12 Exam-like exercises . . . 100
Exercise 1: Race condition and semaphore repair . . . 100
Exercise 2: Ordering and condition variables . . . 101
Exercise 3: Debugging a semaphore construction . . . 102

5.13 Answers . . . 103
Exercise 1 . . . 103
Exercise 2 . . . 105
Exercise 3 . . . 106

6 Paging fundamentals 108
6.1 1. What paging does . . . 108
Fragmentation . . . 109
6.2 2. Address decomposition and powers of two . . . 109
Short worked example . . . 111
Units and useful calculations . . . 112
6.3 3. Normal page tables and translation . . . 113
When translation cannot complete . . . 114
Reconstructing entries from address pairs . . . 114
6.4 4. Inverted page tables . . . 114
Shared memory: the important limitation . . . 115
6.5 5. Calculating page-table memory . . . 116
Normal table . . . 116
Inverted table . . . 116
Worked comparison . . . 117
Address-field width is not complete entry size . . . 117
6.6 Common exam traps . . . 118
6.7 Exam-like exercises . . . 118
Exercise 1 — Two address spaces and a page boundary . . . 119
Exercise 2 — Reconstruct and audit the mappings . . . 119
Exercise 3 — Memory requirements and entry formats . . . 120
Source calibration and corrections . . . 121
6.8 Solutions . . . 122
Exercise 1 — Two address spaces and a page boundary . . . 122
Exercise 2 — Reconstruct and audit the mappings . . . 124
Exercise 3 — Memory requirements and entry formats . . . 128

7 Advanced paging and virtual memory 132
7.1 1. Hierarchical page tables . . . 132
Why divide a page table into levels? . . . 132
Address decomposition and translation . . . 133
Counting tables and entries . . . 134
Short worked example . . . 134
7.2 2. The TLB: caching translations . . . 135
Hit and miss . . . 135
Why a small TLB helps . . . 136
Quantitative access time . . . 136
Process and thread switches . . . 137

7.3 3. Demand paging and page-fault handling . . . 138
The basic sequence . . . 138
Page fault versus replacement . . . 138
Reference and dirty bits . . . 139
7.4 4. FIFO, LRU, and Clock . . . 139
FIFO: oldest arrival . . . 139
LRU: oldest last access . . . 140
One state illustrating the difference . . . 140
Clock / Second Chance . . . 140
Why Clock differs from exact LRU . . . 141
Reliable manual tracing . . . 141
7.5 5. Approximating recency with R bits . . . 142
Periodic sampling . . . 142
Sampling versus Clock . . . 143
R-bit scanning overhead . . . 143
7.6 Common exam traps . . . 144
7.7 Exam-like exercises . . . 145
Exercise 1 — A sparse three-level address space . . . 145
Exercise 2 — Translation misses and missing pages . . . 146
Exercise 3 — Complete FIFO, LRU, and Clock traces . . . 148
Exercise 4 — R-bit evidence and its cost . . . 149
7.8 Solutions . . . 150
Exercise 1 — A sparse three-level address space . . . 150
Exercise 2 — Translation misses and missing pages . . . 153
Exercise 3 — Complete replacement traces . . . 156
Exercise 4 — R-bit evidence and scanning cost . . . 159
7.9 Supplementary Matrial . . . 162
1. How epochs and shifting registers work . . . 162
2. Where is the history stored? . . . 164
3. How important is this for the exam? . . . 164

8 Day 5 — Deadlocks and CPU scheduling 166
8.1 Deadlocks: what actually prevents progress . . . 166
The four necessary Coffman conditions . . . 167
Necessary does not automatically mean sufficient . . . 167
Preventing deadlock through acquisition order . . . 168
8.2 Resource-allocation graphs . . . 168
What a cycle tells you . . . 169
8.3 The course's vectors and matrices . . . 170
8.4 Executing the course's Banker algorithm by hand . . . 172
Why add only C_i? . . . 173
Worked reduction of the graph above . . . 173
Interpreting the result . . . 173

Detection versus avoidance: the meaning of R matters . . . 174
8.5 CPU scheduling: the execution model . . . 174
FCFS — First Come, First Served . . . 175
SJF — Shortest Job First . . . 175
Round Robin — RR . . . 176
8.6 Gantt diagrams and time calculations . . . 177
A reliable manual procedure . . . 178
Fairness and starvation . . . 178
8.7 Common exam traps . . . 179
8.8 Exam-like exercises . . . 179
Exercise 1 — A three-process semaphore deadlock . . . 179
Exercise 2 — Resource situation → graph → matrices → Banker . . . 180
Exercise 3 — Scheduling with arrivals and queue boundaries . . . 181
8.9 Solutions . . . 182
Exercise 1 — A three-process semaphore deadlock . . . 183
Exercise 2 — Resource situation → graph → matrices → Banker . . . 185
Exercise 3 — Scheduling with arrivals and queue boundaries . . . 190
Suggested self-marking guide . . . 196

Preface

Part I

Exam Notes

1 Assignment Sheets Overview

1.1 Assignment Sheet 1

Task	Topic / subtopic	Type	Exam relevance	Revision priority
A1	Learning methodology: focused/diffuse thinking, creativity	Research + conceptual / supplementary	**Low**	Skip for IBN content revision
A2	Computer architecture: x64 general-purpose registers, AVX2 registers, powers of two	Research + calculation	**Medium-low**	Know the underlying ideas, probably don't spend much time on lookup
A3	OS protection: process address spaces, user/kernel mode, controlled resource access	Conceptual / application	**High**	Definitely do
A4	Kernel mode / kernel stack; interrupts and system calls	Conceptual / explanation	**High**	Definitely do
A5	POSIX system calls and minimal interfaces	Conceptual / design	**Medium**	Useful, but selective

Task	Topic / subtopic	Type	Exam relevance	Revision priority
A6	Bash shell scripting	Programming	**Low–medium**	Understand shell concepts; full implementation is lower priority
A7	Advantages/disadvantages of abstractions	Conceptual / supplementary	**Low**	Optional

1.2 Sheet 2

Task	Main topic / subtopic	Problem type	Exam relevance	What I'd do
A1	PCB / Linux `task_struct`	Research + conceptual	**Medium**	Learn PCB concept; skip detailed kernel-source research
A2a	`fork()`, return values, process tree	Trace / code analysis	**Very high**	**Definitely solve fully**
A2b	`fork()`, PID-dependent behavior	Trace + conceptual / exploratory	**Medium**	Understand, but lower priority
A3	Process hierarchy, process states, `ps`, background/suspended processes	Practical experiment + conceptual	**High**	Do conceptually; actual terminal experiment optional
A4	`exec*()` family	Research / API comparison	**Medium-low**	Learn core `exec` semantics; don't memorize all signatures
A5	Process/thread context switching	Conceptual / mechanism	**High**	**Definitely study**

Task	Main topic / subtopic	Problem type	Exam relevance	What I'd do
A6.1	Thread creation/join/order	Programming + synchronization	**Medium**	Useful, but belongs more to Threads
A6.2–3	Thread performance measurement	Empirical / performance	**Low**	Skip for exam revision

1.3 Sheet 3

Topics: Threads, Race Conditions, Mutual Exclusion, Critical Sections, Semaphores, Mutexes, Condition Variables, Active/Busy Waiting, Producer–Consumer Problem, Read–Write Locks, Deadlock, Starvation, Pthreads synchronization APIs, Win32 synchronization APIs.

Aufgabe	Topic / subtopic	Type of task	Exam relevance	Exam-likeness
1	**Concurrency → Race Conditions / Mutual Exclusion**	Code analysis, conceptual reasoning	**Very high**	**Very high**
2	**Synchronization → Semaphores, Mutexes, Condition Variables**	Conceptual / video-based / supplementary theory	**High**	**Medium–High**
3	**Synchronization → Producer–Consumer, Condition Variables, Active Waiting**	Programming / implementation / performance experiment	**High conceptually**, lower for full implementation	**Medium**

Aufgabe	Topic / subtopic	Type of task	Exam relevance	Exam-likeness
4	**Synchronization → Semaphores, Read–Write Locks, Deadlock / Starvation / Mutual Exclusion**	Pseudocode analysis + construction	**Very high**	**Very high**
5 Bonus	**Synchronization APIs → POSIX/Pthreads vs. Win32 Semaphores**	Research / API lookup / coding	**Low–Medium**	**Low**

1.4 Sheet 4

Yes. Looking at **Übungsblatt 4 as a whole**, there is a fairly clear transition in the course here: Aufgaben 1–3 finish the block on **Interprozesskommunikation / Pipes**, while Aufgaben 4–7 begin **Speicherverwaltung**, especially fragmentation, contiguous allocation, logical/physical addresses and the foundations of paging. This matches the organization of the summary: IPC comes before threads/synchronization, while memory management then introduces swapping, fragmentation, MMU/logical vs. physical addresses, followed by paging. ibn-zusammenfassung ibn-zusammenfassung ibn-zusammenfassung

The exam evidence makes the **memory-management half particularly important**. The 2016 exam devoted an entire 12-point problem to paging and address translation, including page-table sizes, logical/physical addresses, page number/offset and inverted page tables. ibn-klausur-2016 The more recent memory protocol likewise reports a large paging/memory-management problem involving page tables, logical/physical addresses and calculations. Gedächtnisprotokoll_Klausur_ IB…

Classification of Übungsblatt 4

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
1	**Interprocess Communication → named vs. anonymous pipes**	Conceptual / application / example construction	**Medium**	**Medium**	The concepts themselves are standard OS material and explicitly appear in the summary. You need to understand the limitations and use cases of named vs. anonymous pipes. The requirement to invent your own scenario is somewhat exercise-like rather than typical written-exam style.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
2	**IPC / OS interfaces → Win32 handles, I/O redirection**	**Research + programming/API investigation**	**Low**	**Very low**	Strongly research-oriented: look up Win32 handle representation and construct Windows API code for redirection. The summary and the available exams concentrate much more on POSIX concepts than on exact Win32 API details. Useful background, but not something I would prioritize for exam preparation.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
3	**IPC → anonymous pipes, bidirectional process communication**	**Programming / implementation**	**Medium**	**Low–Medium**	Anonymous pipes themselves are relevant; the summary explicitly covers `pipe()`, `read()` and `write()`. ibn-zusammenfassung But writing a complete program with two processes and two-way communication is more implementation-heavy than the exam evidence suggests. Understanding *how* the two pipes must be arranged is more important than memorizing code.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
4	**Memory management → fragmentation, swapping, paging, allocation trade-offs**	**Research / conceptual / diagram interpretation**	**Medium–High concepts**, but **Low–Medium as an exercise**	**Low–Medium**	The concepts are important: fragmentation, swapping and paging are core memory-management topics. But the task is built around watching a research talk and interpreting a specific allocator paper, which makes the assignment itself much less exam-like. Part **(b)** on swapping and **(c)** on paging are more useful than the research-specific details.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
5	**Memory management → internal vs. external fragmentation; logical vs. physical addresses**	**Conceptual definition/comparison**	**High**	**High**	Very good exam-prep question. These are fundamental distinctions explicitly covered in the summary. ibn-zusammenfassung Short "explain the difference between X and Y" questions are also exactly the sort of thing that can appear as exam subquestions.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
6	**Memory management → contiguous allocation; First Fit, Best Fit, Worst Fit**	**Calculation / algorithm simulation**	**Medium–High**	**High**	Very exam-like format: small concrete data set, apply three algorithms by hand. The summary explicitly lists First Fit, Best Fit and Worst Fit as the allocation strategies for holes. ibn-zusammenfassung It has not appeared explicitly in the two exam documents we have, so I would put it below paging, but it is exactly the kind of calculation that could easily be asked.

Aufgabe	Topic → subtopic	Type of task	Exam relevance	How exam-like?	Assessment
7	**Paging → splitting a logical address into page number + offset**	**Calculation + tiny coding component**	**Very High**	**Very High**	One of the strongest exam-prep tasks on the sheet. The summary explicitly requires understanding page number and offset. ibn-zusammenfassung More importantly, the **2016 exam asks essentially this exact operation**: given a logical address and page size, determine offset and page index. ibn-klausur-2016 The recent memory protocol again says logical/physical addresses and page calculations appeared. Gedächtnisprotokoll_Klausur_IB…

The actual sheet confirms this progression very clearly: Aufgaben 1–3 are named pipes, Win32 handles/redirection, and anonymous-pipe communication; Aufgaben 4–7 then move to fragmentation/swapping/paging, memory-management terminology, placement algorithms and page-number/offset calculations. ueb04-ibn-SS2025 ueb04-ibn-SS2025

Topics

Topics: Interprocess Communication (IPC), Pipes, I/O and file-handle abstractions, Memory Management, Swapping, Fragmentation, Contiguous Memory Allocation, Logical vs. Physical Addressing, Paging

If we compress that to the **main syllabus-level topics**, rather than listing every subtopic:

Topics: Interprocess Communication, Memory Management, Paging

For exam preparation, I would mentally divide Blatt 4 into roughly three priority groups: **Aufgabe 7 and 5 are especially important**, Aufgabe 6 is also worth being able to solve comfortably, while Aufgaben 1 and 3 are useful IPC knowledge but less strongly represented in the exam evidence. Aufgabe 4 contains important theory inside a relatively non-exam-like research exercise, and Aufgabe 2 is the clearest candidate to deprioritize.

1.5 Sheet 5

Yes. **Übungsblatt 5 is unusually exam-relevant overall**. Unlike some earlier sheets, almost the entire sheet is concentrated on one major syllabus block—**Paging and page tables**—and that block appears prominently in both exam documents. In the more recent *Gedächtnisprotokoll*, Paging/Memory Management was even described as a comparatively extensive exam problem, involving construction of page tables, logical/physical addresses, unit conversions, and an inverted-page-table question. Gedächtnisprotokoll_Klausur_ IB… The 2016 exam likewise devoted an entire 12-point problem to page-table sizes, inverted page tables, address decomposition, and page-table contents. ibn-klausur-2016

Main syllabus-level topics

The sheet belongs almost entirely to **Speicherverwaltung → Paging und Seitentabellen**. More specifically, I would place its material under:

- **Paging and address translation**
  - logical vs. physical addresses
  - page number, frame number, offset
  - direct/normal page tables

- **Page-table organizations**
  - direct page tables
  - inverted page tables
  - hierarchical / multilevel page tables
  - memory consumption of page tables
- **Translation Lookaside Buffer (TLB)**
  - TLB hit / miss
  - TLB capacity
  - locality and repeated accesses
  - TLB invalidation on context switches
  - effective memory-access time
- **Hardware/software interaction in virtual memory**
  - process-specific address mappings
  - why address translation state changes on a process switch
- **Practical address translation**
  - following a translation through a simulator
  - decomposing virtual addresses across levels of a page table

That matches the organization of the summary very closely: the summary's section **"7. Paging und Seitentabellen"** starts with pages, frames, address decomposition and page tables, and then covers TLBs, effective access time, direct/inverted tables and multilevel page tables. ibn-zusammenfassung ibn-zusammenfassung ibn-zusammenfassung

Analysis of Übungsblatt 5

Aufgabe	Topic / subtopic	Type of task	Exam relevance	How exam-like?	Assessment
1	Paging; direct page table; mapping **page number → frame number**; sparsity/size of a direct table	**Conceptual analogy + estimation**	**High**	**Medium**	The student-ID/address analogy itself is pedagogical and somewhat assignment-like, but the underlying ideas—what indexes a page table, what its entries contain, and why a direct table can be huge/sparse—are core exam material. ueb05-ibn-SS2025

Aufgabe	Topic / subtopic	Type of task	Exam relevance	How exam-like?	Assessment
2	Address translation; page number, offset, frame number; constructing direct page-table entries	**Calculation + table construction**	**Very high**	**Very high**	One of the most important tasks on the sheet. Given logical→physical address mappings, you reconstruct the relevant page-table entries. This is extremely close to actual exam material. ueb05-ibn-SS2025
3	TLB; page size; stride accesses; capacity misses; temporal/spatial locality	**Calculation / reasoning from code**	**Medium–High**	**Medium**	Important conceptual TLB exercise, but more elaborate than what we have direct evidence for in the exams. Excellent for understanding *why* a TLB works, rather than a direct historical exam clone. ueb05-ibn-SS2025

Aufgabe	Topic / subtopic	Type of task	Exam relevance	How exam-like?	Assessment
4	TLB invalidation; process switch vs. thread switch; process-specific page tables	**Conceptual explanation + counterexample**	**High**	**Medium–High**	Good exam-style conceptual question: explain a mechanism and construct a failure scenario. Not directly repeated in the supplied exams, but squarely within core paging/TLB knowledge. ueb05-ibn-SS2025
5	Multilevel translation + TLB lookup	**Simulator / practical supplementary exercise**	**Low–Medium**	**Low**	The concepts are important, but "run simulator and submit screenshot" is obviously not an exam format. Use it for visualization, not as a priority exam exercise. ueb05-ibn-SS2025

Aufgabe	Topic / subtopic	Type of task	Exam relevance	How exam-like?	Assessment
6	Direct page-table memory size; inverted page table; RAM overhead	**Calculation**	**Extremely high**	**Extremely high**	Probably the strongest exam-preparation task on the sheet. The 2016 exam asks essentially these calculations, and the older minitest included in the exam document contains almost the *same wording*. ueb05-ibn-SS2025

Aufgabe	Topic / subtopic	Type of task	Exam relevance	How exam-like?	Assessment
7	TLB hit ratio; hierarchical page tables; expected access time; locality	**Formula derivation + conceptual explanation**	**High**	**High**	Very exam-suitable: derive an expected-time formula, then explain why the simplistic probabilistic model is unrealistic. Combines calculation with conceptual understanding. ueb05-ibn-SS2025
8	Multilevel page tables; splitting address bits; entries per level; table contents; maximum memory use	**Diagram + calculation + conceptual explanation**	**High**	**Medium–High**	Very useful and comprehensive, but somewhat longer than a typical short exam subquestion. The individual skills it tests are highly examinable. ueb05-ibn-SS2025

Why Aufgabe 2 and Aufgabe 6 stand out

Aufgabe 2 is almost exactly the skill tested in the recent exam protocol. The protocol says that the Paging problem required students to **create/fill page tables** and

determine **logical and physical addresses**, with PIDs and addresses given. Gedächtnisprotokoll_Klausur_ IB… That is essentially what Aufgabe 2 trains.

The 2016 exam is equally direct. It gives a logical address translated to a physical address, asks for its **offset, page index and frame number**, and then asks for the corresponding entries of both a normal and an inverted page table. ibn-klausur-2016

So for exam preparation I would treat **Aufgabe 2 as mandatory**.

Aufgabe 6 is even more striking. The 2016 exam asks for the memory requirement of a normal page table and then for the memory requirement of an inverted page table. ibn-klausur-2016 Moreover, the old 2012 minitest included in the same PDF asks:

- maximum memory requirement of a page table without hierarchical tables;
- what fraction of physical memory an inverted page table would require. ibn-klausur-2016

That is essentially **Übung 5, Aufgabe 6**, just with the parameters packaged slightly differently. So this is not merely "the same topic"; it is a historically demonstrated **exam question form**.

Aufgabe 1: important concept, less important packaging

The point of Aufgabe 1 is to make you understand the structural relationship

$$\text{page number} \longrightarrow \boxed{\text{page table}} \longrightarrow \text{frame number}.$$

That is exactly the formulation used in the summary: the **page number is the index**, and the corresponding page-table entry contains the **frame number**. ibn-zusammenfassung

The student-residence analogy itself is unlikely to appear verbatim on an exam. But once you strip away the analogy, it tests two important questions:

1. What does a direct page table actually represent?
2. Why can a direct page table contain an enormous number of unused entries?

So I would **understand it thoroughly but not spend much time rehearsing the dormitory analogy itself**.

Aufgabe 3: TLB capacity + locality

This one is more sophisticated. You have a 4 KiB page, 4-byte integers, an array accessed with stride (M), and a TLB containing 64 entries. You have to reason about when successive array accesses hit different pages and when the working set exceeds the TLB capacity. ueb05-ibn-SS2025

The summary explicitly treats the TLB as a cache of page-table entries, including TLB hits/misses and the fact that only a small number of pages tend to be accessed frequently. ibn-zusammenfassung

So the **knowledge is definitely syllabus material**, but the exact "choose (M) and (N)" formulation looks more like a deeper exercise than a historically observed exam problem. I'd therefore study the reasoning, but not prioritize it above Aufgaben 2 and 6.

Aufgabe 4: TLB and context switches

This question tests an important consequence of the fact that **different processes have different page tables**.

The summary explicitly says that each process needs its own direct page table because different processes may use the same logical addresses while referring to different data. ibn-zusammenfassung

Therefore stale entries such as

$$p \to f_A$$

from process A cannot normally simply remain valid when process B, which may map the same page (p) to (f_B), starts executing.

The "does this also apply to threads?" part is particularly useful because it checks whether you understand that threads of one process normally share the same address space.

This is a good **short conceptual-exam-question style**, even though neither of our two exam documents directly asks it.

Aufgabe 5: useful, but supplementary

This is the easiest one to deprioritize. It literally asks you to use the Address Translation Simulator and submit a screenshot. ueb05-ibn-SS2025

The **translation process it illustrates is important**. The activity itself is not.

For exam preparation I'd classify it:

Supplementary visualization exercise — do once if the mechanism is unclear; otherwise skip during final revision.

Aufgabe 7: very useful calculation pattern

This task combines two things:

expected access time

and

locality / why TLB hit rates are high.

The summary explicitly contains an expected-access-time calculation for a TLB:

$$E[X] = h(1 + T) + (1 - h)(2 + T)$$

for the simple one-level case, together with the interpretation of TLB hit and miss costs. ibn-zusammenfassung

Aufgabe 7 generalizes essentially the same reasoning to a **(k)-level page table**. ueb05-ibn-SS2025

That makes it quite valuable: it is not merely memorizing a formula—you need to reconstruct the formula from the number of memory accesses.

I would therefore consider it **strong exam preparation**, even though our two exam samples do not show this exact question.

Aufgabe 8: multilevel tables

This is the sheet's most comprehensive page-table-structure problem. You have to divide a 32-bit logical address among:

- page offset,
- level-1 index,
- level-2 index,
- level-3 index,

and then determine the number of table entries and total maximum memory usage. ueb05-ibn-SS2025

The summary explicitly expects understanding of this structure: upper levels contain addresses of lower-level page tables, while only the lowest level plays the role of the ordinary direct page table. ibn-zusammenfassung

So this is **core syllabus material**. Its only downside from an exam-preparation perspective is that the exercise is rather extensive. An actual exam might split it into smaller subquestions.

My exam-preparation grouping

If we use the same practical categories we were using for the previous sheets, I would group them approximately like this:

Priority	Tasks	Reason
Must be able to solve confidently	**2, 6**	Direct overlap with documented exam questions
Strong exam preparation	**4, 7, 8**	Core mechanisms + highly exam-suitable reasoning/calculation
Important understanding, somewhat more exercise-like	**1, 3**	Core material, but the particular formulation is less directly evidenced by exams
Supplementary	**5**	Simulator activity rather than an exam question

And one especially useful conclusion from this sheet is that **"Paging und Seitentabellen" should be treated as a major exam block rather than a peripheral chapter**. The 2016 exam allocated **12/52 points** to Speicherverwaltung, with almost the entire problem centered on page tables. ibn-klausur-2016 The newer memory protocol again reports a large Paging/Memory-Management problem involving page tables, address translation, size calculations, and inverted page tables. Gedächtnisprotokoll_Klausur_ IB…

So compared with Übungsblätter 2–4, **Übungsblatt 5 has a very high concentration of material that maps directly onto demonstrated IBN exam content.**

1.6 Sheet 6

Main syllabus-level topics

1. **Memory management**
2. **Paging and page tables**
3. **Virtual memory and page replacement**
4. **Locality and working sets**
5. **Segmentation and contiguous memory allocation**

Detailed subtopics

- Page-reference sequences and page faults
- FIFO, LRU, Clock/Second Chance and OPT
- Comparing page-replacement algorithms
- Hardware-supported approximation of LRU
- Reference/accessed bits (**R-bits**)
- Epoch length and page-table scanning overhead
- Working set $WS(t, \Delta)$
- Computing the largest working set in a reference sequence
- Direct page-table size in IA-32
- Memory utilization and fragmentation
- Heap and stack growth
- Comparison of contiguous allocation, segmentation and paging

There is no real **research task** on this sheet. The main categories are calculations, algorithm simulation, conceptual reasoning, pseudocode and practical programming.

Assignment-by-assignment classification

Task	Topic and subtopic	Type	Exam relevance	Exam-like?	Concise assessment
1a	Virtual memory → FIFO and LRU page replacement	Calculation / manual simulation	**Very high**	**Very high**	Almost exactly the kind of page-fault calculation already used in a mini-test: simulate frames and count faults.
1b	Page replacement → Clock versus LRU	Constructive example / algorithm comparison	**High**	**Medium–high**	Requires genuine understanding of how the two algorithms choose victims, not merely memorizing definitions.
2	Page replacement → mathematical relation between OPT and LRU	Theoretical/conceptual reasoning	**Medium**	**Low–medium**	An interesting theoretical insight, but less representative than concrete page-fault calculations.

Task	Topic and subtopic	Type	Exam relevance	Exam-like?	Concise assessment
3	Page replacement → implementation of Clock or LRU	Programming / practical implementation	**Medium for understanding**	**Very low**	Writing and submitting a complete program is unrealistic in a written exam. Understanding its state transitions remains useful.
4a	Paging → R-bits, approximate LRU and page-table scanning	Calculation / performance analysis	**Very high**	**High**	Closely related to the 2016 question about approximating the last-access time using hardware R-bits.
4b	Virtual memory → choice of epoch length	Estimation / design reasoning	**High**	**Medium**	More open-ended than a typical calculation; you must state assumptions and balance accuracy against overhead.

Task	Topic and subtopic	Type	Exam relevance	Exam-like?	Concise assessment
5	Working sets → membership and possible overlap	Short conceptual question	**Medium–high**	**Medium–high**	A compact definition-and-justification question. Working-set definitions have appeared in the supplied exam material.
6	Working sets → maximum distinct pages in a sliding window of length ( )	Algorithm design / pseudocode	**High**	**Medium**	The core sliding-window algorithm is exam-suitable, although the file-processing framing is somewhat assignment-like.

Task	Topic and subtopic	Type	Exam relevance	Exam-like?	Concise assessment
7	Memory management → contiguous allocation vs segmentation vs paging	Conceptual comparison	**High**	**Medium–high**	A classic explain-and-compare question. Paging is demonstrably exam-relevant; segmentation is less visible in the supplied exams but belongs to the core syllabus.

Recommended study priority

Priority A — practise thoroughly

- **1a:** FIFO/LRU calculation
- **4a:** R-bit scanning calculation
- **4b:** epoch-length reasoning
- **7:** comparison of memory-management schemes

These combine central syllabus material with realistic written-exam formats.

Priority B — understand and be able to reproduce

- **1b:** Clock versus LRU
- **5:** working-set membership
- **6:** largest-working-set pseudocode

These test whether you understand the algorithms and definitions beyond memorization.

Priority C — understand the main idea

- **2:** OPT–LRU mathematical relationship

Useful theoretical knowledge, but comparatively specialized.

Priority D — optional implementation work

- **3:** full simulator implementation

For exam preparation, manually tracing the simulator's states matters much more than writing the complete program.

Overall, **sheet 6 has high exam relevance**. The strongest evidence is that page-replacement calculations appear almost directly in the supplied material, the 2016 exam asks about R-bit-based recency approximation, and the recent exam recollection describes paging as a substantial exam topic.

1.7 Sheet 7

Main syllabus-level topics

Assignment sheet 7 is primarily about:

1. **File systems**
   - Links, FAT, I-nodes, directories and pathname resolution
2. **Secondary-storage management**
   - Hard-disk access times and disk-scheduling algorithms
3. **RAID**
   - Reliability, redundancy and read performance
4. **Paging and page tables**
   - Only in Aufgabe 3, as an analogy to I-nodes

This corresponds closely to Chapter 10, "Dateien, Dateisysteme, Festplatten," of the summary. 01-ibn-zusammenfassung.pdf

A particularly important observation is that the recent exam memory protocol explicitly mentions **both FAT and I-nodes**. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

Detailed subtopics

- Hard links versus symbolic links
- FAT and linked file allocation
- Cluster allocation and traversal
- I-node structure
- Direct and indirect block addressing
- Maximum file-size calculations
- Directory traversal and pathname lookup
- Analogy between paging and I-nodes
- Seek time and rotational latency
- Conversion between RPM and rotational latency
- FCFS, SSTF, SCAN, C-SCAN and C-LOOK
- RAID 0 versus RAID 1
- Failure probabilities
- Parallel and load-balanced reads

Analysis of the individual assignments

The following classification is based on the full sheet. 07-ueb07-ibn-SS2025.pdf

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
1	File systems: hard vs. symbolic links	Conceptual comparison / recall	**Medium–high**	**High**	A typical short conceptual exam question. Know differences involving I-nodes, cross-filesystem links, dangling links and deletion.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
2	File allocation: FAT, clusters, allocation and traversal	Programming / simulation / practical application	**Very high content relevance**	**Low for the complete task; high for its concepts**	FAT appeared in the recent exam protocol. Writing the entire simulator is unlikely in an exam, but tracing a FAT or allocating clusters is highly relevant.
3	I-nodes and paging: structural analogy, multilevel tables	Conceptual transfer / supplementary deepening	**Medium–high**	**Low–medium**	Both constituent topics are important, but the analogy itself is somewhat artificial and less representative of a direct exam question.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
4	I-nodes: maximum file size with direct and single-indirect pointers	Calculation	**Very high**	**Very high**	One of the strongest exam-preparation tasks. Almost the same type of I-node calculation appears in the older mini-test material appended to the 2016 PDF.
5	Directories and I-nodes: pathname resolution and disk operations	Calculation / process tracing	**High**	**High**	Tests whether you understand how each pathname component requires directory-block and I-node access. A classic filesystem exam problem.
6	Hard disks: rotational latency, seek time, total access time, RPM	Numerical calculation	**Medium–high**	**High**	The historical information is contextual; the underlying calculations are standard and exam-like.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
7	Disk scheduling: FCFS, SSTF, SCAN, C-SCAN, C-LOOK	Algorithm execution / calculation	**High**	**Very high**	A classic "execute the algorithm and calculate the cost" task. Pay attention to initial direction and whether the head travels to the physical endpoint.
8	RAID 0/1: failure probabilities	Probability calculation + explanation	**Very high**	**Very high**	Especially important: essentially this exact RAID 0/1 probability task occurs in the older mini-test material contained in the 2016 PDF.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
9	RAID 0/1: comparative read performance	Conceptual reasoning / performance analysis	**Medium–high**	**High**	A good short exam question. Requires distinguishing a single sequential read from multiple independent reads that can be distributed across mirrors.

The comparison material comes from the 2016 examination PDF, including the older mini-tests appended to it. 12-ibn-klausur-2016.pdf

Important distinction for Aufgabe 2

I would not treat the entire FAT simulator as something you must reproduce under exam conditions. Split it into two layers:

- **Exam-relevant:** understanding the FAT representation, following a cluster chain, allocating clusters, calculating the number of required clusters and recognizing error cases.
- **Mostly practical:** designing the class, writing complete code, testing it and printing outputs.

Thus, Aufgabe 2 is simultaneously **very important in subject matter** and **not very exam-like in its full implementation format**.

Recommended exam-preparation priority

1. **Aufgabe 4** — direct I-node calculation and closely matched by older exam material

2. **Aufgabe 8** — essentially an exact previous mini-test question

3. **Aufgabe 2, conceptual portion** — FAT explicitly appeared in the recent exam protocol

4. **Aufgabe 5** — fundamental I-node/directory mechanics

5. **Aufgabe 7** — standard algorithm-execution problem

6. **Aufgaben 1 and 9** — compact conceptual questions

7. **Aufgabe 6** — standard calculations, but less directly supported by the supplied exams

8. **Aufgabe 3** — useful for deeper understanding, but the least directly exam-like formulation

Overall, sheet 7 is **highly relevant**. It contains two exceptionally strong past-exam matches—Aufgaben 4 and 8—and directly practises FAT and I-nodes, which the recent memory protocol identifies as actual exam topics.

1.8 Sheet 8

Main syllabus-level topics

1. **Deadlocks and resource allocation**
2. **CPU scheduling**

Sheet 8 is especially important for exam preparation because deadlocks and the Banker algorithm occur directly in both exam references. Scheduling calculations also occur in an older mini-test included in the 2016 exam document.

The classification below is based on the assignment sheet 08-ueb08-ibn-SS2025.pdf, the syllabus structure in the summary 01-ibn-zusammenfassung.pdf, and comparison with the 2016 exam 12-ibn-klausur-2016.pdf and the recent exam memory protocol 13-Gedchtnisprotokoll_Klausur_-IBN_Ersttermin.pdf.

Detailed subtopics

Deadlocks

- Definition of a resource deadlock
- Four necessary Coffman conditions:
  - mutual exclusion

  - hold and wait
  - no preemption
  - circular wait

- Necessary versus sufficient conditions
- Deadlocks caused by incorrect semaphore acquisition
- Prevention through a globally consistent resource-acquisition order
- Resource-allocation graphs
- Multiple instances of a resource type
- Resource vector E and available-resource vector A
- Allocation matrix C and request matrix R
- Banker algorithm
- Safe execution sequences
- Deadlock detection versus deadlock avoidance

CPU scheduling

- CPU-bound versus I/O-bound processes
- CPU bursts and I/O bursts
- FCFS
- Shortest Job First
- Round Robin and the time quantum
- Gantt diagrams
- Turnaround time
- Waiting time
- Average waiting time
- Fairness and starvation
- Priority scheduling

Analysis of the individual assignments

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
1	Deadlocks: necessary conditions, multiple resource instances, necessary vs. sufficient	Conceptual proof; counterexample construction	**High**	**High**	Very close to the 2016 question asking for graphs with cycles both with and without a necessary deadlock. Although marked bonus, the underlying distinction is exam-relevant.
2	Semaphore deadlock; four Coffman conditions; deadlock prevention	Execution trace; conceptual explanation; design/fix	**Very high**	**Very high**	Almost a direct exam pattern: show how opposite lock ordering produces deadlock, identify the four conditions, and remove circular wait through consistent lock ordering.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
3	Banker algorithm; matrices (E,C,R,A); identifying deadlocked processes	Programming; algorithm implementation; testing	**High conceptually**	**Medium as a full task**	Implementing a complete program is unlikely in a written exam. Understanding the algorithm, writing pseudocode and tracing it manually are extremely relevant.
4	Resource-allocation graph; resource vectors and matrices; manual Banker algorithm	Graph construction; data representation; hand calculation	**Very high**	**Very high**	The strongest exam-preparation task on the sheet. The recent exam reportedly used essentially this sequence: draw the graph and apply the Banker algorithm.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
5	Scheduling: CPU-bound versus I/O-bound processes	Conceptual explanation	**Medium**	**Medium**	A plausible short theory question. Know that I/O-bound processes usually have short CPU bursts and should receive CPU time quickly to keep devices and the system responsive.
6	FCFS, SJF and Round Robin; Gantt diagrams; turnaround and waiting times	Calculation; algorithm trace; comparison	**High**	**High**	A standard scheduling calculation. An older mini-test in the 2016 document asks for nearly the same SJF/RR execution trace. Practise this carefully.

Aufgabe	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
7	Starvation under scheduling algorithms	Conceptual classification	**Medium–High**	**Medium–High**	A very typical one-point conceptual question. Under the usual assumptions, starvation can occur with **SJF** and **priority scheduling**, but not with FCFS or Round Robin.

Small issue in Aufgabe 6

Part (b) refers to *„die vier genannten Scheduling-Algorithmen“*, but part (a) names only three:

- FCFS
- Shortest Job First
- Round Robin

This is almost certainly a typo. Unless another algorithm was specified orally, calculate the values for these three.

Why these tasks are strongly exam-relevant

The match with the exam evidence is unusually direct:

- The **2016 exam** contains an entire 11-point section on deadlocks, including:
  - deadlock definition,
  - semaphore code producing a deadlock,
  - resource-allocation graphs,
  - cycles that do or do not imply deadlock,

  - the purpose and main idea of the Banker algorithm.

- The **recent exam protocol** explicitly reports:
  - drawing a resource/allocation graph,
  - applying the Banker algorithm,
  - concluding that no deadlock exists.

- An older mini-test included in the 2016 document asks for **SJF and Round-Robin scheduling traces**, closely matching Aufgabe 6.

Recommended preparation priority

1. **Aufgabe 4** — closest match to the recent exam
2. **Aufgabe 2** — direct classical exam pattern
3. **Aufgabe 6** — central scheduling calculation
4. **Aufgabe 1** — important conceptual distinction about multiple instances
5. **Aufgabe 3, algorithmic core** — understand and trace it; full programming is secondary
6. **Aufgabe 7** — easy, compact exam knowledge
7. **Aufgabe 5** — useful short conceptual background

Overall, **sheet 8 is one of the most exam-relevant sheets so far**, mainly because Aufgaben 2 and 4 correspond almost directly to documented exam questions.

1.9 Sheet 9

Sheet 9 is almost entirely about **computer networks**, progressing through the protocol stack:

1. **Fundamentals of packet-switched networks**
2. **Network delay and queueing**
3. **Internet structure, routing paths and topology**
4. **Protocol layers, encapsulation and addressing**
5. **Transport protocols and reliable communication**
6. **Application-layer protocols, especially HTTP**
7. **IPv4 fragmentation**

This classification is based on the exercises in sheet 9, the networking section of the summary, and the two available exam documents. 09-ueb09-ibn-SS2025.pdf 01-ibn-zusammenfassung.pdf

Detailed subtopics

- Packet switching versus circuit switching
- Statistical multiplexing
- Binomial probability for simultaneously active users
- Queueing delay and transmission time
- Traceroute and interpretation of network paths
- ISPs, BelWü, network redundancy and Internet exchange points
- Internet protocol stack
- Protocol data units: message, segment/datagram, IP datagram and frame
- Endpoints and addressing at different layers
- Encapsulation and protocol robustness
- TCP over extremely long-delay or intermittently connected networks
- HTTP cookies and state management
- HTTP requests and responses
- Conditional HTTP GET and caching
- Relationship between HTTP messages and TCP segments
- IPv4 fragmentation, MTU, identification, flags and fragment offsets

Exercise-by-exercise analysis

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
1	Packet switching; statistical multiplexing; binomial probability	Calculation / modelling	**Medium**	**Medium**	The underlying idea is important, but the large binomial-tail calculation is more specialised than the observed exam questions. Understand the setup and interpretation.
2	Queueing delay; transmission time (L/R)	Formula derivation / calculation	**High**	**High**	Short, self-contained and closely related to earlier exam-style calculations involving transmission and propagation delays.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
3	Traceroute; ISP topology; redundancy; Internet exchange points	Practical investigation / research	**Medium**	**Low**	Useful networking intuition, but dependent on live infrastructure and external research. It does **not** directly practise Dijkstra or subnetting, despite being routing-related.
4	Protocol stack; encapsulation; PDUs; endpoints; addressing	Conceptual knowledge / structured table	**Very high**	**Very high**	One of the most important exercises. The 2016 exam contained an extremely similar task about encapsulation, layer headers and address-like information.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
5	Encapsulation; layer-dependent attack scope; protocol security	Conceptual reasoning / security enrichment	**Low–medium**	**Low**	Tests whether you understand the scope of transport, network and link-layer communication, but the recursive ZIP scenario is unusual and supplementary.
6	TCP reliability under enormous delay and disrupted connectivity; delay-tolerant networking	Research / conceptual transfer / bonus	**Medium**	**Low–medium**	TCP reliability and connection establishment are relevant, but the Mars/DTN setting and required reading make this primarily enrichment.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
7	HTTP cookies; maintaining state over stateless HTTP	Short conceptual explanation	**High**	**High**	Directly represented in the summary and well suited to a short written exam question. Know the first response, cookie storage, later request and server-side association.
8	HTTP in Wireshark; GET/response; conditional GET; TCP segmentation; embedded objects	Practical laboratory / protocol analysis	**Medium–high**	**Low** as a complete exercise	Wireshark operation is unlikely in a paper exam, but the protocol observations it reveals can become conceptual exam questions.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
9	IPv4 fragmentation; MTU; header length, ID, MF flag and fragment offset	Calculation / header analysis	**High**	**Very high**	A classic written-exam calculation. It directly matches the fragmentation material in the summary and requires precise handling of byte lengths and offsets.

The particularly strong rating for Aufgabe 4 comes from the 2016 exam, whose Aufgabe 4 asks for the encapsulated frame, the layer headers and their addressing information. 12-ibn-klausur-2016.pdf The more recent memory protocol emphasizes Dijkstra, forwarding tables and subnetting; this makes Aufgabe 3 only indirectly useful for that routing component. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

Recommended exam-preparation priority

First priority

1. **Aufgabe 4 — protocol layers, encapsulation and addressing**
2. **Aufgabe 9 — IPv4 fragmentation**

These are the most conventionally exam-like tasks on the sheet.

Second priority

3. **Aufgabe 2 — queueing-delay derivation**
4. **Aufgabe 7 — HTTP cookies**
5. **Aufgabe 1 — statistical multiplexing**, especially the general probabilistic model

Selective preparation

6. **Aufgabe 8** — learn the HTTP concepts and packet observations; reproducing the complete Wireshark lab is less important for the written exam.
7. **Aufgabe 6** — learn what very long RTTs and interruptions do to TCP, but the article-specific details are secondary.
8. **Aufgabe 3** — useful practical background, but not a substitute for practising Dijkstra, forwarding tables and subnetting.
9. **Aufgabe 5** — understand the layer-dependent scope of the attack; the specific construction is low priority.

Overall, sheet 9 contains several highly relevant networking fundamentals, but it mixes them with substantial research and laboratory work. For efficient exam preparation, **Aufgaben 4 and 9 deserve the most careful complete solutions**.

1.10 Sheet 10

Yes. Sheet 10 is one of the most exam-relevant sheets so far because it concentrates on the three topics explicitly reported in the newer exam: **subnetting, forwarding tables, and Dijkstra**. 10-ueb10-ibn-SS2025.pdf

Main syllabus-level topics

1. **Network layer and IPv4 addressing**
   - Classful and classless addressing
   - Subnetting and address allocation
   - Network and host portions of an IP address
2. **Packet forwarding**
   - Forwarding/routing tables
   - Prefix representation
   - Longest-prefix matching
3. **Routing algorithms**
   - Link-state routing and Dijkstra's algorithm
   - Distance-vector routing and the Bellman–Ford equation
   - Properties of shortest paths

These correspond closely to the IPv4, subnetting, forwarding, Dijkstra, and Bellman–Ford sections of the course summary. 01-ibn-zusammenfassung.pdf

Detailed subtopics

- IPv4 address classes A, B and C
- CIDR notation such as `/16`, `/23`, `/25`
- Network ID, subnet ID and host ID
- Subnet masks in binary and decimal notation
- Calculating the number of addresses and hosts
- Fixed-size and variable-size subnet allocation
- Dividing address blocks among networks and router links
- Translating address ranges into prefixes
- Prefix ordering and longest-prefix matching
- Constructing forwarding tables
- Traceroute and changing network routes
- Dijkstra tables: (N'), tentative distances and predecessors
- Optimal-substructure property of shortest paths
- Bellman–Ford recurrence
- Why Dijkstra requires nonnegative edge weights

Classification of the individual exercises

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
1	IPv4 classes; Whois; traceroute; real network topology	Conceptual + research + practical investigation	**Low–medium**	**Low**	Identifying address classes is relevant, but RIPE lookup and performing traceroute are mainly supplementary practical work.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
2	Subnet masks; address counts; network/host ID; subdivision of an address block	Calculation + subnet design	**Very high**	**Very high**	Direct practice for dividing networks into subnets and determining prefixes or masks.
3a	CIDR/VLSM address allocation for host networks and router links	Advanced calculation + network design	**Very high**	**High–very high**	Covers subnet allocation under capacity constraints. Slightly more intricate than a typical short exam question.
3b	Forwarding tables; route aggregation; longest-prefix matching	Table construction + application	**Very high**	**Very high**	Almost exactly the combination of subnetting and forwarding-table construction reported in the newer exam.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
4	Converting address ranges to prefixes; prefix order; longest-prefix matching	Binary calculation + routing-table reasoning	**Very high**	**Very high**	Excellent preparation for prefix manipulation and routing decisions, especially overlapping prefixes.
5	Dijkstra's shortest-path algorithm	Algorithm execution + table calculation	**Very high**	**Extremely high**	Directly matches both reference exams: execute Dijkstra and fill in the algorithm's table.
6a	Optimal substructure of shortest paths	Proof + theory	**Medium**	**Medium**	Useful theoretical justification, but less likely than executing the algorithm.
6b	Traceroute and changing routes	Conceptual application	**Low–medium**	**Low–medium**	Tests understanding that routes and routing information can change over time.

Exercise	Topic and subtopic	Type	Exam relevance	Exam-likeness	Concise assessment
6c	Bellman–Ford equation and shortest-path decomposition	Conceptual theory	**High**	**Medium–high**	Bellman–Ford appeared in the 2016 exam, although there it was used for a concrete distance-vector update.
6d	Negative edge weights and limitations of Dijkstra	Algorithmic reasoning	**Medium**	**Medium**	Important for understanding Dijkstra, but less directly reflected in the reference exams.

Evidence from the reference exams

The newer memory protocol explicitly lists, in a single routing exercise:

- performing Dijkstra,
- filling a routing/forwarding table,
- dividing a network into subnets,
- determining prefixes and subnet masks.

That makes Exercises **2**–**5** unusually strong direct preparation. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

The 2016 exam independently confirms the importance of:

- executing Dijkstra in tabular form,
- deriving forwarding entries from predecessor pointers,
- explaining and applying the Bellman–Ford equation,
- using a subnet mask to determine whether two hosts belong to the same subnet.

Thus Exercise **5** is almost an exact match, while Exercises **2–4** and **6c** cover closely related examinable skills. 12-ibn-klausur-2016.pdf

Recommended study priority

1. **Exercise 5** — exact Dijkstra exam practice
2. **Exercise 3** — strongest combined subnetting and forwarding-table exercise
3. **Exercise 2** — essential subnetting calculations
4. **Exercise 4** — prefixes and longest-prefix matching
5. **Exercise 6c** — Bellman–Ford connection
6. **Exercises 6a and 6d** — conceptual reinforcement
7. **Exercise 1 and 6b** — supplementary practical context

Overall, **Exercises 2–5 form the exam core of Sheet 10**. If time is limited, those should all be completed carefully rather than merely read.

1.11 Overview

The third column is useful, especially if it contains only the exercises with the strongest combination of **topic relevance** and **exam-like format**.

Here, "A" means *Aufgabe*. The ratings are absolute, not merely relative to the other exercises on the same sheet.

Sheet	Main syllabus topics	Most exam-like and exam-relevant exercises
1 02-ueb01-ibn-SS2025.pdf	OS foundations; CPU registers and powers of two; protection and privilege modes; interrupts and system calls; kernel stack; POSIX/Unix interfaces; shell scripting	**A2(ii)** — powers-of-two calculations (**medium–high**); **A3–4** — protection, kernel mode and kernel stack (**medium**)

Sheet	Main syllabus topics	Most exam-like and exam-relevant exercises
2 03-ueb02-ibn-SS2025.pdf	Processes and threads; PCB; `fork()` and process hierarchies; process states; `exec()`; context switches; pthreads	**A2(a)** — tracing `fork()` and drawing the process tree (**very high**); **A3** — process hierarchy and process states (**very high**)
3 11-ueb03-ibn-SS2025.pdf	Process synchronization; race conditions; critical sections; semaphores, mutexes and condition variables; producer–consumer; readers–writers	**A1** — detect and prevent a race condition (**very high**); **A2** — synchronization primitives (**high**); **A4(a–b)** — trace and correct semaphore-based synchronization (**high**)
4 04-ueb04-ibn-SS2025.pdf	Interprocess communication and pipes; I/O redirection; memory management; fragmentation and swapping; contiguous allocation; logical and physical addresses; paging basics	**A5** — core memory-management distinctions (**high**); **A6** — First/Best/Worst Fit calculation (**medium–high**); **A7** — page number and offset (**high**)
5 05-ueb05-ibn-SS2025.pdf	Paging and address translation; page tables; hierarchical and inverted page tables; TLB; address-space calculations; effective access time	**A2** — construct page-table entries from address mappings (**very high**); **A6** — direct and inverted page-table size (**very high**); **A8** — multilevel page tables (**very high**)
6 06-ueb06-ibn-SS2025.pdf	Virtual memory; page replacement; FIFO, LRU, Clock and OPT; reference bits; working sets; segmentation versus paging	**A1** — execute FIFO/LRU/Clock-style replacement (**very high**); **A4** — reference bits and LRU approximation (**high**); **A5** — working-set concept (**medium**)

Sheet	Main syllabus topics	Most exam-like and exam-relevant exercises
7 07-ueb07-ibn-SS2025.pdf	File systems; hard and symbolic links; FAT; i-nodes; directory traversal; disk performance and scheduling; RAID	**A4** — maximum file size from direct/indirect i-node pointers (**very high**); **A5** — directory traversal and disk operations (**very high**); **A2** — FAT structure (**high relevance**, but implementation format is less exam-like)
8 08-ueb08-ibn-SS2025.pdf	Deadlocks; necessary deadlock conditions; resource-allocation graphs; Banker's algorithm; CPU scheduling; starvation	**A2** — semaphore deadlock, conditions and prevention (**very high**); **A4** — resource graph, matrices and Banker's algorithm by hand (**extremely high**); **A3** — Banker's algorithm (**high relevance**, but programming format is less exam-like)
9 09-ueb09-ibn-SS2025.pdf	Network performance and delays; packet switching; protocol layers; encapsulation and addressing; TCP; HTTP and cookies; IPv4 fragmentation	**A4** — layers, encapsulation, endpoints and addressing (**extremely high**); **A9** — IPv4 fragmentation calculation (**high**); **A2** — queueing-delay calculation (**medium–high**)
10 10-ueb10-ibn-SS2025.pdf	IPv4 addressing and subnetting; CIDR and prefixes; forwarding tables; longest-prefix matching; Dijkstra; Bellman–Ford and distance-vector routing	**A2** — subnet masks and network/host IDs (**very high**); **A3** — subnet allocation and forwarding tables (**extremely high**); **A4** — prefixes and longest-prefix matching (**very high**); **A5** — Dijkstra table (**extremely high**)

The syllabus classification follows the organization and terminology of the course summary. 01-ibn-zusammenfassung.pdf

The "exam-like" judgments are based particularly on the direct matches in the 2016 exam—race conditions and semaphores, page tables, deadlocks, protocol layers, Dijkstra and Bellman–Ford—and in the newer memory protocol—`fork()`, race conditions, Banker's algorithm, FAT/i-nodes, paging, subnetting, forwarding tables and Dijkstra. 12-ibn-klausur-2016.pdf 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

Strongest sheets overall

If you prioritize entire sheets rather than individual exercises:

1. **Sheet 10** — subnetting, forwarding tables and Dijkstra
2. **Sheet 8** — deadlocks, resource graphs and Banker's algorithm
3. **Sheet 5** — paging and page tables
4. **Sheet 3** — race conditions and synchronization
5. **Sheet 2** — `fork()` and process states
6. **Sheet 7** — FAT and i-nodes
7. **Sheet 9** — network layers and addressing

Sheets **1, 4 and 6** still contain useful exam material, but their exercises have fewer direct matches with the two reference exams.

2 Plan

Yes—this is still realistic, especially because you solved the material before. You are **reactivating knowledge**, not learning the entire course from zero.

The crucial change is: **do not redo all ten sheets sequentially**. They contain research, programming, and practical tasks that are useful pedagogically but inefficient for a written exam. The sheets collapse into about seven examinable clusters.

2.1 Priority hierarchy

Priority 1: direct reference-exam matches

These should become automatic:

- `fork()`, process trees and process states
- Race conditions, semaphores and synchronization
- Deadlocks, resource-allocation graphs and Banker's algorithm
- Paging, page-table calculations and address translation
- FAT and i-nodes
- Protocol layers, encapsulation and addressing
- Subnetting, prefixes and forwarding tables
- Dijkstra and Bellman–Ford

Priority 2: likely supporting material

- Page replacement: FIFO, LRU and Clock
- TLB and reference bits
- CPU scheduling
- IP fragmentation
- Disk scheduling and RAID
- Memory fragmentation and allocation algorithms

Priority 3: postpone unless time remains

- Long programming exercises
- Internet research and API-detail questions
- Traceroute, Whois and Wireshark experiments
- Historical hardware details
- Learning-methodology questions
- Reimplementing FAT, Banker, LRU or producer–consumer in code

For programming exercises, understand the underlying algorithm and execute it **by hand**. Do not spend hours coding it again.

2.2 Ten-day core plan

Assume approximately 4–6 focused hours per day. If you have 11–13 days, use the extra days for consolidation rather than expanding the syllabus.

Day	Focus	Essential exercises and outcomes
1	Diagnostic + processes	Attempt the 2016 exam under time pressure, even if incomplete. Then review Sheet 2 A2(a), A3. Know `fork()` return values, process trees and the five process states.
2	Synchronization	Sheet 3 A1, A2 and A4(a–b). Trace a race condition instruction by instruction; repair it with semaphores; understand `wait`, `signal`, mutexes and condition variables.
3	Paging fundamentals	Sheet 4 A5, A7; Sheet 5 A2, A6. Split addresses into page number/offset, construct page-table entries, and calculate normal/inverted page-table sizes.

Day	Focus	Essential exercises and outcomes
4	Advanced paging and virtual memory	Sheet 5 A8; Sheet 6 A1, A4. Multilevel tables, TLB, FIFO/LRU/Clock and reference-bit approximation.
5	Deadlocks and scheduling	Sheet 8 A2 and A4 first; A6 if time remains. Know the four deadlock conditions, resource graphs, Banker matrices and hand execution.
6	File systems	Sheet 7 A4 and A5; review the FAT mechanism from A2 without implementing it. Practise i-node maximum-file-size and pathname traversal calculations.
7	Network stack	Sheet 9 A4 and A9. Reconstruct encapsulation, endpoints and addressing at each layer; then practise IP fragmentation.
8	Subnetting and forwarding	Sheet 10 A2, A3 and A4. Work entirely from scratch: masks, address counts, network/host IDs, prefixes and longest-prefix matching.
9	Routing algorithms	Sheet 10 A5 and A6(c); 2016 exam Question 5. Complete Dijkstra tables, derive next hops from predecessor pointers, and perform Bellman–Ford updates.

Day	Focus	Essential exercises and outcomes
10	Timed mixed mock + repair	Assemble unused exercise parts into a 90-minute mock. Mark it immediately. Redo every incorrect question from a blank page without consulting the solution.

2.3 If you have 11–13 days

Use the additional days as follows:

- **Day 11:** Repair your weakest two topics.
- **Day 12:** Second full timed mock covering all major clusters.
- **Day 13:** Light closed-book recall, formulas and common mistakes. No major new topics.

If you have only ten days, the core plan already covers the important material. Do not sacrifice mock-exam practice to squeeze in every supplementary topic.

2.4 How to structure each study day

A good five-hour day would look like this:

1. **30 minutes: closed-book retrieval**
    - Write everything remembered from yesterday.
    - Reproduce algorithms, definitions and calculation procedures.
2. **60–75 minutes: targeted theory recovery**
    - Use the summary only to repair what you could not recall.
    - Avoid reading an entire chapter passively.
3. **Two 75-minute problem blocks**
    - Solve selected exercises without notes.
    - Work in a handwritten, exam-style format.
    - Impose time limits.
4. **30 minutes: correction and error log**

- Record each substantive mistake.
- Classify its cause: forgotten fact, wrong method, arithmetic error or misreading.

5. **20–30 minutes: delayed repetition**
    - Reattempt one problem from two or three days earlier.

If you have only three hours, keep the same structure but shorten the theory section—not the problem-solving section.

2.5 Build an error log, not another long summary

Use five columns:

Topic	Mistake	Cause	Correct rule/method	Retest
Subnetting	Used 2^h as usable-host count	Forgot network/broadcast addresses	Normally $2^h - 2$, unless stated otherwise	Tomorrow
Dijkstra	Updated an already-finalized node	Algorithm rule forgotten	Nodes in N' are final	In two days

Your final revision should be driven primarily by this list.

2.6 Learn reusable solution templates

For each major exam type, have a standard procedure:

- **`fork()`:** draw one branch per call; label parent and child return values.
- **Race condition:** expand operations into load–modify–store and show the interleaving.
- **Banker:** calculate available resources, find a satisfiable process, release its allocation, repeat.
- **Paging:** derive offset bits from page size, then split the logical address.
- **I-node:** calculate pointers per indirect block before calculating capacity.
- **Subnetting:** determine required host bits, then prefix length, block size and aligned boundaries.
- **Longest-prefix matching:** list all matching prefixes and choose the longest.
- **Dijkstra:** maintain N', $D(v)$ and $p(v)$ systematically.
- **Bellman–Ford:** minimize link cost plus the neighbour's advertised distance.

2.7 The most important rule

Do not measure progress by “pages read” or “sheets reviewed.” Measure it by:

> Can I solve this problem from a blank page, correctly and within exam time?

After about five days, some topics will already feel substantially more familiar. That is the advantage of having solved the sheets before. A focused ten-day recovery is much more plausible than the current breadth of the material makes it appear.

3 Processes and `fork()`

3.1 Unit 1: `fork()`, process trees, and process states

This is a particularly high-priority unit: the newer exam contained code using `fork()` and asked for the number of processes, return values, executed branches, and the five process states. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

1. What is a process?

A **program** is passive executable code stored in a file. A **process** is an instance of a program currently being executed.

A process includes:

- its program code;
- stack, heap and global data;
- CPU state, such as the program counter and registers;
- operating-system information stored in its **Process Control Block (PCB)**:
  - process ID;
  - current process state;
  - scheduling information;
  - memory-management information;
  - open resources.

Multiple processes can therefore execute the same program while having different variables, stacks, registers and process IDs.

3.2 2. What does `fork()` do?

In a POSIX system,

```
pid_t p = fork();
```

creates **one new child process**.

Immediately after a successful call, there are two processes:

- the original **parent process**;
- the newly created **child process**.

Both continue execution at the instruction immediately following `fork()`.

Conceptually, the child receives a copy of the parent's state at the moment of the call:

- program code;
- stack, heap and global variables;
- environment;
- open file descriptors;
- current position in the program.

Ordinary variables are subsequently independent. Changing a variable in the child does not change the parent's corresponding variable.

In practice, the memory pages are usually initially shared using **copy-on-write** and copied only when one process modifies them. For the exam, however, you can reason as if each process has its own copy.

Return value

The processes distinguish themselves using the return value:

Result of `fork()`	Meaning
`p > 0`	This is the parent; `p` is the child's PID
`p == 0`	This is the child
`p == -1`	Creation failed; no child was created

A very common pattern is therefore:

```c
pid_t p = fork();

if (p == 0) {
    // Executed by the child
} else if (p > 0) {
    // Executed by the parent
} else {
    // Error handling
}
```

The exact positive PID is normally unknown unless it is provided.

Crucial distinction

`fork()` is called once, but it **returns twice**:

- once in the parent, with a positive value;
- once in the child, with zero.

3.3 3. Counting processes

Every process that reaches a successful `fork()` creates one additional process.

Thus, if there are (k) processes immediately before a `fork()` and all (k) execute it successfully, there will be

$$2k$$

processes afterwards.

For n successive unconditional calls,

```c
fork();
fork();
fork();
```

the total number is

$$2^n.$$

Here, three calls produce $2^3 = 8$ processes, including the original process.

This shortcut works only when **every existing process executes every call**. With conditions, loops, `exit()`, or `exec()`, you must trace the individual execution paths.

3.4 4. The safe process-tree method

For each `fork()`:

1. Identify every process that reaches the call.
2. Give each of them one new child.
3. Copy the parent's current variable values into the child.
4. Assign the appropriate return value:
   - parent receives a positive PID;
   - child receives zero.
5. Continue tracing every resulting process separately.

Do not count the number of occurrences of the word `fork` and immediately use 2^n. A conditional call may be reached by only part of the process tree.

Example

```
pid_t p = fork();

if (p == 0)
    fork();
```

After the first call:

- parent: `p > 0`;
- child: `p == 0`.

Only the child enters the `if` and executes the second call. Consequently, there are three processes, not four.

3.5 5. Process hierarchy

Each child has:

- its own PID;
- the PID of its parent, called its PPID.

This produces a process tree. A child created later by an existing child becomes a **grandchild** of the original process.

The parent and child normally execute concurrently. The scheduler may run either one first, so the order of their output is generally not predictable.

However, the order of instructions **inside one process** remains fixed.

3.6 6. The five basic process states

The five-state model required by the course is: 01-ibn-zusammenfassung.pdf

State	Meaning
New	The process is being created.
Ready	It can execute but is waiting for a CPU.
Running	Its instructions are currently being executed by a CPU.
Waiting / Blocked	It cannot continue until some event occurs, such as completion of I/O.
Terminated	It has finished execution or has been terminated.

The important distinction is:

- **Ready:** capable of running, but no CPU currently assigned;
- **Waiting:** not capable of continuing yet, even if a CPU were available.

State transitions

Transition	Typical cause
New → Ready	The process is admitted into the ready queue
Ready → Running	The scheduler dispatches it
Running → Ready	Time slice expires or the process is preempted
Running → Waiting	It requests I/O or waits for an event
Waiting → Ready	I/O completes or the awaited event occurs
Running → Terminated	It finishes or calls `exit()`

Normally, a process does not move directly from Waiting to Running. Once its event occurs, it becomes Ready and waits to be selected by the scheduler.

Where does `fork()` fit?

A parent must be **Running** to execute `fork()`. The operating system creates a child, which passes through New and becomes Ready. The scheduler then decides whether the parent or the child runs next.

Calling `fork()` does **not** guarantee that the child executes first.

3.7 7. Common exam traps

- Counting only the children and forgetting the original parent.
- Assuming that only the child continues after `fork()`.
- Reversing the return values: the child gets `0`; the parent gets the child's PID.
- Assuming the parent always runs before the child.
- Forgetting that descendants inherit the current value of previously assigned variables.
- Using (2^n) even though some calls are conditional.
- Confusing Ready with Waiting.
- Claiming that I/O completion causes Waiting → Running directly.

Sheet 2 contains the corresponding course practice involving return values, conditional calls and process trees. 03-ueb02-ibn-SS2025.pdf

4 Exam-like exercise

Try this closed-book and give yourself approximately **15 minutes**.

Assume that every `fork()` succeeds.

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    int x = 0;

    pid_t p = fork();             // F1

    if (p == 0) {
        x = 1;
        fork();                   // F2
    } else {
        x = 2;
        fork();                   // F3
    }

    if (p == 0) {
        x = x + 10;
        fork();                   // F4
    }

    printf("p-category=%s, x=%d\n",
           p == 0 ? "zero" : "positive", x);

    return 0;
}
```

Part A: `fork()` and the process tree

1. How many processes exist at the end, including the original process?

2. How many times is `printf()` executed?
3. Draw the process tree. Label the original process (P_0), and label each new process according to the call that created it.
4. How many final processes have:
   - `p == 0`;
   - `p > 0`?
5. Which processes execute:
   - the first `if` branch;
   - the `else` branch;
   - the second `if` branch?
6. What is the final value of `x` in each process?
7. Is the order of the printed lines uniquely determined? Explain briefly.
8. Can the exact positive value of `p` be determined from the code alone? What does that value represent?

A table may make your tracing easier:

Final process	Parent	Created by	`p` category	Final `x`	Executes second `if`?
P_0	—	—			
…					

Part B: Process states

Give the state transition caused by each event:

1. A process has just been admitted by the operating system and may now compete for the CPU.
2. The scheduler selects it for execution.
3. Its time slice expires.
4. It is selected again and then requests data from the disk.
5. The disk operation finishes.
6. The scheduler selects it again.
7. The process calls `exit()`.

Write the answer in the form:

$$\text{Ready} \rightarrow \text{Running}$$

Send me your solution—even if parts are uncertain—and I'll correct it step by step before we move to race conditions and synchronization.

4.1 Answers

Part A

1. there are 6 processes
2. each process executes printf once, therefore 6 times

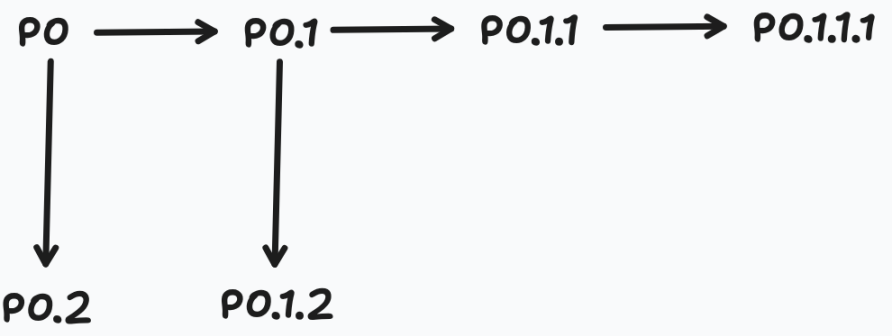


Figure 4.1: process-tree

3.
4. P0, P0.2 have `p > 0`, i.e. 2 processes have positive `p`, and P0.1, P.0.1.1, P0.1.1.1, P0.1.2 have `p == 0`, i.e. 4 have zero `p`.
5. first if branch executed by: P0.1
   else branch: P0
   second if branch: P0.1, P0.1.1
6. final x value of each process: P0: 2
   P0.1: 11
   P0.1.1: 11
   P0.1.1.1: 11
   P0.2 : 2
   P0.1.2: 11
7. no, because the scheduler arbitrarily decides in which order the parent and children processes are scheduled
8. No, a positive `p` value represents the PID of the generated child process and is internally determined by the OS

Part B

1. new -> ready
2. ready -> running
3. running -> ready
4. running -> waiting
5. waiting -> ready
6. ready -> running
7. running -> terminated

5 Race conditions, semaphores and synchronization

This is one of the highest-priority units. The 2016 exam devoted an entire 11-point problem to instruction-level race tracing, semaphore repair, semaphore semantics and execution ordering. 12-ibn-klausur-2016.pdf The recent exam protocol also reports an independent pseudocode-based race-condition problem. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

The coverage below follows Day 2 of your plan and the selected Sheet 3 exercises. 15-Operating-Systems-and-Networks-SoSe-25-Notes.pdf 11-ueb03-ibn-SS2025.pdf

5.1 Why synchronization is necessary

Threads belonging to the same process have:

- separate registers and stacks;
- a shared address space containing global variables, heap objects and other shared resources.

Sharing makes communication inexpensive, but it allows multiple threads to access the same data concurrently.

Consider:

```
counter = counter + 1
```

Although this looks like one operation, it normally consists conceptually of:

```
LOAD  counter into register R
ADD   1 to R
STORE R into counter
```

A thread can be interrupted between any two of these instructions.

Suppose `counter = 0` and two threads both increment it:

Step	Thread A	Thread B	Shared `counter`
1	Load `counter` into `RA` (`RA=0`)		0
2		Load `counter` into `RB` (`RB=0`)	0
3	Add 1 to `RA` (`RA=1`)		0
4	Store `RA`		1
5		Add 1 to `RB` (`RB=1`)	1
6		Store `RB`	1

The correct result would be 2, but the final result is 1. One increment has been overwritten: a **lost update**.

This can happen on a single-core CPU. The threads do not need to execute physically at the same instant; preemption and interleaving are enough.

5.2 Race conditions

A **race condition** exists when the correctness or result of a program unintentionally depends on the timing or execution order of concurrent operations.

A typical race condition requires:

1. shared state;
2. concurrent accesses to that state;
3. at least one access that modifies it;
4. insufficient synchronization.

The output may differ between executions because the scheduler can produce different interleavings.

Critical region

A **critical region** or **critical section** is a part of a program that accesses shared data and must not be executed simultaneously with a conflicting critical region.

```
entry protocol

critical region

exit protocol

non-critical region
```

The entry and exit protocols are implemented using a mutex, semaphore or another synchronization primitive.

Requirements of a good mutual-exclusion solution

A correct solution should provide:

- **Mutual exclusion:** conflicting threads are never simultaneously inside their critical regions.
- **Progress:** a thread outside its critical region must not unnecessarily prevent others from entering.
- **No timing assumptions:** correctness must not depend on CPU speed or the number of cores.
- **No starvation/bounded waiting:** a thread requesting entry should not be postponed forever.

The first is a safety property: "nothing bad happens." Progress and absence of starvation are liveness properties: "something good eventually happens."

5.3 Atomicity

An operation is **atomic** if other threads cannot observe or interrupt a partially completed execution of that operation.

For example, the abstract operation

```
wait(S)
```

must itself be atomic. If two threads could simultaneously test and decrement the semaphore non-atomically, the semaphore would not provide synchronization.

Protecting a critical region does not mean that all its ordinary instructions become individually atomic. It means that the entry protocol ensures that only the permitted thread or threads can execute that region.

5.4 Semaphores

A semaphore contains:

- a nonnegative count representing available permits or resource instances;
- an associated queue of blocked threads;

- two atomic operations, conventionally called `wait` and `signal`.

Other common names are:

Operation	Alternative names
`wait(S)`	`P(S)`, `down(S)`, `sem_wait(S)`
`signal(S)`	`V(S)`, `up(S)`, `post(S)`, `sem_post(S)`

`wait(S)`

Conceptually:

```
if a permit is available:
    consume one permit
    continue
else:
    block until a permit becomes available
```

Thus, `wait` may block.

`signal(S)`

Conceptually:

```
make one permit available
if a thread is waiting:
    wake one waiting thread
```

`signal` does not normally block.

The implementation may represent blocked threads using negative values or separate queues, but exam solutions should normally describe the abstract behaviour rather than depend on a particular representation.

Binary and counting semaphores

A **binary semaphore** has at most one available permit:

```
Semaphore mutex = 1
```

A **counting semaphore** can represent several identical resource instances:

```
Semaphore printers = 3
```

Up to three successful `wait(printers)` operations can occur before another thread must release a permit.

5.5 Mutual exclusion with a semaphore

Initialize the semaphore to 1:

```
Semaphore mutex = 1
```

Every thread uses:

```
wait(mutex)

    // critical region

signal(mutex)
```

The first thread consumes the only permit. Other threads calling `wait(mutex)` must block until the first thread calls `signal(mutex)`.

The protected operation must lie entirely between `wait` and `signal`:

```
wait(mutex)
counter = counter + 1
signal(mutex)
```

Protecting only the load or only the store would still allow an unsafe interleaving.

Mutex versus binary semaphore

At the level required by many introductory exercises, a mutex can be viewed as a binary lock. More precisely:

- a mutex normally has an owner;
- only the thread that locked it should unlock it;
- a semaphore represents permits and does not necessarily have ownership.

Semaphores can consequently express both mutual exclusion and dependencies between different threads.

5.6 Enforcing an execution order

Suppose block `X` in Thread 1 may execute only after block `Y` in Thread 2.

Initialize a semaphore to zero:

```
Semaphore finished_Y = 0
```

Thread 1:

```
wait(finished_Y)
X
```

Thread 2:

```
Y
signal(finished_Y)
```

Why initialize it to zero?

- If Thread 1 reaches `wait` first, it blocks.
- After Thread 2 completes `Y`, `signal` provides the permit.
- If Thread 2 finishes first, the semaphore retains the permit. Thread 1's later `wait` consumes it and continues immediately.

This is what is meant by saying that a semaphore has **memory**: an earlier signal can be remembered in its counter.

5.7 Blocking versus busy waiting

A primitive can wait in two ways.

Busy waiting

```
while lock is unavailable:
    repeatedly test lock
```

The thread remains runnable and consumes CPU time. A lock implemented this way is often called a **spinlock**.

Busy waiting can be reasonable for extremely short waits inside kernels or multiprocessor code, but it is inefficient for long waits.

Blocking

If the resource is unavailable:

1. the thread is placed in a waiting queue;
2. its state becomes `Waiting/Blocked`;
3. another runnable thread receives the CPU;
4. a later `signal` makes the blocked thread `Ready`.

Notice that waking a thread normally produces:

```
Waiting → Ready
```

not immediately `Waiting → Running`.

5.8 Condition variables

A condition variable allows a thread to sleep until some condition involving shared state may have become true.

It must be used together with:

- a **predicate**, such as `count > 0` or `production_done`;
- a mutex protecting that predicate;
- the condition variable on which threads wait.

Canonical pattern:

```
lock(mutex)

while (!condition) {
    cond_wait(condition_variable, mutex)
}

// condition is true; access shared state

unlock(mutex)
```

`cond_wait(cv, mutex)` performs three important actions:

1. atomically releases `mutex` and puts the caller to sleep;
2. waits for a notification;
3. reacquires `mutex` before returning.

The atomic release-and-sleep step prevents the following lost-wakeup race:

1. waiter checks that the condition is false;
2. notifier makes it true and sends a signal;
3. waiter goes to sleep after the signal;
4. no future signal arrives.

Why `while`, not `if`?

After waking, the thread must check the predicate again because:

- wakeups may be spurious;
- another awakened thread may acquire the mutex first and consume the resource;
- a notification means "the condition may now be true," not "the condition is guaranteed to remain true."

Correct:

```
while (count == 0)
    cond_wait(not_empty, mutex)
```

Unsafe:

```
if (count == 0)
    cond_wait(not_empty, mutex)
```

`signal` versus `broadcast`

- `cond_signal(cv)` wakes at least one waiting thread.
- `cond_broadcast(cv)` wakes all threads waiting on `cv`.

A producer adding one item normally needs to wake only one consumer. If `production_done` becomes true and every consumer must notice it and terminate, a broadcast is appropriate.

The awakened threads do not all enter the critical region simultaneously: each must first reacquire the mutex.

Condition variables do not remember notifications

If `cond_signal` or `cond_broadcast` is called while nobody is waiting, the notification disappears.

This differs from a semaphore:

```
signal(S)
```

with no waiter increments or preserves the semaphore's permit count, so a later `wait(S)` can consume it.

A mutex remembers whether it is currently locked, but an `unlock` is not a stored notification that a future thread can consume.

Why update and signal under the mutex?

The standard protocol is:

```
lock(mutex)
done = true
cond_broadcast(done_changed)
unlock(mutex)
```

The mutex makes changing the predicate and notifying waiters part of one synchronized protocol. A waiter cannot slip between checking `done` and beginning to wait. When it wakes and reacquires the mutex, it sees a consistent value of `done`.

Example: producer-consumer queue

- producers create items / jobs and insert them into a shared queue.
- consumers remove items
- a consumer cannot remove an item while the queue is empty
- instead of repeatedly checking the queue in a busy loop, it sleeps on `not_empty`

Concrete Example: downloading and playing a video

The producer downloads video chunks:

```
network -> buffer
```

The consume decodes and siplays them

```
buffer -> video player
```

1. the downloader puts some chunks into the buffer
2. the player begins consuming them
3. both continue concurrently

Other canonical examples:

- a network interface receives packets, while an application processes them
- a web server accepts requests while worker threads handle earlier requests
- one thread reads files, while another processes their contents.
- keyboard input arriving while the application processe earlier input

The general pattern is that

> a buffer crates a pipleline and absorbs temporary differences in speed between producers and consumers

Benefits of concurrent production and consumption

1. **lower latency**: the consumer can start as soon as the first item exists instead of waiting for all items
2. **higher throughput**: production and consumption may overlap
    1. one core produces while another consumes
    2. a network or disk operation runs while the cpu processes previous data

3. **bounded memory use**: for a large or infinite stream, we may use a bounded buffer:

```
producer -> fixed-size buffer -> consumer
```

A bounded buffer typically uses two predicates and two condition variables:

```
std::condition_variable not_empty; // consumers wait here
std::condition_variable not_full; // producers wait here
```

Complete producer-consumer program

```
shared Queue jobs
shared Mutex mutex
shared ConditionVariable not_empty
shared Boolean production_done = false

function producer():
    for number from 1 to 10:
        job = create_job(number)

        lock(mutex) //begin critical region 1

        jobs.push(job)
        signal(not_empty)

        unlock(mutex)

        // end critical region 1

    // no more jobs will be produced
    lock(mutex) // begin critical region 2

    production_done = true
    broadcast(not_empty)

    unlock(mutex) // end cr 2

function consumer(worker_id):
    while true:
        lock(mutex)

        while jobs.empty() and not production_done:
```

```
            wait(not_empty, mutex)

        if jobs.empty() and production_done:
            unlock(mutex)
            return

        job = jobs.pop()

        unlock(mutex)

        // processing is done outside of critical region
        process(job)

function main():
    producer_thread = start_thread(producer)
    consumer_thread_1 = start_thread(consumer, 1)
    consumer_thread_2 = start_thread(consumer, 2)

    join(producer_thread)
    join(consumer_thread_1)
    join(consumer_thread_2)

    print("all jobs completed")
```

Bounded-buffer producer-consumer problem

uses a shared queue with a **fixed maximum capacity**

- consumers wait on `not_empty` when the buffer is empty
- producers wait on `not_full` when the buffer is full
- a mutex protexts the buffer and related shared state

The bounded buffer provides **backpressure**: if consumers cannot keep up, producers are forced to wait instead of allowing the queue to grow indefiniely.

Practical Example

Consider a program that reads large imagems from disk and processes them:

- the producer reads images and places them in memory
- several consumers resize / transform or analyze the images

- each image may occupy considerable memory

An unbounded queue could consumer all abailable memory if disk reading is temporarily faster than image processing. A buffer limited to, say, five images ensures that at most five unprocessed images are held in memory

Complete typed pseudocode

```
// shared state

constant BUFFER_CAPACITY = 5
constant NUMBER_OF_JOBS = 20
constant NUMBER_OF_CONSUMERS = 2

shared Queue<Job> jobs

shared Mutex mutex
shared ConditionVarialbe not_empty
shared ConditionVarialbe not_full

shared production_done = false


// producer
function producer() -> Void:
    for job_number from 1 to NUMBER_OF_JOBS:
        job = new Job(job_number)

        lock(mutex)

        while jobs.size() == BUFFER_CAPACITY:
            wait(not_full, mutex)

        // buffer is not full
        jobs.push(job)

        // buffer is not empty
        signal(not_empty)

        unlock(mutex)
```

```
    lock(mutex)

    production_done = true
    // every sleeping consumer must discover that production is finished
    broadcast(not_empty)

    unlock(mutex)

// consumer

function consumer() -> Void:
    while true:
        lock(mutex)

        while jobs.empty() and not production_done:
            wait(not_empty, mutex)

        // no buffered jobs remain, and no new jobs will arrive
        if jobs.empty() and production_done:
            unlock(mutex)
            return

        // buffer not empty
        job = jobs.pop()

        // at least one producer may now be able to proceed
        signal(not_full)

        unlock(mutex)

        process(job)


// main program

function main() -> Void:
    consumers = new Array<Thread>(NUMBER_OF_CONSUMERS)

    for i from 0 to NUMBER_OF_CONSUMERS - 1:
        consumers[i] = start_thread(consumer)

    producer_thread = start_thread(producer)
```

```
    join(producer_thread)

    for i in [1, NUMBER_OF_CONSUMERS]:
        join(consumers[i])

    print("all jobs completed")
```

join()

`join()` does not end the thread. It makes the calling thread wait until another thread has finished.

```
consumer_thread_1 = start_thread(consume, 1)

...

join(consumer_thread_1)
```

1. start a new htread executing `consumer(1)`
2. continue executing the main thread
3. When `join(consumer_thread_1)` is reached, pause the main thread until `consumer_thread_1` terminates

This guarantees that the main thread will not prematurely terminate before `consumer_thread_1` does, which would be possible because the scheduler can decide to execute the return or the final statement of main, ending the main thread, before `consumer_thread_1` is finished

5.9 Producer–consumer pattern

For a bounded buffer of capacity (N), a semaphore solution uses:

```
Semaphore empty = N
Semaphore full  = 0
Semaphore mutex = 1
```

Producer:

```
produce(item)

wait(empty)
wait(mutex)
insert(item)
signal(mutex)
signal(full)
```

Consumer:

```
wait(full)
wait(mutex)
item = remove()
signal(mutex)
signal(empty)

consume(item)
```

Interpretation:

- `empty` counts free slots;
- `full` counts stored items;
- `mutex` protects the buffer data structure.

Do not acquire `mutex` before waiting for `empty` or `full`. Otherwise a thread might sleep while holding the buffer lock, preventing the other side from changing the condition it is waiting for.

5.10 Simplified read–write lock

The reader-writer problem arises whenever many threads access the same shared data, where:

- readers only inspect the data and may safely run concurrently
- writers modify the data nad therefore need exclusive access

Example 5.1 (In-memory cache)**.** A web server may maintain a shared cache

- readers: look up cached pages or database results
- writers:
  - insert new entries
  - remove expoired entries
  - refresh existing entries

Naive Solution

Suppose:

```
Semaphore S = N
```

A reader consumes one permit:

```
lock_read:
    wait(S)

unlock_read:
    signal(S)
```

A writer attempts to consume all (N) permits:

```
lock_write:
    repeat N times:
        wait(S)

unlock_write:
    repeat N times:
        signal(S)
```

This appears to permit either:

- up to (N) simultaneous readers; or
- one writer holding every permit.

But `lock_write` is not atomic.

For (N=3), two writers could execute:

```
W1: wait(S)   // S: 3 → 2
W2: wait(S)   // S: 2 → 1
W1: wait(S)   // S: 1 → 0
W2: wait(S)   // blocks
W1: wait(S)   // blocks
```

Neither writer can acquire all three permits, and neither can release its partial acquisition because it has not entered the critical region. This is a **deadlock**.

A simple repair serializes writers while they collect the permits:

```
Semaphore S = N
Semaphore writer_gate = 1

lock_write:
    wait(writer_gate)
    repeat N times:
        wait(S)
    signal(writer_gate)

unlock_write:
    repeat N times:
        signal(S)
```

Only one writer may now attempt the multi-step acquisition.

This repair prevents the writer–writer split deadlock, but it does not necessarily guarantee fairness: continuously arriving readers might still starve a writer.

One correction to retain from your earlier work: your old Sheet 3 answer identified the interleaved writer acquisition, but described it as permitting simultaneous writing. The direct consequence is instead the deadlock shown above. 14-Operating-Systems-and-Networs-SoSe-25-Solutions.pdf

Solving fairness Problem

First approach: sared entrance gate

```
Semaphore S = N
Semaphore entry_gate = 1

// reader
lock_read:
    wait(entry_gate)
    wait(S)
    signal(entry_gate)

unlock_read:
    signal(S)


// writer
```

```
lock_write:
    wait(entry_gate)
    repeat N times:
        wait(S)

unlock_write:
    repeat N times:
        signal(S)
    signal(entry_gate)
```

Here, once a writer acquires `entry_gate`

- no new reader can beign acquiring a permit
- existing readers may finish and release their permits
- the writer eventually collects all N permits
- the writer holds both the entrance gate and all permits during its critical region

But the fairness is still not guaranteed, if the semaphore is completely unfair. It is still possible for newly arriving readers repaetedly to acquire the gate before hte writer:

```
reader 1 releases gate
reader 2 immediately acquires it
reader 2 releases gate
reader 3 immediately acquires it
```

The the writer could still starve. But with a **fair / FIFO** semaphore

```
waiting order
    writer -> reader 2 -> reader 3
```

new readers cannot overtake the writer.

> **Note**
>
> A fair / FIFO alone would solve the starvation problem but the writer could still wait very long, because its wait requests in the loop could be often intersparsed with the requests of readers continiously arriving

5.11 Common exam traps

- Treating `counter++` as an indivisible operation.
- Thinking a race requires multiple CPU cores.
- Protecting different threads with different mutexes.
- Forgetting to initialize a mutex semaphore to 1.
- Initializing an ordering semaphore to 1 instead of 0.
- Forgetting that semaphore operations must themselves be atomic.
- Omitting `signal` on some control-flow path.
- Sleeping while holding a mutex needed by the thread that could wake you.
- Using `if` instead of `while` around `cond_wait`.
- Assuming condition-variable notifications are remembered.
- Confusing deadlock with starvation.
- Calling every timing-dependent output a race: nondeterministic ordering can be intentional and harmless.

5.12 Exam-like exercises

Do these closed-book. Allow approximately 35–45 minutes in total.

Exercise 1: Race condition and semaphore repair

Two threads share:

```
integer counter = 0
```

Thread A:

```
repeat 3 times:
    counter = counter + 1
```

Thread B:

```
repeat 2 times:
    counter = counter + 1
```

Assume a single-core system and that an increment consists of:

```
LOAD counter into private register R
ADD 1 to R
STORE R into counter
```

Questions

1. Define the term **race condition**.
2. What should the final value of `counter` be?
3. Give an instruction-level interleaving in which one update is lost.
4. Explain why the race can occur despite there being only one CPU core.
5. Repair the program using one semaphore. State its initial value.
6. Explain completely but concisely what `wait` and `signal` do.
7. Would it be sufficient for A and B to use two different mutexes? Explain.

Use this table for Part 3:

Step	Thread	Instruction	Private register	Shared `counter`
1				
2				

Exercise 2: Ordering and condition variables

Part A: Ordering

Block `send_result` in Thread A may execute only after Thread B has completed `calculate_result`.

Write the simplest semaphore-based pseudocode for both threads. Give the semaphore's initial value.

Your solution must work regardless of which thread starts first.

Part B: Find the condition-variable errors

A consumer uses:

```
lock(mutex)

if (count == 0)
    cond_wait(not_empty, mutex)

item = remove_from_buffer()
count = count - 1

unlock(mutex)
```

The producer uses:

```
add_to_buffer(item)
count = count + 1
cond_signal(not_empty)
```

Answer:

1. Identify at least two synchronization errors.
2. Rewrite the relevant consumer and producer pseudocode correctly.
3. Why must the consumer test the condition in a `while` loop?
4. What happens if `cond_signal(not_empty)` is called while nobody is waiting?
5. What would happen in the analogous situation with `signal(S)` on a semaphore?
6. When would `broadcast` be preferable to `signal`?

Exercise 3: Debugging a semaphore construction

Let (N=4):

```
Semaphore S = 4

lock_read:
    wait(S)

unlock_read:
    signal(S)

lock_write:
    for i = 1 to 4:
        wait(S)
```

```
unlock_write:
    for i = 1 to 4:
        signal(S)
```

1. Give an exact execution involving two writers that leads to failure.
2. Classify the failure as:
    - mutual-exclusion violation,
    - deadlock, or
    - starvation.
3. Explain precisely why neither writer can continue.
4. Repair the construction using one additional semaphore or mutex.
5. Does your repair also guarantee that a writer can never starve? Explain briefly.

5.13 Answers

Exercise 1

1. a race condition occurs when the reseult of a program depends on the timing or interleaving of unsynchronized accesses to shared state, with at least one acccess modifying that state, so that some interleavings can produce an incorrect result
2. 5
3. table:

step	Thread	Instruction	private register	shared `counter`
1	A	load counter RA	RA == 0	0
2	A	add 1 RA	RA == 1	0
3	A	store RA counter	RA == 1	1
4	B	load counter RB	RB == 1	1
5	B	add 1 RB	RB == 2	1
6	B	store RB counter	RB == 2	2
7	A	load counter RA	RA == 2	2
8	A	add 1 RA	RA == 3	2
9	A	store RA counter	RA == 3	3
10	A	load counter RA	RA == 3	3
11	A	add 1 RA	RA == 4	3

step	Thread	Instruction	private register	shared `counter`
12	B	load counter RB	RB == 3	3
13	B	add 1 RB	RB == 4	3
14	B	store RB counter	RB == 4	4
15	A	store RA counter	RA == 4	4

4. A race condition does not require physical parallelism. On a single-core CPU, the scheduler may preempt one thread between its `LOAD` and `STORE` instruction and run the other thread. The resulting interleaving allows both treads to operate on the same stale value

5. code:

```
semaphore mutex = 1

thread A:
 repeat 3 times:
     wait(mutex)
     counter = counter + 1
     signal(mutex)
thread B:
 repeat 2 times:
     wait(mutex)
     counter = counter + 1
     signal(mutex)
```

6. answer:

   - wait: atomically decrements the value of a semaphore if it is possitive, otherwise blocks the calling thread and places it in the waiting queue associated with the semaphore
   - signal: atomically increments the value of the semaphore if there are no waiting threads in the queue, or if the queue is not empty it wakes up a waiting thread

7. No, if thread A is holding mutexA and executing its critical region a thread B could still lock mutexB, because it is not held by thread A, and thus enter the critical region. This would mean both threads are executing in their conflicting critical regions.

Exercise 2

Part A

```
Semaphore result_ready = 0

Thread A:
    wait(result_ready)
    send_result()

Thread B:
    calculate_result()
    signal(result_ready)
```

Part B

1. answer:
    - Error 1: `cond_wait(not_empty, mutex)` should be checked inside a while loop and not an if statement
    - Error 2: in producer the access to shared variables must be protected with a mutex lock
2. correction:

```
// consumer:
lock(mutex)

while (count == 0) :
    cond_wait(not_empty, mutex)

item = pop(buffer)
count = count - 1

unlock(mutex)

// producer:
lock(mutex)
push(buffer, item)
count = count + 1
cond_signal(not_empty)
unlock(mutex)
```

3. after a producer pushes an item into the buffer and sends a signal to `not_empty`, it doesn't necessarily mean that a sleeping thread will immediately run. First it becomes ready to run:

   ```
   blocked -> ready
   ```

   scheduler may schedule it at an arbitrary point in time. Meanwhile another consumer previously in ready state might be scheduled and already consumed the item. Which means that the buffer may be empty again. But if we have an if-statement instead of a while loop, after `cond_wait(not_empty, mutex)` reacquires the mutex it will exit without rechecking the condition whether the buffer is empty and continue with the next statement, which is usually popping the item from the buffer. But this will cause an error since the buffer is empty. Therefore after exiting `cond_wait()` the consumer must recheck the condition, which is realized with a while loop

   more concisely:

   A notification only means that the predicate may now be true. Before the awakened consumer reacquires the mutex, another consumer may consume the item; wakeups may also be spurious. Therefore, the consumer must recheck count == 0 after every wakeup, which requires while, not if.

4. The signal goes undetected, it is not stored or rememberd. This is not necessarily an error because the shared predicate is persistent. A later consumer checks `count` while holding the mutex and will not wait if an item is already available.

5. `S` is incremented, which means that the signal is 'remembered', and a later `wait(S)` can consumer that permit

6. when all waiting threads must be able continue. For example if the buffer is empty and production is finished in the producer-consumer example, the sleeping threads must all continue. Otherwise they will wait indefinitely, since no new items will arrive and therefore the producer will not send a signal to wake them up.

Exercise 3

1. answer:

   ```
   W1: wait(S) // S: 4 -> 3
   W1: wait(S) // S: 3 -> 2
   W2: wait(S) // S: 2 -> 1
   W2: wait(S) // S: 1 -> 0
   W1: wait(S) // blocks
   W2: wait(S) // blocks
   ```

2. deadlock

3. In this interleaving, W1 and W2 each hold two permits, while S==0. Each needs two additional permits before it can enter the critical region. Neither reaches unlock_write, so neither can release its already acquired permits. Consequently, both remain blocked forever. More generally, a deadlock occurs if the permits are divided between the writers before either writer acquires all four

4. answer

```
Semaphore S = 4
Semaphore writer_gate = 1

lock_read: wait(S)
unlock_read: signal(S)

lock_write:
 wait(writer_gate)
 repeat 4 times:
     wait(S)
 signal(writer_gate)

 unlock_write:
     repeat 4 timtes: signal(S)
```

5. No. The gate prevents competing writers from splitting the permits, but it does not prevent newly arriving readers from repeatedly acquiring permits before the waiting writer. Therefore, writer starvation remains possible unless additional fairness—such as a common entry gate or an appropriate FIFO guarantee—is provided.

6 Paging fundamentals

The central skill for today is:

virtual address → page number and offset → frame number → physical address.

The other main skill is determining **what a page table has one entry for**, then multiplying its number of entries by the stated entry size.

6.1 1. What paging does

A process uses **logical/virtual addresses** (*logische/virtuelle Adressen*). A **physical address** (*physische Adresse*) identifies a location in physical memory. The **MMU** (*Memory Management Unit*) translates addresses using mappings established by the operating system.

Paging divides:

- a process's virtual address space into fixed-size **pages** (*Seiten*);
- physical memory into equally sized **frames** (*Seitenrahmen*).

A page and a frame have the same size. A resident virtual page occupies one physical frame. Consecutive virtual pages can occupy nonconsecutive frames.

For example, virtual pages 0, 1 and 2 could occupy frames 7, 2 and 11. The program still accesses a continuous sequence of virtual addresses; translation accounts for the physical arrangement.

Virtual addresses are process-specific. Page 1 of process A and page 1 of process B need not contain the same data or map to the same frame. In the normal-table model, each process has its own table. Threads sharing one address space share its mappings.

Fragmentation

External fragmentation means free memory is split into separate holes, preventing an allocation that requires a sufficiently large contiguous region. Basic paging avoids this problem for frame allocation: a process's pages can use scattered free frames.

Internal fragmentation is unused space inside an allocated unit. Paging can waste space in a partially used page, for example at the end of a page-aligned buffer.

For a page-aligned region of L bytes and page size P:

$$N = \left\lceil \frac{L}{P} \right\rceil, \qquad \text{allocated bytes} = NP, \qquad \text{unused bytes} = NP - L.$$

Example: 5,000 bytes with 4 KiB pages require two pages, allocating 8,192 bytes and leaving 3,192 bytes unused.

Use the **ceiling**, not ordinary rounding or rounding down. This calculation counts the region's data pages, excluding page-table storage.

6.2 2. Address decomposition and powers of two

Assume **byte-addressable memory**: one address identifies one byte. Page and frame numbers start at zero.

Symbol	Meaning
V	Virtual byte address
A	Physical byte address
$P = 2^n$ bytes	Page/frame size
p	Virtual page number
f	Physical frame number
d	Offset within the page/frame
v	Virtual address width in bits

Division with remainder gives:

$$V = pP + d,$$

so:

$$\boxed{p = \left\lfloor \frac{V}{P} \right\rfloor, \qquad d = V \bmod P.}$$

The offset satisfies:

$$0 \leq d < P.$$

Because $P = 2^n$, the offset occupies the **lowest** n **bits**. The page number occupies the remaining $v - n$ **bits**.

Address	Upper bits	Lower bits
Virtual, v bits	Page number: $v - n$ bits	Offset: n bits
Physical, w bits	Frame number: $w - n$ bits	Offset: n bits

The reasoning is straightforward: n bits distinguish 2^n byte positions inside one page.

For a full v-bit virtual address space:

$$\text{virtual bytes} = 2^v, \qquad \boxed{\text{virtual pages} = 2^{v-n}.}$$

For R bytes of physical RAM, assumed here to consist of complete, consecutively numbered frames:

$$\boxed{F = \frac{R}{P}.}$$

Representing F frame numbers requires:

$$\lceil \log_2 F \rceil \text{ bits.}$$

If $R = 2^r$ bytes, this is $r - n$ bits.

A physical address width does not necessarily mean all those addresses have RAM installed. Use the stated RAM capacity when counting physical frames.

Short worked example

Take:

- 24-bit virtual addresses;
- 2 KiB pages;
- 8 MiB RAM.

Then:

$$P = 2^{11} \text{ bytes}$$

means **11 offset bits**.

The virtual page number has:

$$24 - 11 = 13 \text{ bits},$$

giving 2^{13} possible virtual pages.

The RAM contains:

$$\frac{2^{23}}{2^{11}} = 2^{12}$$

frames, requiring **12 bits per frame number**.

For $V = 6,197$:

$$6197 = 3 \cdot 2048 + 53,$$

so:

$$p = 3, \qquad d = 53.$$

Unit	Bytes

Units and useful calculations

Unit	Bytes
1 KiB	$2^{10} = 1,024$
1 MiB	$2^{20} = 1,048,576$
1 GiB	$2^{30} = 1,073,741,824$
1 kB	$10^{3} = 1,000$
1 MB	$10^{6} = 1,000,000$
1 GB	$10^{9} = 1,000,000,000$

Also:

$$8 \text{ bits} = 1 \text{ byte}.$$

Thus a **32-bit entry occupies 4 bytes**.

Simplify powers before converting units:

$$\frac{2^a}{2^b} = 2^{a-b}, \qquad 2^a \cdot 2^b = 2^{a+b},$$

and, for example:

$$2^{22} \text{ bytes} = 4 \text{ MiB}.$$

Some course sheets write "KB" while intending a binary page size. State that interpretation where necessary; this unit uses KiB explicitly.

For unsigned byte addresses in C:

```
page_number = V / P;
offset      = V % P;
```

For $P = 2^n$, the equivalent operations are `V >> n` and `V & (P - 1)`. Use integer types wide enough for the addresses and calculations.

6.3 3. Normal page tables and translation

A **normal/direct, single-level page table** (*normale/direkte, einstufige Seitentabelle*) is indexed by the virtual page number.

In the simplified mapping-only model:

$$\mathrm{PT}_{\mathrm{PID}}[p] = f.$$

The index is the page number; the entry contains the frame number.

The page number need not be stored again in the entry because its position already identifies it.

A complete **page-table entry**, or **PTE**, can also contain:

Field	Purpose
Frame number	Identifies the resident page's physical frame
Present/resident indicator	Indicates whether a resident mapping is available
Access permissions	Indicate whether a read, write or execution is allowed

Whether the virtual page belongs to an allocated region must also be known. The encoding of allocation and residency varies; use the exercise's stated model.

To translate a permitted access to a resident page:

1. Split V into p and d.
2. Select the current process's table and obtain f from entry p.
3. Form the physical address:

$$\boxed{A = fP + d.}$$

The offset is unchanged. Paging moves an entire page to a frame while preserving each byte's position inside it.

Continuing the example, if PT[3] = 9:

$$A = 9 \cdot 2048 + 53 = 18,485.$$

Frame 9 is a number; its base address is $9P$. These are different quantities.

Frame 0 is a valid frame, not automatically an indication that a page is absent.

When translation cannot complete

- If a page belongs to the address space but is **not resident**, the access faults; the OS may bring the page into RAM and retry the instruction.
- If the address is **not allocated**, or the access violates its permissions, the access is invalid. The OS handles the fault, normally reporting an error or terminating the process.

Do not calculate a successful physical access from an absent or disallowed entry. The detailed handling and page-replacement policies belong to later topics.

Reconstructing entries from address pairs

If a successful mapping $V \to A$ is given, calculate:

$$p = \lfloor V/P \rfloor, \qquad d = V \bmod P, \qquad f = \lfloor A/P \rfloor.$$

First check:

$$\boxed{V \bmod P = A \bmod P.}$$

Different offsets make the pair **impossible under ordinary paging with that page size**.

For a fixed mapping snapshot, two observations from the **same process and same virtual page** must also produce the same frame. A mapping can change over time, so the snapshot assumption matters.

Once the checks pass, enter $p \to f$ in that process's table.

Multiple observations from one page establish **one entry**, not one entry per observation. Unmentioned pages cannot be classified as unmapped merely because they do not appear in a partial trace.

6.4 4. Inverted page tables

An **inverted page table** (*invertierte Seitentabelle*) has one entry per **physical frame**, usually in one system-wide table.

In the basic model with one recorded mapping per occupied frame:

$$\mathrm{IPT}[f] = (\mathrm{PID}, p).$$

The frame number is the **index**, so it need not also be stored in the entry. The entry stores the process identifier and virtual page number, plus any specified flags.

The PID is necessary because different processes can use identical virtual page numbers.

Property	Normal table	Basic inverted table
One entry for each	Virtual page	Physical frame
Index	Page number p	Frame number f
Mapping information stored	Frame number f	PID and page number p
Organization	One table per process/address space	One system-wide table
Translation lookup	Index the process's table with p	Find (PID, p); its row index gives f

For PID 6 with page 3 in frame 9:

$$\text{PT}_6[3] = 9, \qquad \text{IPT}[9] = (6, 3).$$

An inverted table is **not directly indexed by the virtual page number**. Conceptually, translation searches for (PID, p). Practical implementations can accelerate this search; those mechanisms are outside today's scope.

Once f is found:

$$A = fP + d$$

still applies.

Shared memory: the important limitation

Normal tables express shared memory directly:

$$\text{PT}_A[2] = 11, \qquad \text{PT}_B[5] = 11.$$

Both virtual pages map to frame 11. With equal offsets, they refer to the same physical byte.

A basic inverted entry containing only **one** (PID, p) pair cannot represent both mappings in row 11. Supporting this requires additional mapping information, for example a list of aliases or a separate structure.

Shared memory is problematic for this simple representation; it is not fundamentally impossible with inverted tables.

6.5 5. Calculating page-table memory

Always separate:

$$\boxed{\text{table bytes} = \text{number of entries} \times \text{bytes per entry.}}$$

Normal table

For a **full, flat table** covering every page in a v-bit virtual address space, with page size 2^n and E_N bytes per entry:

$$\boxed{T_N = 2^{v-n} E_N \text{ bytes per process.}}$$

The entry count depends on the **virtual address space**, not RAM capacity or the number of resident pages.

Unmapped pages still have slots in this full-table model.

With k processes, each having its own such table:

$$T_{N,\text{total}} = kT_N.$$

Inverted table

With R bytes of RAM, page size P and E_I bytes per inverted entry:

$$\boxed{T_I = \frac{R}{P} E_I \text{ bytes system-wide.}}$$

Its fraction of RAM is:

$$\boxed{\frac{T_I}{R} = \frac{E_I}{P}.}$$

Multiply by 100 to express this as a percentage.

This fraction is independent of RAM capacity when page size and entry size remain fixed. **Do not multiply a single global inverted table's size by the number of processes.**

These formulas count the stated entries and exclude auxiliary search structures. Normal and inverted entries need not have the same size.

Worked comparison

Assume:

- 28-bit virtual addresses;
- 4 KiB pages;
- 64 MiB RAM;
- 4-byte normal entries;
- 8-byte inverted entries.

The normal table requires:

$$T_N = 2^{28-12} \cdot 4 = 2^{18} \text{ bytes} = 256 \text{ KiB per process.}$$

The inverted table requires:

$$T_I = \frac{2^{26}}{2^{12}} \cdot 8 = 2^{17} \text{ bytes} = 128 \text{ KiB total.}$$

Its fraction of RAM is:

$$\frac{8}{4096} = \frac{1}{512},$$

or approximately:

$$0.1953\%.$$

Address-field width is not complete entry size

If there are 2^{14} frames, a frame number needs **14 bits**.

That does not mean a complete normal PTE occupies 14 bytes, or even exactly 14 bits. Flags, storage alignment and the stipulated format also matter.

If a question specifies a **4-byte entry**, use 4 bytes. If it does not, state your assumed entry size or leave the answer in terms of E_N. This assumption is needed for Sheet 5 A6(1).

For a tightly packed B-bit inverted entry containing a $(v-n)$-bit page number and s flag bits, the remaining PID capacity is:

$$b_{\text{PID}} = B - (v - n) - s,$$

giving:

$$2^{b_{\text{PID}}} \text{ possible PID values.}$$

This assumes no other fields and no reserved PID values. It bounds how many process identifiers the format can distinguish; the actual simultaneous process count also depends on other resources.

The frame number and offset need not be stored: **the row identifies the frame, and the input address supplies the offset.**

6.6 Common exam traps

- **Confusing page size with offset width:** 4 KiB means 12 offset bits.
- **Changing the offset during translation:** valid address pairs have the same remainder modulo P.
- **Mixing numbers and addresses:** the physical address is $fP + d$, not $f + d$.
- **Ignoring the PID:** the same virtual address can translate differently in different processes.
- **Swapping indices and contents:** normal $p \to f$; inverted $f \to (\text{PID}, p)$. Frame 0 is valid.
- **Confusing entries, bits and bytes:** multiply the entry count by the complete entry size in bytes.
- **Counting the wrong address space:** normal tables count virtual pages; inverted tables count physical frames.
- **Mixing binary and decimal units or rounding allocations down:** state KB conventions and use a ceiling for required pages.

6.7 Exam-like exercises

Work **closed-book**, allowing approximately **45–60 minutes in total**. Show decompositions and short justifications, not only final values.

All addresses are byte addresses, page/frame numbering starts at zero, and mappings remain fixed unless a question explicitly changes them.

Exercise 1 — Two address spaces and a page boundary

A machine has **16-bit virtual addresses**, **32 KiB RAM** and **2 KiB pages**. These excerpts describe two normal page tables. R means readable and W writable.

Virtual page	PID 11	PID 22
0	Frame 7, present, R/W	Frame 4, present, R/W
1	Frame 2, present, R	Frame 7, present, R/W
2	Allocated, not resident	Frame 12, present, R/W
3	Frame 0, present, R/W	Not allocated

1. Determine the offset width, page-number width, number of virtual pages per process, number of physical frames and minimum frame-number width.
2. Complete the following table. For successful accesses, give p, d, f and the physical address. Otherwise explain why the access cannot complete normally.

PID	Operation	Virtual address
11	Read	`0x0834`
22	Write	`0x0834`
11	Write	`0x0834`
11	Read	`0x1000`
22	Read	`0x1800`
11	Read	`0x1800`

3. Identify the shared physical frame. Give one virtual address from each process that refers to its byte at **offset 52**. Explain why the virtual addresses differ.
4. PID 11 reads two successive virtual bytes at addresses **2,047 and 2,048**. Determine both physical addresses. Must successive virtual bytes be successive physical bytes? Explain using the page boundary.

Exercise 2 — Reconstruct and audit the mappings

A system has **1 KiB pages** and **16 KiB RAM**. These reliable observations were collected from one fixed mapping snapshot; all listed accesses succeeded.

PID	Virtual address	Physical address
4	1,037	7,181
9	1,037	2,061
4	2,053	11,269

PID	Virtual address	Physical address
9	4,105	9
4	1,535	7,679

1. Construct the relevant entries of both processes' normal tables. Show the page, offset and frame for each observation. Explain why five observations do not imply five distinct normal-table entries.
2. Construct the known rows of one basic system-wide inverted table. Label the row index and contents precisely. Can the observations determine whether the remaining frames are free?
3. Two additional records are proposed for the same snapshot:

   - PID 4, $V = 2,100,\ A = 11,322$;
   - PID 4, $V = 1,600,\ A = 8,768$.

   Assess each record. Their problems, if any, are different; explain each precisely.
4. Under the basic model with one recorded mapping per occupied frame, which process and virtual address correspond to physical address **2,258**?
5. The OS now additionally maps **PID 9's page 5 to frame 11** for shared memory. Show the new normal-table entry and explain why the basic inverted entry for frame 11 needs an extension. At **offset 17**, give both processes' virtual addresses and their common physical address.

Exercise 3 — Memory requirements and entry formats

A byte-addressable system has:

- **32-bit virtual addresses**, with all virtual addresses representable;
- **512 MiB RAM**;
- **8 KiB pages**;
- **4-byte normal entries** and **8-byte inverted entries**;
- **12 processes**, each with a full, flat normal table if normal tables are used.

Count only page-table entries, excluding auxiliary structures. RAM comprises complete frames.

1. Derive the number of virtual pages per process and physical frames. Give the page-number, offset and minimum frame-number widths.
2. Calculate the normal-table memory **per process** and **for all 12 processes**. Express the total as a percentage of RAM. Would having only ten resident pages per process reduce these full, flat tables?

3. Calculate the single inverted table's memory in **KiB** and its percentage of RAM. Derive its fraction of RAM directly from entry size and page size as a consistency check.
4. A designer proposes a different, tightly packed inverted entry of **exactly 32 bits**, containing only the virtual page number, **one present bit**, and a PID field. No PID values are reserved. How many PID values fit? Explain why the frame number and offset do not require fields, and what this result does and does not say about the simultaneous process count.
5. A **30,000-byte buffer** starts at a virtual page boundary and is fully resident. How many data pages/frames does it need, and how many allocated bytes remain unused? Recalculate the unused bytes if its length is **30 KiB** instead. Express **512 MiB in decimal MB**.
6. Suppose the page size becomes **16 KiB**, while both original entry sizes and the other parameters remain fixed. How do both table sizes change? State one potential cost of using larger pages.

Source calibration and corrections

The scope and style follow the assignment analysis, Day 3 plan and earlier study units in 15-Operating-Systems-and-Networks-SoSe-25-Notes.pdf. The selected course tasks are A5/A7 in 04-ueb04-ibn-SS2025.pdf and A2/A6 in 05-ueb05-ibn-SS2025.pdf. The terminology and theoretical framing follow chapter 7 of 01-ibn-zusammenfassung.pdf.

Aufgabe 2.1–2.4 of 12-ibn-klausur-2016.pdf directly supports address decomposition, table entries, table sizes and PID-capacity reasoning. Aufgabe 6 in 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf additionally supports two-process problems, unit conversions, required-page counts and shared memory. The exercises above are original practice calibrated to that evidence.

One important correction to 14-Operating-Systems-and-Networs-SoSe-25-Solutions.pdf: Sheet 5 A2 gives 8203 → 12299, implying **page 2 → frame 3**. The supplied solution changes the physical address and incorrectly concludes frame 2. Also, its Sheet 5 A6 result is exactly **4 MiB**, under the assumed 4-byte normal-entry size.

Here are the worked solutions, numbered to match the exercises. **All addresses below are decimal unless prefixed with `0x`.**

6.8 Solutions

Exercise 1 — Two address spaces and a page boundary

1. Address widths and counts

The page size is:

$$P = 2 \text{ KiB} = 2048 = 2^{11} \text{ bytes.}$$

Therefore:

Quantity	Calculation	Result
Offset width	$\log_2 2048$	**11 bits**
Page-number width	$16 - 11$	**5 bits**
Virtual pages per process	2^5	**32**
Physical frames	32 KiB/2 KiB	**16**
Minimum frame-number width	$\log_2 16$	**4 bits**

Virtual page numbers range from 0 to 31; physical frame numbers range from 0 to 15.

2. Address translations and access checks

Use:

$$p = \lfloor V/2048 \rfloor, \qquad d = V \bmod 2048, \qquad A = f \cdot 2048 + d.$$

In particular:

$$\texttt{0x0834} = 2100 = 1 \cdot 2048 + 52.$$

PID	Operation	Virtual address	p	d	Mapped frame	Result
11	Read	`0x0834`	1	52	2	**4,148 = `0x1034`**
22	Write	`0x0834`	1	52	7	**14,388 = `0x3834`**

PID	Operation	Virtual address	p	d	Mapped frame	Result
11	Write	`0x0834`	1	52	2	**Protection fault:** page is read-only
11	Read	`0x1000`	2	0	—	**Page fault:** allocated but not resident
22	Read	`0x1800`	3	0	—	**Invalid access:** page is not allocated
11	Read	`0x1800`	3	0	0	**Physical address 0**

For the successful accesses:

$$2 \cdot 2048 + 52 = 4148, \qquad 7 \cdot 2048 + 52 = 14388, \qquad 0 \cdot 2048 + 0 = 0.$$

The nonresident page may be loaded and the instruction retried. An unallocated page or forbidden write is a different situation: loading a page alone does not make that access valid.

3. Shared frame

Frame 7 is shared:

$$\mathrm{PT}_{11}[0] = 7, \qquad \mathrm{PT}_{22}[1] = 7.$$

At offset 52:

Process	Virtual address	Physical address
PID 11	$0 \cdot 2048 + 52 = \mathbf{52}$	$7 \cdot 2048 + 52 = \mathbf{14388}$
PID 22	$1 \cdot 2048 + 52 = \mathbf{2100}$	$7 \cdot 2048 + 52 = \mathbf{14388}$

The virtual addresses differ because the shared frame is mapped at different virtual page numbers.

4. Crossing a page boundary

For PID 11:

$$2047 = 0 \cdot 2048 + 2047.$$

Page 0 maps to frame 7:

$$A_1 = 7 \cdot 2048 + 2047 = \boxed{16383}.$$

The next virtual address is:

$$2048 = 1 \cdot 2048 + 0.$$

Page 1 maps to frame 2:

$$A_2 = 2 \cdot 2048 + 0 = \boxed{4096}.$$

Thus, **successive virtual bytes need not be successive physical bytes**. Within a page, offsets remain consecutive; crossing a page boundary can move to an entirely different physical frame.

Exercise 2 — Reconstruct and audit the mappings

Here:

$$P = 1 \text{ KiB} = 1024 \text{ bytes}.$$

PID	V	Virtual page p	Offset d	A	Frame f	Physical offset

1. Normal page-table entries

Decompose both addresses:

PID	V	Virtual page p	Offset d	A	Frame f	Physical offset
4	1,037	1	13	7,181	7	13
9	1,037	1	13	2,061	2	13
4	2,053	2	5	11,269	11	5
9	4,105	4	9	9	0	9
4	1,535	1	511	7,679	7	511

For example:

$$1037 = 1 \cdot 1024 + 13, \qquad 7181 = 7 \cdot 1024 + 13.$$

The relevant normal-table entries are:

Process	Entry index: virtual page	Entry content: frame
PID 4	1	7
PID 4	2	11
PID 9	1	2
PID 9	4	0

There are **four distinct entries**. The first and fifth observations both describe PID 4's page 1 mapping to frame 7; they merely access different offsets.

2. Inverted table

The inverted table has:

$$16 \text{ KiB}/1 \text{ KiB} = 16$$

rows, indexed by frame numbers 0–15.

Its known rows are:

Row index: frame	Stored PID	Stored virtual page
0	9	4
2	9	1
7	4	1
11	4	2

Equivalently:

$$\mathrm{IPT}[0] = (9, 4), \quad \mathrm{IPT}[2] = (9, 1), \quad \mathrm{IPT}[7] = (4, 1), \quad \mathrm{IPT}[11] = (4, 2).$$

The other frames cannot be classified as free. A partial access trace does not reveal mappings that were never accessed.

3. Audit the additional records

Record A: PID 4, $V = 2100,\ A = 11322$

$$2100 = 2 \cdot 1024 + 52,$$

but:

$$11322 = 11 \cdot 1024 + 58.$$

The offsets are **52 and 58**, so this mapping is impossible under the stated paging system. Translation must preserve the offset.

Record B: PID 4, $V = 1600,\ A = 8768$

$$1600 = 1 \cdot 1024 + 576, \qquad 8768 = 8 \cdot 1024 + 576.$$

The offsets agree, but the record claims:

$$\mathrm{PT}_4[1] = 8.$$

The reliable observations establish:

$$\mathrm{PT}_4[1] = 7.$$

It therefore **contradicts the fixed mapping snapshot**. Offset agreement is necessary, but it does not establish consistency with other observations.

4. Reverse lookup of physical address 2,258

$$2258 = 2 \cdot 1024 + 210.$$

Thus frame $f = 2$, offset $d = 210$. Since:

$$\mathrm{IPT}[2] = (9, 1),$$

the corresponding process is **PID 9**, and its virtual address is:

$$V = 1 \cdot 1024 + 210 = \boxed{1234}.$$

5. Adding shared memory

The new normal-table entry is:

$$\boxed{\mathrm{PT}_9[5] = 11.}$$

Frame 11 now represents both:

$$(\mathrm{PID} = 4, p = 2) \quad \text{and} \quad (\mathrm{PID} = 9, p = 5).$$

A basic inverted row storing only one pair cannot describe both. An extension could store both pairs in an alias list associated with frame 11.

At offset 17:

Process	Virtual address
PID 4	$2 \cdot 1024 + 17 = \mathbf{2065}$
PID 9	$5 \cdot 1024 + 17 = \mathbf{5137}$

Their common physical address is:

$$A = 11 \cdot 1024 + 17 = \boxed{11281}.$$

Exercise 3 — Memory requirements and entry formats

Given:

$$P = 8 \text{ KiB} = 2^{13} \text{ bytes}, \qquad R = 512 \text{ MiB} = 2^{29} \text{ bytes}.$$

1. Page/frame counts and widths

Quantity	Calculation	Result
Offset width	$\log_2 P$	**13 bits**
Page-number width	$32 - 13$	**19 bits**
Virtual pages per process	2^{19}	**524,288**
Physical frames	$2^{29}/2^{13} = 2^{16}$	**65,536**
Minimum frame-number width	$\log_2 2^{16}$	**16 bits**

2. Normal-table memory

Each full normal table has 2^{19} entries of 4 bytes:

$$T_N = 2^{19} \cdot 4 = 2^{21} \text{ bytes} = \boxed{2 \text{ MiB per process}}.$$

For 12 processes:

$$T_{N,\text{total}} = 12 \cdot 2 \text{ MiB} = \boxed{24 \text{ MiB}}.$$

As a percentage of RAM:

$$\frac{24}{512} \cdot 100\% = \boxed{4.6875\%}.$$

Only ten resident pages would not reduce these full, flat tables. Every possible virtual page still has a table slot; nonresident or unallocated pages are represented by appropriate status information.

3. Inverted-table memory

There are 2^{16} physical frames and 8 bytes per entry:

$$T_I = 2^{16} \cdot 8 = 2^{19} \text{ bytes} = \boxed{512 \text{ KiB}}.$$

Its percentage of RAM is:

$$\frac{2^{19}}{2^{29}} \cdot 100\% = \frac{1}{1024} \cdot 100\% = \boxed{0.09765625\%}.$$

The direct consistency check gives:

$$\frac{T_I}{R} = \frac{E_I}{P} = \frac{8}{8192} = \boxed{\frac{1}{1024}}.$$

This is **one system-wide table**, so no multiplication by 12 is needed.

4. Proposed 32-bit inverted entry

The entry contains:

- virtual page number: **19 bits**;
- present indicator: **1 bit**;
- PID: the remaining bits.

Therefore:

$$b_{\text{PID}} = 32 - 19 - 1 = \boxed{12 \text{ bits}}.$$

This accommodates:

$$2^{12} = \boxed{4096 \text{ PID values}},$$

numbered 0–4095 under the assumption that none are reserved.

The frame number is implicit in the **row index**. The offset comes from the **address being translated**.

The result bounds how many process identities this format can distinguish. It does **not prove that 4,096 processes can simultaneously run**: their memory requirements and other resource limits also matter.

5. Buffer allocation and unit conversion

For a **30,000-byte** page-aligned buffer:

$$N = \left\lceil \frac{30000}{8192} \right\rceil = \boxed{4}.$$

It needs **four data pages and four resident frames**.

Allocated bytes:

$$4 \cdot 8192 = 32768.$$

Unused bytes:

$$32768 - 30000 = \boxed{2768 \text{ bytes}}.$$

For a **30 KiB** buffer:

$$L = 30 \cdot 1024 = 30720 \text{ bytes}.$$

It still requires four pages, but the unused space is:

$$32768 - 30720 = \boxed{2048 \text{ bytes} = 2 \text{ KiB}}.$$

Finally:

$$512 \text{ MiB} = 512 \cdot 2^{20} = 536870912 \text{ bytes},$$

so:

$$\boxed{512 \text{ MiB} = 536.870912 \text{ MB}}.$$

6. Increasing page size to 16 KiB

The new page size is 2^{14} bytes. Doubling page size **halves both entry counts**, while the stipulated entry sizes remain unchanged.

Normal table:

$$T'_N = 2^{32-14} \cdot 4 = 2^{20} \text{ bytes} = \boxed{1 \text{ MiB per process}}.$$

For all 12 processes:

$$\boxed{12 \text{ MiB total}}.$$

Inverted table:

$$T'_I = \frac{2^{29}}{2^{14}} \cdot 8 = 2^{18} \text{ bytes} = \boxed{256 \text{ KiB}}.$$

One potential cost is **greater internal fragmentation**: a partially used page can leave more allocated space unused.

7 Advanced paging and virtual memory

Day 3 established how a virtual page number maps to a physical frame. Today adds three mechanisms:

- **Hierarchical page tables** organize large, mostly unused mapping tables.
- **The TLB** speeds up address translation.
- **Demand paging and page replacement** manage pages that are not all resident in RAM.

The strongest direct Day 4 evidence in the **2016 exam** is the question about approximating last-access times using hardware reference bits. Multilevel tables, TLB calculations, and replacement traces are supported primarily by the assignments and the explicit study plan. The **2026 memory protocol** confirms paging's importance but does not explicitly report these advanced mechanisms.

Throughout this unit, memory is byte-addressable, page sizes are powers of two, and KiB/MiB use binary units.

7.1 1. Hierarchical page tables

Why divide a page table into levels?

A full flat page table contains one entry for **every possible virtual page**, including unused virtual regions.

For example, 32-bit virtual addresses and 4 KiB pages give:

$$2^{32-12} = 2^{20}$$

entries. At four bytes per entry, that is **4 MiB per process**, even if the process uses little memory.

A hierarchical table divides this mapping into smaller tables. The root contains references to tables at the next level; only the final level contains mappings to **data frames**.

Its main space advantage is:

Tables for unused virtual-address regions need not be allocated.

Hierarchy does **not** make a fully populated table smaller. When every possible mapping is represented, the leaf entries still exist, and the upper levels add overhead.

Address decomposition and translation

With v-bit virtual addresses, page size 2^n bytes, and k table levels:

$$\boxed{v = b_1 + b_2 + \cdots + b_k + n}$$

Here, b_i is the number of bits indexing level i; the final n bits are the offset.

For three levels:

Virtual-address field	Function
i_1: b_1 bits	Select an entry in the root table
i_2: b_2 bits	Select an entry in the chosen level-2 table
i_3: b_3 bits	Select an entry in the chosen leaf table
d: n bits	Select a byte within the mapped data frame

The translation proceeds as follows:

1. The current process's page-table root address identifies the root.
2. Root entry i_1 identifies the physical base address of a level-2 table.
3. Entry i_2 of that table identifies a leaf table.
4. Leaf entry i_3 contains the data-frame number f, together with flags.
5. Form the physical address:

$$A = f\,2^n + d.$$

The offset remains unchanged. The table levels collectively translate the virtual page number.

If entries occupy E_i bytes, the selected entry at level i is located at:

$$\text{table base} + i_i E_i.$$

Intermediate entries identify **next-level tables**; leaf entries identify **data frames**. Confusing those contents is a common mistake.

Counting tables and entries

An index with b_i bits selects among 2^{b_i} entries in **one table** at that level.

For a fully populated three-level hierarchy:

Level	Number of tables	Entries per table	Total entries at that level
1	1	2^{b_1}	2^{b_1}
2	2^{b_1}	2^{b_2}	$2^{b_1+b_2}$
3	$2^{b_1+b_2}$	2^{b_3}	$2^{b_1+b_2+b_3}$

Thus, with E bytes per entry at every level:

$$\boxed{T_{\max} = E\left(2^{b_1} + 2^{b_1+b_2} + 2^{b_1+b_2+b_3}\right)}$$

More generally:

$$T_{\max} = \sum_{i=1}^{k} E_i \, 2^{b_1+\cdots+b_i}.$$

This counts **page-table storage only**, excluding the process's data pages.

Short worked example

Suppose a system has:

- 24-bit virtual addresses;
- 256-byte pages;
- two levels, each using eight index bits;
- four-byte entries.

The address split is:

$$\underbrace{i_1}_{8} \mid \underbrace{i_2}_{8} \mid \underbrace{d}_{8}.$$

Each table has 256 entries and occupies:

$$256 \cdot 4 = 1 \text{ KiB}.$$

The maximum hierarchy contains one root and 256 leaf tables:

$$T_{\max} = 257 \text{ KiB}.$$

If the process uses pages in only **two root-index regions**, it needs one root and two leaf tables:

$$T_{\text{allocated}} = 3 \text{ KiB}.$$

This assumes complete tables are allocated and ignores allocator overhead. Even if only one entry in a leaf table is used, the whole leaf table occupies 1 KiB.

Do not assume a table occupies exactly one page. That is a common design choice, but follows only when the specified entry count and entry size make it true.

7.2 2. The TLB: caching translations

A **Translation Lookaside Buffer**, or TLB, is a small, fast hardware cache of page translations.

Conceptually, an entry contains:

$$(\text{address-space identity}, \text{virtual page}) \longrightarrow (\text{physical frame}, \text{permissions}).$$

It caches a **translation**, rather than the page's data.

Hit and miss

For a permitted access:

Event	What happens
TLB hit	Obtain the frame number from the TLB; access the data at $fP + d$
TLB miss, page resident	Walk the page table, obtain the mapping, refill the TLB, and complete the access
TLB miss, page nonresident	The page-table lookup detects absence; a page fault transfers control to the OS

A TLB miss therefore **does not imply a page fault**.

A page-table walk may be performed by hardware or by software, depending on the architecture. For these exercises, the essential point is the extra translation work on a miss.

Why a small TLB helps

Programs exhibit:

- **Temporal locality:** recently accessed pages are often accessed again.
- **Spatial locality:** nearby addresses often lie within the same page.

For example, a page-aligned array of four-byte integers contains 1,024 integers per 4 KiB page. Sequentially reading those integers reuses one translation many times.

The TLB's nominal **coverage**, sometimes called its reach, is:

$$\text{TLB entries} \times \text{page size}.$$

A 64-entry TLB with 4 KiB pages can cache translations covering 256 KiB.

Coverage alone does not determine the miss rate. Access order, alignment, associativity, and replacement policy also matter. For a simplified fully associative LRU TLB, repeatedly cycling through more distinct pages than it can hold can produce a miss on every access.

Quantitative access time

The course explicitly includes this calculation.

Assume:

- k page-table levels;
- one RAM access per level on a miss;
- all pages and page tables are resident;
- no additional caches or overlapping accesses;
- TLB lookup time t_{TLB};
- RAM access time t_{RAM};
- TLB hit ratio h;
- **every access first performs a TLB lookup**.

Then:

$$t_{\text{hit}} = t_{\text{TLB}} + t_{\text{RAM}},$$

$$t_{\text{miss}} = t_{\text{TLB}} + (k+1)t_{\text{RAM}}.$$

The $+1$ is the actual data access after the k table accesses.

Therefore:

$$\boxed{\text{EAT} = h\,t_{\text{hit}} + (1-h)t_{\text{miss}} = t_{\text{TLB}} + [1 + k(1-h)]\,t_{\text{RAM}}}$$

For two levels, $t_{\text{TLB}} = 5$ ns, $t_{\text{RAM}} = 80$ ns, and $h = 0.95$:

$$\text{EAT} = 5 + [1 + 2(0.05)]80 = 93 \text{ ns}.$$

Without a TLB, the simplified two-level translation plus data access costs $3 \cdot 80 = 240$ ns.

Convention note: the supplied solutions omit TLB lookup time from the miss branch. That requires a different timing assumption or an approximation. State your model; with the sequential lookup model above, it belongs in both branches.

These formulas exclude page faults. Fetching a missing page can be much more expensive than a resident-page access.

Process and thread switches

The same virtual page number can map to different frames in different processes.

Without address-space tags, cached translations must be invalidated when switching address spaces. Otherwise, process B could use process A's translation.

With address-space identifiers, translations from multiple address spaces can coexist safely if correctly tagged.

Switching between threads that share one address space does **not by itself** require a TLB flush.

7.3 3. Demand paging and page-fault handling

Paging provides address translation. **Demand paging** additionally allows a valid virtual page to be absent from RAM until needed.

A nonresident page may have contents in an executable file, another mapped file, or swap storage. A newly allocated page may instead be initialized with zeros. A page fault therefore does not always require reading an existing page from disk.

For today's replacement exercises, a **page fault** means that the referenced page is absent from the available frames.

The basic sequence

When an access references a valid but nonresident page:

1. The hardware detects the missing resident mapping and raises a page fault.
2. The OS checks whether the access is valid and permitted.
3. The OS obtains a frame:
    - use a free frame if one is available;
    - otherwise select a resident victim page.
4. If necessary, preserve the victim's modified contents.
5. Load or initialize the requested page.
6. Update page-table entries and invalidate stale cached translations.
7. Restart the faulting instruction.

An invalid address or forbidden operation cannot be repaired simply by loading a page.

Page fault versus replacement

A page fault requires replacement only when no suitable free frame is available.

Consequently:

- Loading the first page into initially empty memory causes a page fault.
- It does not evict another page.
- Once all allocated frames are occupied, a fault requires a victim under the exercise's fixed-frame model.

There are also two distinct kinds of eviction:

Eviction	What is removed?
TLB-entry eviction	A cached translation
Page replacement	A resident page's contents from its frame

Removing a TLB entry leaves the page resident.

Reference and dirty bits

Hardware maintains flags associated with pages:

Bit	Meaning	Set by
R / accessed bit	The page has been referenced since the bit was cleared	Hardware on access
M / dirty bit	The page has been modified since its contents were made clean	Hardware on write

Reads and writes both count as references. A read alone does not make a page dirty.

The OS clears bits when appropriate. A dirty victim generally needs its modified contents preserved before its frame is reused. A clean page with an unchanged backing copy can usually be discarded without write-back.

7.4 4. FIFO, LRU, and Clock

All three policies select among **resident pages** when replacement is necessary.

FIFO: oldest arrival

First In, First Out replaces the page that entered memory earliest.

Maintain a queue from oldest to newest:

- Hit: leave the queue unchanged.
- Fault with free space: load the page and append it.
- Fault with full memory: remove the oldest page and append the new page.

A frequently accessed page can still be FIFO's victim because hits do not renew its arrival time.

LRU: oldest last access

Least Recently Used replaces the page whose most recent access occurred furthest in the past.

For manual simulation, maintain an order from least to most recently used:

- Hit: move the referenced page to the most-recent end.
- Fault: load the page as most recently used.
- If memory is full, first remove the least-recent page.

Exact LRU needs recency information updated on **every reference**. Maintaining that precisely can be expensive; hardware bits support cheaper approximations.

One state illustrating the difference

Suppose three frames receive:

$$A,\ B,\ C,\ A.$$

All three pages are resident.

- FIFO order: A, B, C. Its next victim is **A**.
- LRU order: B, C, A. Its next victim is **B**.

FIFO asks **"Who arrived first?"** LRU asks **"Who was used least recently?"**

Clock / Second Chance

Clock implements a second-chance policy using:

- frames arranged in a circular order;
- one reference bit per resident page;
- a hand pointing to the next candidate.

Use this explicit convention:

- Every completed reference sets the page's $R = 1$, including the reference that loads it.
- Hits do not move the hand.
- On replacement:

  - if the candidate has $R = 0$, replace it;
  - if it has $R = 1$, clear the bit and advance;
  - repeat until a victim is found.
- Load the new page with $R = 1$, then advance the hand to the next frame.

Queue-based Second Chance applies the same idea by moving an oldest page with $R = 1$ to the back after clearing its bit. Clock represents that rotation with a circular hand.

Initialization conventions can change a trace. Some teaching examples initialize newly loaded pages differently. Always follow the stated convention.

Why Clock differs from exact LRU

After A, B, C, A, suppose all bits are 1 and the hand points to A.

On a fault for D, Clock:

1. Clears A's bit.
2. Clears B's bit.
3. Clears C's bit.
4. Returns to A, now with $R = 0$, and replaces it.

Exact LRU would replace B.

Clock knows that all three pages were referenced since their bits were cleared. It does not know the exact ordering of those references.

Reliable manual tracing

For each reference:

1. Check whether the page is already resident.
2. Mark **hit** or **fault**.
3. On a fault, fill a free frame or select a victim.
4. Update the policy's metadata.
5. Record the complete state **after** the reference.

Keep physical frame positions fixed in your table. Record FIFO/LRU order separately; changing recency order does not move page contents between frames.

Policy	Metadata you must update
FIFO	Arrival queue; unchanged on hits
LRU	Recency order; updated on every reference

Policy	Metadata you must update
Clock	R bits and hand position

FIFO can exhibit **Belady's anomaly**: adding frames can increase faults for some reference strings. Exact LRU does not have this anomaly. Neither FIFO nor LRU wins on every individual reference string.

7.5 5. Approximating recency with R bits

An R bit answers:

> Has this page been referenced at least once since the bit was last cleared?

It does **not** record:

- how many references occurred;
- their exact times;
- their ordering relative to references to other pages.

Periodic sampling

Divide time into **epochs**. At each epoch boundary, the OS conceptually performs:

```
for each page-table entry being monitored:
    if entry.R == 1:
        entry.last_observed_epoch = current_epoch

    entry.R = 0
```

During the next epoch, hardware sets the bit again if the page is accessed.

A page with an older `last_observed_epoch` has not been observed in use for longer and is a plausible victim.

However, two pages accessed in the same epoch receive the same approximate timestamp, even if one was accessed at its beginning and the other immediately before the scan.

This is **LRU-like approximation**, not exact LRU.

Real implementations must ensure that clearing the bit makes subsequent accesses observable despite cached translations. For exam purposes, assume the required synchronization and invalidation are included in the stated operation cost.

Sampling versus Clock

Both use hardware reference bits, but differently:

Mechanism	When are bits inspected and cleared?
Periodic epoch sampling	At regular time intervals
Basic Clock	While searching for a victim

Clock's hand position and previous scans influence its information. Its bits do not necessarily describe one common sampling interval.

R-bit scanning overhead

The Sheet 6 A4 model scans full flat page tables.

Let:

- m: number of processes;
- v: virtual-address width;
- 2^n: page size;
- c: cycles to sample and clear one entry;
- f_{CPU}: CPU cycles per second.

Then:

$$N_{\text{scan}} = m\,2^{v-n},$$

$$C_{\text{scan}} = c\,m\,2^{v-n},$$

$$\boxed{t_{\text{scan}} = \frac{c\,m\,2^{v-n}}{f_{\text{CPU}}}}$$

This counts entries in the stipulated tables. It does **not** require every virtual page to be resident or even allocated.

For the original sheet's parameters:

$$t_{\text{scan}} = \frac{10 \cdot 20 \cdot 2^{20}}{10^9} \approx 0.2097 \text{ s}.$$

If scans start every T seconds, including scanning time within that interval, their CPU-time fraction is:

$$\boxed{\alpha = \frac{t_{\text{scan}}}{T}}.$$

To stay at or below a budget $\alpha_{\max}$:

$$\boxed{T \geq \frac{t_{\text{scan}}}{\alpha_{\max}}}.$$

For a 10% budget, the original parameters require at least about **2.10 seconds**, rather than exactly 2 seconds.

The tradeoff is:

- **Short epochs:** finer recency information, more scanning overhead.
- **Long epochs:** less overhead, more references grouped into indistinguishable intervals.

There is no universally correct epoch length. Derive it from the stated budget and explain the loss of precision.

7.6 Common exam traps

- **Counting one table instead of a whole level:** distinguish entries per table, tables per level, and total entries per level.
- **Claiming hierarchy always saves memory:** sparse allocation saves memory; a fully populated hierarchy adds upper-level overhead.
- **Confusing intermediate pointers with data-frame numbers.**
- **Equating a TLB miss with a page fault**, or a page fault with an eviction.
- **Forgetting the final data access** in TLB timing calculations.
- **Updating FIFO on hits**, or failing to update LRU on hits.
- **Tracing Clock without specifying the hand and R-bit conventions.**

- **Treating R bits as exact timestamps:** they record occurrence within an interval, not access order.
- **Scanning physical frames when the problem specifies full per-process flat tables**, or mixing CPU cycles with seconds.

7.7 Exam-like exercises

Allow approximately **55–70 minutes**. Show intermediate states and short explanations. Solutions are intentionally omitted so you can work through these first.

Exercise 1 — A sparse three-level address space

A machine uses:

- 30-bit virtual byte addresses;
- 1 KiB pages;
- three page-table levels with index widths **6, 7, and 7 bits**, from root to leaf;
- four-byte entries at every level.

Intermediate entries contain physical base addresses of next-level tables. Leaf entries contain data-frame numbers and flags. Complete tables are allocated; ignore alignment padding and other metadata.

A. Structure and translation

1. Give the address decomposition, including the bit positions of all fields.
2. Determine entries per table and bytes per table at each level.
3. A virtual address has fields:

$$i_1 = 5, \quad i_2 = 9, \quad i_3 = 17, \quad d = 37.$$

The root is based at physical address `0x4000`.

- Root entry 5 contains next-table base `0x8000`.
- Entry 9 of that table contains leaf-table base `0xA000`.
- Leaf entry 17 maps to frame 73 and permits the access.

Calculate:

- the virtual byte address;
- the physical address of each of the three selected entries;
- the resulting physical data address.

B. Full versus sparse allocation

4. For a fully populated hierarchy, calculate:
    - the number of tables at each level;
    - the total entries at each level;
    - the total page-table memory.
5. Compare this with a full flat table using four-byte entries. Explain the difference.

The process actually has allocated virtual pages only at these index triples:

$$(0, 0, 0), \quad (0, 0, 127), \quad (0, 3, 2), \quad (63, 127, 127).$$

6. Determine the number of allocated tables at each level and the total table memory.
7. The process additionally allocates page $(0, 3, 3)$. Does this require another page-table allocation? What about allocating $(0, 4, 0)$? Explain.

Exercise 2 — Translation misses and missing pages

A system uses two-level page tables and 4 KiB pages.

For process P:

Virtual page	Page-table state
0	Resident in frame 8, readable
1	Resident in frame 3, readable
2	Valid and readable, but nonresident

Initially:

- P's TLB contains only page $0 \to 8$.
- The TLB can hold all translations used in this exercise.
- One free data frame, frame 6, is available.
- Page-table pages are resident.
- The OS loads page 2 into frame 6 when necessary.

A. Trace the mechanisms

P performs these reads in order:

$$0x0020, \quad 0x1040, \quad 0x1080, \quad 0x2008, \quad 0x200C.$$

1. For each read, give:
   - virtual page and offset;
   - whether the **initial** TLB lookup hits or misses;
   - whether a page-table walk occurs;
   - whether a page fault occurs;
   - whether a resident data page is evicted;
   - the physical address after successful completion.
2. Give the total initial TLB misses, page faults, and data-page evictions.

B. Timing and capacity

Assume a sequential TLB lookup model with:

$$t_{\text{TLB}} = 8 \text{ ns}, \qquad t_{\text{RAM}} = 70 \text{ ns}, \qquad h = 0.90.$$

3. Calculate hit time, resident-page miss time, and expected access time. Exclude page faults.
4. A separate workload repeatedly reads one byte from pages:

$$4, 5, 6, 7, 4, 5, 6, 7, ...$$

All four pages remain resident. Its initially empty TLB is fully associative, holds three translations, and uses exact LRU. Determine the hit/miss behavior and explain whether it implies page faults.

C. Address-space identity

5. Process Q maps its page 0 to frame 12. Explain what could go wrong if an untagged TLB retains P's page-0 translation when switching to Q.
6. Does switching between two threads of P create the same problem? Explain.

Exercise 3 — Complete FIFO, LRU, and Clock traces

A process has **three initially empty frames**, numbered 0, 1, and 2. All referenced pages are valid. The reference string is:

$$A,\ B,\ C,\ A,\ D,\ B,\ E,\ A,\ B,\ C,\ D,\ E.$$

A. FIFO and exact LRU

1. Simulate FIFO and exact LRU separately.
2. After **every reference**, record:
   - all three frame contents;
   - hit or fault;
   - victim, if any;
   - FIFO queue or LRU order.
3. Count page faults and evictions for each policy.
4. Identify the first reference at which their victim choices differ. Explain using their metadata.

B. Clock

Use these conventions:

- The hand initially points to frame 0.
- Empty frames are filled at the hand, then the hand advances.
- Every loaded or referenced page has $R = 1$.
- Hits leave the hand unchanged.
- During replacement, clear and pass over candidates with $R = 1$.
- Replace the first candidate with $R = 0$, set the new page's bit to 1, and advance the hand.

5. Simulate Clock for the same complete string.
6. After each reference, record:
   - frame contents and R bits;
   - the hand position;
   - hit/fault and victim.
7. At the **first replacement**, list every frame inspected, including repeated inspections. Compare Clock's victim with exact LRU's.

8. Explain why Clock can evict a page whose last access was more recent than another resident page's last access.

You may use this table layout separately for each policy:

Step	Reference	Frame 0	Frame 1	Frame 2	Hit/fault	Victim	Policy meta-data

---

Exercise 4 — R-bit evidence and its cost

A. What can sampling reveal?

Four resident pages A, B, C, and D initially have:

$$R = 0, \qquad \text{last observed epoch} = 0.$$

At the end of each epoch, the OS updates the approximate timestamp of every page with $R = 1$, then clears all R bits.

The references are:

Epoch	References in chronological order
1	A, B, A
2	C, B
3	A, C

1. Give every page's R bit immediately before each scan and its approximate timestamp immediately after each scan.
2. After epoch 3's scan, which page would an "oldest observed epoch" policy select first?
3. Suppose D has already been removed and another victim must immediately be selected among A, B, and C. Which page does exact LRU choose? Which page does the epoch approximation choose?
4. The OS then needs to distinguish A from C. Can their epoch timestamps establish their exact LRU order? State the actual order from the reference history and explain what sampling lost.
5. Does accessing A ten times during epoch 3 give its R bit a different value from accessing it once?

B. Scanning overhead

A different system has:

- 32-bit virtual addresses;
- 8 KiB pages;
- 12 processes, each with a full flat page table;
- a 2 GHz CPU;
- 16 CPU cycles to read, record, and clear one entry's R bit.

Assume a single CPU executes the scans sequentially. One scan occurs per epoch.

6. Calculate:
    - entries scanned per process;
    - total entries;
    - cycles per scan;
    - scan time in milliseconds.
7. The scan must consume at most **4%** of CPU time. Calculate the minimum epoch duration, measured from the start of one scan to the start of the next.
8. What fraction of CPU time would scanning consume with 100 ms epochs?
9. Explain why increasing the epoch duration solves the overhead problem while reducing recency precision.
10. Only a small fraction of virtual pages are resident. Does that reduce the cost under the specified **full-table scanning model**? What would have to change in the scanning method to exploit sparse residency?

The unit follows the plan and prerequisite chapter in 15-Operating-Systems-and-Networks-SoSe-25-Notes.pdf, the mechanisms and calculations in 05-ueb05-ibn-SS2025.pdf and 06-ueb06-ibn-SS2025.pdf, checked against 14-Operating-Systems-and-Networs-SoSe-25-Solutions.pdf and the theoretical framing in 01-ibn-zusammenfassung.pdf. Exam calibration uses 12-ibn-klausur-2016.pdf and 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf.

7.8 Solutions

Exercise 1 — A sparse three-level address space

A. Structure and translation

1–2. Address fields and table sizes

A 1 KiB page contains 2^{10} bytes, so the offset uses **10 bits**.

$$\boxed{i_1 : 6 \text{ bits} \mid i_2 : 7 \text{ bits} \mid i_3 : 7 \text{ bits} \mid d : 10 \text{ bits}}$$

Field	Bit positions
i_1	29–24
i_2	23–17
i_3	16–10
Offset d	9–0

At four bytes per entry:

Level	Entries in one table	Bytes in one table
1	$2^6 = 64$	$64 \cdot 4 = 256$
2	$2^7 = 128$	$128 \cdot 4 = 512$
3	$2^7 = 128$	$128 \cdot 4 = 512$

These tables need not occupy whole page frames under this exercise's assumptions.

3. Translation

The virtual address is:

$$\begin{aligned} V &= 5 \cdot 2^{24} + 9 \cdot 2^{17} + 17 \cdot 2^{10} + 37 \\ &= \boxed{85\,083\,173}. \end{aligned}$$

Each selected entry is at its table's base plus the index multiplied by four:

Selected entry	Physical address of entry	Entry content
Root entry 5	`0x4000 + 5·4 = 0x4014`	Next table at `0x8000`
Level-2 entry 9	`0x8000 + 9·4 = 0x8024`	Leaf table at `0xA000`
Leaf entry 17	`0xA000 + 17·4 = 0xA044`	Data frame 73

The physical data address is:

$$A = 73 \cdot 1024 + 37 = \boxed{74\,789}.$$

Check: the addresses of the table entries are intermediate lookup locations; none is the final data address.

B. Full versus sparse allocation

4. Fully populated hierarchy

Level	Number of tables	Entries per table	Total entries	Total bytes
1	1	64	64	256
2	64	128	8 192	32 768
3	8 192	128	1 048 576	4 194 304

Therefore:

$$T_{\max} = 256 + 32\,768 + 4\,194\,304 = \boxed{4\,227\,328 \text{ bytes}}.$$

Equivalently:

$$\boxed{4 \text{ MiB} + 32 \text{ KiB} + 256 \text{ bytes}}.$$

5. Flat-table comparison

There are:

$$2^{30-10} = 2^{20}$$

virtual pages. A full flat table occupies:

$$2^{20} \cdot 4 = \boxed{4 \text{ MiB}}.$$

The fully populated hierarchy uses **32 KiB + 256 bytes more**, because it contains the same number of leaf entries plus the upper-level entries.

Its advantage appears when unused branches are not allocated.

6. Sparse allocation

The mapped triples are:

$$(0,0,0), \quad (0,0,127), \quad (0,3,2), \quad (63,127,127).$$

Count distinct prefixes:

- One root.
- Two distinct first indices, 0 and 63: **two level-2 tables**.
- Three distinct pairs, $(0,0), (0,3), (63,127)$: **three leaf tables**.

Thus:

$$T = 256 + 2 \cdot 512 + 3 \cdot 512 = \boxed{2\,816 \text{ bytes} = 2.75 \text{ KiB}}.$$

The first two mappings share a leaf table.

7. Additional mappings

New page	New table allocation?	Reason
$(0,3,3)$	No	Leaf table for $(0,3)$ already exists
$(0,4,0)$	One new leaf table: **512 bytes**	Level-2 table for $i_1 = 0$ exists, but its branch $i_2 = 4$ needs a leaf

Allocating a data page also requires a data frame when made resident, but that is separate from the table-storage question.

Exercise 2 — Translation misses and missing pages

A. Trace the mechanisms

The page size is `0x1000` bytes.

Read	Page	Offset	Initial TLB	Table walk?	Page fault?	Data-page evic-tion?	Final physical address
`0x0020`	0	`0x020`	Hit	No	No	No	`0x8020` = 32,800
`0x1040`	1	`0x040`	Miss	Yes	No	No	`0x3040` = 12,352
`0x1080`	1	`0x080`	Hit	No	No	No	`0x3080` = 12,416
`0x2008`	2	`0x008`	Miss	Yes	Yes	No	`0x6008` = 24,584
`0x200C`	2	`0x00C`	Hit	No	No	No	`0x600C` = 24,588

After the access to page 1, its translation is cached. Page 2 is loaded into the free frame 6, and its translation is made available for subsequent accesses.

Totals:

$$\boxed{\text{2 initial TLB misses,} \quad \text{1 page fault,} \quad \text{0 evictions}}.$$

The fault does not require eviction because a free frame exists.

B. Timing and capacity

3. Access times

With two levels:

$$t_{\text{hit}} = 8 + 70 = \boxed{78 \text{ ns}},$$

$$t_{\text{miss}} = 8 + 3 \cdot 70 = \boxed{218 \text{ ns}}.$$

The miss needs two table accesses and one data access.

$$\text{EAT} = 0.9 \cdot 78 + 0.1 \cdot 218 = \boxed{92 \text{ ns}}.$$

4. Four pages cycling through a three-entry LRU TLB

Reference sequence:

$$4, 5, 6, 7, 4, 5, 6, 7, ...$$

Every access misses.

After loading translations for 4, 5, and 6, accessing 7 evicts 4. The next access to 4 evicts 5, and so on. Each page is removed before its next reference.

Nevertheless:

$$\boxed{\text{No page faults and no data-page replacement.}}$$

All data pages remain resident; only their cached translations are evicted.

C. Address-space identity

5. Switching to Q

An untagged cached entry for page 0 still says:

$$0 \to 8.$$

Q's correct mapping is:

$$0 \to 12.$$

If the old entry remains usable, Q could access **frame 8 instead of frame 12**, accessing the wrong process's memory.

The OS must invalidate the relevant untagged translations when changing address spaces, or use correctly managed address-space tags.

6. Switching between P's threads

Threads sharing P's address space use the same page mappings. Switching between them does **not itself** require invalidating those mappings.

Exercise 3 — Complete replacement traces

In all tables:

- States are recorded **after** the reference.
- – denotes an empty frame or no victim.
- **F** = page fault; **H** = hit.
- Frame positions stay fixed.

A. FIFO

Queue order is **oldest arrival → newest arrival**.

Step	Ref.	Frame 0	Frame 1	Frame 2	F/H	Victim	FIFO queue
1	A	A	—	—	F	—	A
2	B	A	B	—	F	—	A, B
3	C	A	B	C	F	—	A, B, C
4	A	A	B	C	H	—	A, B, C
5	D	D	B	C	F	A	B, C, D
6	B	D	B	C	H	—	B, C, D
7	E	D	E	C	F	B	C, D, E
8	A	D	E	A	F	C	D, E, A
9	B	B	E	A	F	D	E, A, B
10	C	B	C	A	F	E	A, B, C
11	D	B	C	D	F	A	B, C, D
12	E	E	C	D	F	B	C, D, E

FIFO: 10 faults, 7 evictions.

Hits at steps 4 and 6 leave the arrival queue unchanged.

Exact LRU

Order is **least recently used → most recently used**.

Step	Ref.	Frame 0	Frame 1	Frame 2	F/H	Victim	LRU order
1	A	A	—	—	F	—	A
2	B	A	B	—	F	—	A, B

Step	Ref.	Frame 0	Frame 1	Frame 2	F/H	Victim	LRU order
3	C	A	B	C	F	—	A, B, C
4	A	A	B	C	H	—	B, C, A
5	D	A	D	C	F	B	C, A, D
6	B	A	D	B	F	C	A, D, B
7	E	E	D	B	F	A	D, B, E
8	A	E	A	B	F	D	B, E, A
9	B	E	A	B	H	—	E, A, B
10	C	C	A	B	F	E	A, B, C
11	D	C	D	B	F	A	B, C, D
12	E	C	D	E	F	B	C, D, E

LRU: 10 faults, 7 evictions.

The first different victim choice is **step 5**:

- FIFO evicts **A**, the oldest arrival.
- LRU evicts **B**, because A was referenced again at step 4.

Both algorithms happen to have equal total faults here, despite different victims and different hit locations.

B. Clock

Notation `A/1` means page A with reference bit 1. The hand column gives the **next candidate frame** after completing the reference.

Step	Ref.	Frame 0	Frame 1	Frame 2	F/H	Victim	Hand
1	A	A/1	—	—	F	—	1
2	B	A/1	B/1	—	F	—	2
3	C	A/1	B/1	C/1	F	—	0
4	A	A/1	B/1	C/1	H	—	0
5	D	D/1	B/0	C/0	F	A	1
6	B	D/1	B/1	C/0	H	—	1
7	E	D/1	B/0	E/1	F	C	0
8	A	D/0	A/1	E/1	F	B	2
9	B	B/1	A/1	E/0	F	D	1
10	C	B/1	A/0	C/1	F	E	0
11	D	B/0	D/1	C/1	F	A	2
12	E	E/1	D/1	C/0	F	B	1

Step	Ref.	Frame 0	Frame 1	Frame 2	F/H	Victim	Hand

Clock: 10 faults, 7 evictions.

For checking your hand movements, replacement scans inspect these frame sequences:

Step	Frames inspected, in order
5	0, 1, 2, 0
7	1, 2
8	0, 1
9	2, 0
10	1, 2
11	0, 1
12	2, 0

First replacement: step 5

Before D:

- Frames contain A, B, C.
- All bits are 1.
- Hand points to frame 0.

Clock:

1. Inspects A in frame 0: clears $R = 1$ to 0.
2. Inspects B in frame 1: clears its bit.
3. Inspects C in frame 2: clears its bit.
4. Returns to A in frame 0: its bit is now 0, so replaces A.

Exact LRU replaces **B** instead.

Why Clock can evict a more recently used page

A was last accessed at step 4; B at step 2. Nevertheless, Clock evicts A.

The bits tell Clock that all three pages were referenced since their bits were cleared. They do not encode the order of those references. The hand breaks that ambiguity according to its circular scanning position.

Exercise 4 — R-bit evidence and scanning cost

A. Sampling

In the table below, each tuple lists values in order **A, B, C, D**.

Epoch	References	R bits immediately before scan	Approximate timestamps after scan
Initially	—	$(0, 0, 0, 0)$	$(0, 0, 0, 0)$
1	A, B, A	$(1, 1, 0, 0)$	$(1, 1, 0, 0)$
2	C, B	$(0, 1, 1, 0)$	$(1, 2, 2, 0)$
3	A, C	$(1, 0, 1, 0)$	$(3, 2, 3, 0)$

After every scan, all R bits are cleared to zero. The timestamps remain stored separately.

2. First victim

D has the oldest timestamp:

$$\boxed{D}.$$

It was never referenced during these epochs.

3. After D has been removed

Among A, B, and C, timestamps are:

$$A : 3, \qquad B : 2, \qquad C : 3.$$

The approximation chooses **B**.

Exact LRU also chooses **B**: its last reference was in epoch 2, while both A and C were referenced in epoch 3.

4. Distinguishing A from C

Their approximate timestamps are tied at 3, so the sampling information **cannot establish their exact order**.

The actual history ends with A, then C. Therefore:

$$\boxed{\text{A is less recently used than C.}}$$

Sampling loses the ordering of accesses within an epoch.

5. Ten accesses versus one

Both produce:

$$\boxed{R_A = 1}.$$

A single bit records whether at least one reference occurred, not the number of references.

B. Scanning overhead

6. Entries, cycles, and time

An 8 KiB page uses 13 offset bits. Thus, per process:

$$N_{\text{process}} = 2^{32-13} = \boxed{2^{19} = 524\,288 \text{ entries}}.$$

Across 12 processes:

$$N_{\text{total}} = 12 \cdot 524\,288 = \boxed{6\,291\,456 \text{ entries}}.$$

At 16 cycles per entry:

$$C = 6\,291\,456 \cdot 16 = \boxed{100\,663\,296 \text{ cycles}}.$$

At 2 GHz:

$$t_{\text{scan}} = \frac{100\,663\,296}{2 \cdot 10^9} = 0.050331648 \text{ s}.$$

$$\boxed{t_{\text{scan}} \approx 50.33 \text{ ms}}.$$

7. Maximum 4% overhead

The epoch duration includes the scan:

$$\frac{t_{\text{scan}}}{T} \leq 0.04.$$

Therefore:

$$T \geq \frac{0.050331648}{0.04} = \boxed{1.2582912 \text{ s}}.$$

A practical rounded choice is **1.26 seconds or longer**.

8. With 100 ms epochs

$$\alpha = \frac{50.331648 \text{ ms}}{100 \text{ ms}} = 0.50331648.$$

$$\boxed{\text{Scanning consumes approximately } 50.33\% \text{ of CPU time.}}$$

9. Longer epochs

Increasing T reduces the overhead fraction t_{scan}/T.

However, accesses over a longer interval receive the same epoch timestamp. The approximation therefore loses more information about which page was accessed most recently.

10. Sparse residency

No reduction under the specified model. It scans every entry of every full flat table, regardless of residency.

To exploit sparse residency, the scanner would need to visit only relevant entries—for example, by maintaining a collection of resident-page mappings rather than scanning the full virtual-page range.

For a quick comparison, your main numerical checkpoints are:

Exercise	Key results
1	Maximum hierarchy **4,227,328 B**; sparse hierarchy **2,816 B**; physical data address **74,789**
2	**2** initial TLB misses; **1** fault; **0** evictions; EAT **92 ns**
3	FIFO, LRU, Clock each have **10 faults and 7 evictions**, with different traces
4	Scan **50.33 ms**; minimum epoch **1.2583 s**; 100 ms epochs incur **50.33%** overhead

7.9 Supplementary Matrial

I agree with your suggestion to keep the detailed implementation relatively low priority for now. However, after checking the uploaded materials, epoch-based LRU approximation is actually more exam-relevant than you suspected.

The IBN summary explicitly describes storing the last 8, 16, or 32 observations of each page's R bit. Moreover, the 2016 exam (Aufgabe 2.5, 4 points) asks how hardware R bits can be used to approximate the last access time. Übungsblatt 6, Aufgabe 4 asks about scanning overhead and choosing an epoch length.

ibn-zusammenfassung.pdf

ibn-klausur-2016.pdf

Our previous assignment-sheet analysis also classified these calculations as high priority. So I wouldn't interpret the omission as a deliberate decision that the mechanism is unimportant. A concise explanation should have been included.

1. How epochs and shifting registers work

An epoch (Epoche) is simply a fixed time interval, for example 100 ms.

For each relevant page, the OS maintains:

- The hardware-maintained R bit.
- A software-maintained history register, say 8 bits long.

At the end of each epoch, the OS performs three operations:

1. Read the current R bit.
2. Shift the history register one position to the right, inserting R as the leftmost bit.

3. Clear R for the next epoch.

In pseudocode:

```
for each resident page p:
    history[p] = (history[p] >> 1) | (R[p] << 7)
    R[p] = 0
```

Here `history[p]` is an 8-bit value, and `>>` and `<<` are bit shifts.

Concrete example

Suppose page A is accessed during epochs 1 and 3, but not during epochs 2 and 4.

Epoch	R observed	History after update
Initially	—	00000000
1	1	10000000
2	0	01000000
3	1	10100000
4	0	01010000

The leftmost bit represents the most recent epoch; older observations move to the right.

Compare two pages:

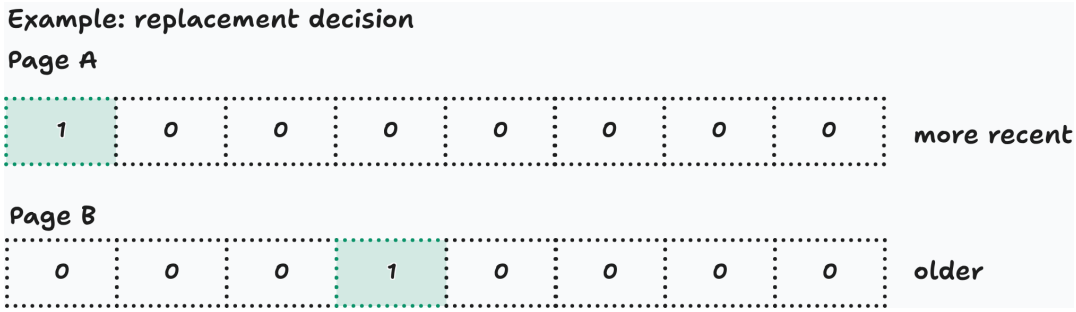


Figure 7.1: epoch bits

The aging algorithm treats these bit histories as unsigned integers and preferentially evicts the page with the smallest value.

In this case, B is selected.

This approximates LRU because an older reference contributes a smaller value than a recent one. It is not exact LRU: the history also reflects how many epochs contained references, and accesses within an epoch cannot be ordered.

2. Where is the history stored?

Normally, the history register is maintained by the operating system in software, not by the MMU in a dedicated hardware field.

It is useful to distinguish:

Information	Typical storage	Maintained by
R bit	Page-table entry	CPU hardware; OS can clear
M bit	Page-table entry	CPU hardware; OS manages as needed
8/16/32-bit history	Kernel-managed page metadata	Operating system

The history could be stored in an auxiliary array indexed by virtual page, or in metadata associated with resident physical pages.

Some architectures offer software-available PTE bits, but a conventional hardware-defined PTE does not provide a dedicated 8-, 16-, or 32-bit aging register.

One interesting correction to the IBN summary: it suggests an inverted page table is necessary to examine the pages of all processes. That is not generally required. The OS can scan conventional page tables belonging to different processes or maintain separate metadata structures. In fact, Übungsblatt 6, Aufgabe 4 explicitly assumes direct page tables, one per process.

ueb06-ibn-SS2025.pdf

3. How important is this for the exam?

I would distinguish the following levels of preparation.

Knowledge	Priority
Explain that the CPU sets R on access	Very high
Explain periodic sampling and clearing of R	High
Explain storing several epochs of R-bit history	High
Calculate scanning overhead and reason about epoch length	High
Simulate a few history-register updates	Medium

Knowledge	Priority
Know kernel data structures and exact implementation details	Low

The trade-off for epoch length is also worth remembering:

- Short epochs: Better time resolution, but more frequent scanning and therefore higher overhead.
- Long epochs: Lower scanning overhead, but less precise information about recent accesses.

My recommendation

Spend enough time to understand the shifting-history mechanism and be able to explain it in 3–4 sentences. You now have sufficient background for that.

Then continue with the main Day 4 material. The detailed implementation of aging counters, page metadata, and reverse mappings can wait until after the core preparation plan.

One especially useful connection: The 2016 exam's question about approximating last-access time can be answered directly with the R-bit sampling and history mechanism above. So this is one of those small details worth retaining, even under time pressure.

8 Day 5 — Deadlocks and CPU scheduling

The main skills today are:

- **Resource situation → graph → E, A, C, R → Banker hand execution → conclusion.**
- **Scheduling policy → execution timeline → completion, turnaround and waiting times.**

This follows the Day 5 plan and Sheet 8 priorities in 15-Operating-Systems-and-Networks-SoSe-25-Notes.pdf and 08-ueb08-ibn-SS2025.pdf.

Deadlocks receive somewhat greater emphasis: the 2016 exam has an 11-point deadlock problem covering definitions, semaphore code, graphs and Banker's main idea. Its appended older minitest also tests SJF and Round Robin. 12-ibn-klausur-2016.pdf The 2026 protocol explicitly reports graph construction followed by Banker execution; it does not explicitly report scheduling, but is an incomplete reconstruction. 13-Ged-chtnisprotokoll_Klausur_-IBN_Ersttermin.pdf

8.1 Deadlocks: what actually prevents progress

A group of processes is **deadlocked** (*verklemmter Zustand*) when every process in the group waits for an event that only another blocked process in that group can cause.

For a resource deadlock, that event is normally the release of a resource.

For example:

- P_1 holds resource X and waits for Y.
- P_2 holds Y and waits for X.
- Both release their resources only after acquiring both and completing their work.

Neither reaches its release operations.

Ordinary blocking is different: a process can wait for a resource held by another process that is still able to finish and release it.

Starvation is also different: a process is repeatedly passed over while other processes continue making progress. In a deadlock, the participating processes cannot make progress through their normal execution.

A deadlock can involve only part of a system. Other processes may continue running.

The four necessary Coffman conditions

For the resource model used here, deadlock requires all four:

Condition	Meaning	Why it matters
Mutual exclusion (*wechselseitiger Ausschluss*)	Each relevant resource **instance** can be held by only one process at a time.	If everyone could use the resource simultaneously, exclusive ownership would not block access.
Hold and wait	A process retains resources while waiting for additional ones.	Its existing allocation remains unavailable to others.
No preemption (*Ununterbrechbarkeit*)	Allocated resources cannot simply be taken away; the owner must release them.	The system cannot resolve the dependency by forcibly reclaiming the needed resource.
Circular wait (*zyklische Wartebedingung*)	There is a cyclic chain of resource dependencies between processes.	The waiting dependencies can lead back to the original process.

Mutual exclusion applies to an **instance**, not necessarily to an entire resource type. A type with three instances can serve three processes simultaneously.

Also, **CPU preemption is different from resource preemption**. Suspending a process and scheduling another does not release its mutexes or allocated devices.

Necessary does not automatically mean sufficient

Two distinctions matter:

1. A system may **permit** all four conditions without currently being deadlocked. Opposite lock orders, for example, allow a bad interleaving, but one process might acquire both locks and finish before the other starts.
2. With multiple instances, a graph can contain a cycle while another process can release an interchangeable instance and break the dependency.

To establish a particular deadlock, show why the relevant waiting cannot be resolved.

Preventing deadlock through acquisition order

Assign exclusive locks a global order, for example:

$$L < M < N.$$

Every process must acquire the locks it needs in increasing order. A process needing L and N acquires L first; one needing M and N acquires M first.

For distinct single-instance locks, a circular wait would require resource ranks to keep increasing and eventually return to the starting rank:

$$L < M < N < L,$$

which is impossible. This eliminates circular wait.

The rule must apply to **all relevant acquisition paths**. Merely changing the release order does not repair inconsistent acquisition.

Your Day 2 example in which writers acquire several permits also illustrates hold and wait: partial allocations remain held while further acquisition blocks. Ordering distinct lock types is a useful prevention rule, but it does not automatically repair repeated partial acquisition from one multi-instance semaphore.

8.2 Resource-allocation graphs

A **resource-allocation graph** (*Ressourcenbelegungsgraph*, RBG) represents both ownership and outstanding requests.

Element	Meaning
Circle P_i	Process
Rectangle for resource type X	Resource type
Dots inside the rectangle	Individual instances
$P_i \to X$	P_i requests an additional instance of X
Instance of $X \to P_i$	That instance is allocated to P_i

Requests normally target a **resource type**: any available instance of that type can satisfy them. Allocation edges originate from particular instances.

If several instances are allocated or requested, show the corresponding number of edges, or clearly label their multiplicity.

What a cycle tells you

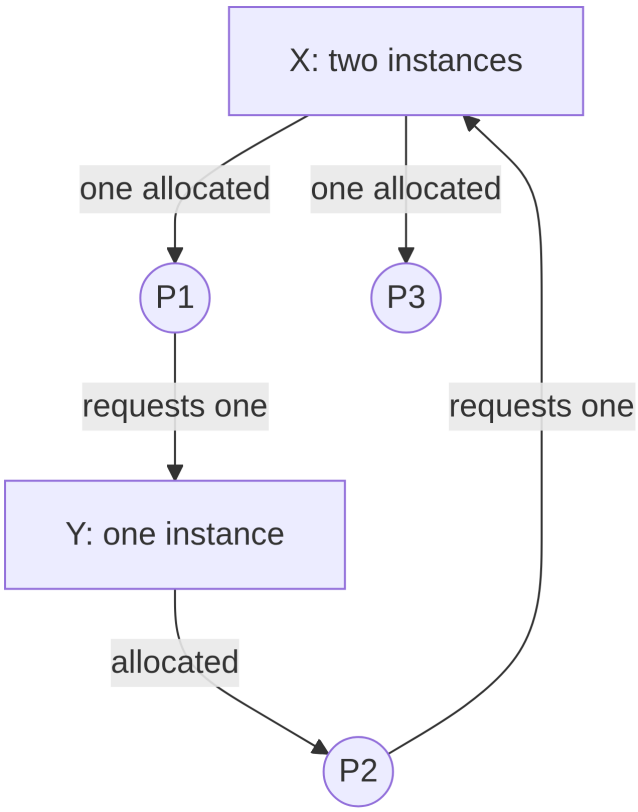
Under the ordinary resource model, where all stated requests must be satisfied:

Graph situation	Conclusion
No directed cycle	No resource deadlock
Cycle, with one instance of each resource type	Deadlock
Cycle, with multiple instances	Deadlock is possible; the cycle alone does not prove it

Consider two instances of X, one instance of Y, and three processes:

- P_1 holds one X and requests Y.
- P_2 holds Y and requests one X.
- P_3 holds the other X and needs nothing further.

The following compact diagram labels instance counts instead of drawing dots:



There is a cycle:

$$P_1 \to Y \to P_2 \to X \to P_1.$$

Nevertheless, P_3 can finish and release its X. Then P_2 can obtain that instance, finish and release Y, allowing P_1 to finish.

The cycle therefore does **not** represent an unavoidable deadlock.

8.3 The course's vectors and matrices

Use the notation from the IBN summary: 01-ibn-zusammenfassung.pdf

For n processes and m resource types:

Symbol	Course meaning
E_j	**Total** number of existing instances of resource type j
A_j	Number of currently free instances of type j, the *Ressourcenrestvektor*
C_{ij}	Number of instances of type j currently allocated to process i, the *Belegungsmatrix*
R_{ij}	Number of additional instances of type j requested by process i, the *Anforderungsmatrix*

Rows correspond to processes; columns correspond to resource types. State their order before writing matrices.

For every resource type:

$$\boxed{E_j = A_j + \sum_{i=1}^{n} C_{ij}}$$

and therefore:

$$\boxed{A_j = E_j - \sum_{i=1}^{n} C_{ij}}.$$

Outstanding requests are **not allocated resources**. Do not subtract R when calculating the initially free resources.

For the preceding graph, using columns (X, Y) and rows (P_1, P_2, P_3):

$$E = (2, 1), \qquad C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad R = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

Thus:

$$A = (2, 1) - (2, 1) = (0, 0).$$

No free resources does not imply deadlock: P_3 has no further request and can finish.

8.4 Executing the course's Banker algorithm by hand

The course procedure repeatedly asks:

> Can some unmarked process obtain its additional requested resources, finish, and release its allocation?

For these hand exercises, assume that once a process receives its stated additional resources, it eventually finishes without further requests and releases everything it holds.

The eligibility test is:

$$\boxed{R_i \leq A}$$

component by component. There must be enough of every resource type.

For example:

$$(1, 0, 2) \not\leq (2, 3, 1),$$

because only one instance of the third type is available. Having enough resources *in total* does not help.

The procedure is:

1. Calculate the initial A.
2. Leave all processes unmarked.
3. Find an unmarked process satisfying $R_i \leq A$.
4. Mark it and record its index.
5. Update:
$$\boxed{A \leftarrow A + C_i}.$$
6. Reconsider the remaining processes with the updated A.
7. Stop when all processes are marked or no remaining process is eligible.

This is the hand procedure used in the Sheet 8 solutions. 14-Operating-Systems-and-Networs-SoSe-25-Solutions.pdf

Why add only C_i?

Conceptually, the process first receives R_i, then releases both its old and newly obtained resources:

$$A_{\text{after}} = A_{\text{before}} - R_i + (C_i + R_i) = A_{\text{before}} + C_i.$$

The newly granted resources cancel. The net increase comes from its **previous allocation**.

Worked reduction of the graph above

Step	Selected process	Request R_i	A before	A after releasing C_i
1	P_3	$(0, 0)$	$(0, 0)$	$(1, 0)$
2	P_2	$(1, 0)$	$(1, 0)$	$(1, 1)$
3	P_1	$(0, 1)$	$(1, 1)$	$(2, 1)$

All processes can be marked. A completion sequence is:

$$P_3,\ P_2,\ P_1.$$

There is no deadlock under the stated model.

If several processes are eligible, any may be selected unless the question specifies a rule. Completing one only increases the working availability vector, so you do not need to search through permutations or backtrack.

Interpreting the result

- **All marked:** a complete execution sequence exists.
- **Some remain unmarked and none is eligible:** the remaining requests cannot be resolved through this reduction; a deadlocked group remains in the current-request model. Report the unmarked processes as the course asks.

Strictly, a process holding no resources can be blocked behind a deadlocked group without belonging to its circular dependency. Distinguish that case if a question specifically asks for the core cycle.

Detection versus avoidance: the meaning of R matters

The IBN summary explicitly uses the procedure in two related ways:

Use	Meaning of R_i	Meaning of failure to find a complete sequence
Deadlock detection	Current outstanding requests	Deadlock remains under the exercise model
Deadlock avoidance / safety test	Maximum remaining additional demand until completion	The state is unsafe; guaranteed completion cannot be established

Textbooks commonly reserve "Banker's algorithm" for avoidance using declared maximum demands. The course also calls the current-request reduction **Bankieralgorithmus**. For the exam, follow the supplied meaning of R.

An **unsafe state is not necessarily already deadlocked**: maximum future demands may never all be requested.

For avoidance, the system tentatively grants a request, updates the hypothetical allocation and availability, and runs the safety test. It grants the request permanently only if a safe completion sequence remains.

8.5 CPU scheduling: the execution model

The CPU scheduler selects a process from the **ready** processes. A process blocked on I/O or a resource is not currently eligible.

A process normally alternates between:

- a **CPU burst**: execution on the CPU;
- an **I/O phase**: performing or waiting for I/O.

Process type	Typical behaviour	Scheduling implication
CPU-bound (*rechenintensiv*)	Long CPU bursts	Frequent switching can add overhead to long computations.
I/O-bound (*I/O-intensiv*)	Many short CPU bursts separated by I/O	Prompt CPU access lets it issue its next I/O operation, helping responsiveness and device utilisation.

"I/O-bound" refers especially to **short CPU bursts and frequent I/O**, not necessarily exceptionally long individual I/O operations.

For the calculation exercises below, use one CPU, one CPU burst per process, no I/O, and zero context-switch overhead.

FCFS — First Come, First Served

Maintain a FIFO ready queue.

When the CPU becomes free, run the process that has waited in that queue longest. It runs until its CPU burst finishes.

FCFS is **non-preemptive**: a new arrival does not interrupt the running process.

A long burst can delay many short ones—the **convoy effect**.

SJF — Shortest Job First

Whenever the CPU becomes free, select the **shortest next CPU burst among processes currently ready**.

For Day 5, use **non-preemptive SJF**, as in the course exercise. Once selected, the process finishes its burst even if a shorter process arrives meanwhile.

- If all processes are ready initially, sort them by burst length.
- With different arrival times, reconsider only the processes that have arrived whenever the CPU becomes free.

Use a stated tie rule, such as earlier arrival, then lower process index.

When all jobs are available initially, burst lengths are known, and overhead is ignored, SJF minimises average waiting time among non-preemptive schedules. Putting a short job before a long one reduces how long that short job waits.

Real systems usually have to estimate future burst lengths.

Round Robin — RR

Maintain a FIFO ready queue and choose a positive **time quantum** q.

For each CPU allocation:

1. Remove the process at the front.
2. Run it for:
$$\Delta t = \min(q, \text{remaining burst}).$$
3. If it finishes, remove it permanently.
4. Otherwise, put it at the back of the ready queue.

Newly arriving processes join the back. They do not interrupt the current slice.

If an arrival coincides exactly with quantum expiry, the ordering can depend on the chosen convention. **State the convention and apply it consistently.**

A process that finishes early uses only the time it needs; the unused part of its quantum is not an idle interval.

Short RR example

All three processes arrive at $t = 0$, initially queued as P_1, P_2, P_3, with bursts $3, 1, 2$ ms. Let $q = 2$ ms.

CPU interval	Process	Remaining burst afterwards	Ready queue afterwards
0–2	P_1	1	P_2, P_3, P_1
2–3	P_2	0	P_3, P_1
3–5	P_3	0	P_1
5–6	P_1	0	Empty

These intervals provide the bars and boundary labels for the Gantt diagram.

Effect of the quantum

- Small q: frequent opportunities to run, but more context-switch overhead in a real system.
- Large q: fewer switches, but longer delays between turns.
- If q is at least every burst length in these one-burst exercises, RR behaves like FCFS.

RR gives regular CPU access to ready processes; it does not necessarily minimise average waiting time.

8.6 Gantt diagrams and time calculations

A **Gantt diagram** places labelled execution intervals along a time axis. Mark every boundary at which execution changes. Include an idle interval if no process is ready.

For scheduling, use:

Symbol	Meaning
a_i	Arrival time
b_i	Total CPU burst
f_i	Completion time: end of the process's **last** execution interval
T_i	Turnaround time (*Durchlaufzeit*)
W_i	Total waiting time (*Wartezeit*)

The formulas are:

$$\boxed{T_i = f_i - a_i}$$

and, for these one-burst exercises without I/O:

$$\boxed{W_i = T_i - b_i = f_i - a_i - b_i}.$$

Average waiting time:

$$\boxed{\overline{W} = \frac{1}{n}\sum_{i=1}^{n} W_i}.$$

If every arrival is at zero, $T_i = f_i$. Otherwise, completion time and turnaround time differ.

Waiting time includes all time spent ready but not running. For non-preemptive execution, it is simply start time minus arrival time. Under RR, it also includes the waits between slices.

In the short RR example, P_1 starts immediately but then waits from $t = 2$ to $t = 5$:

$$W_1 = 6 - 0 - 3 = 3 \text{ ms}.$$

Its first-start delay is zero. That delay, often called **response time**, is different from total waiting time.

The formula $W = T - b$ relies on the exercise's no-I/O model. In a general process, turnaround time also contains blocked time.

A reliable manual procedure

1. Write down arrivals, bursts, policy, quantum and tie rules.
2. Track the ready set; for RR, also track queue order and remaining bursts.
3. Construct the complete execution timeline.
4. Read each f_i from its last interval.
5. Calculate T_i, then W_i, then the average.

Check that:

- no process runs before arriving;
- each process receives exactly its stated burst;
- no waiting time is negative;
- with no idle time or overhead, final completion equals the sum of all bursts.

Fairness and starvation

Under the usual assumptions of finite bursts and a functioning scheduler:

Policy	Can a ready process starve?
FCFS	No: new arrivals cannot overtake it.
SJF	Yes: a continuing stream of short jobs can indefinitely postpone a long job.
RR	No: the FIFO queue repeatedly gives it a turn.
Fixed-priority scheduling	Yes: continuing higher-priority work can postpone it indefinitely.

In a **finite closed exercise set**, SJF eventually runs every process. Its starvation risk concerns continuing arrivals.

Sheet 8 A6 names **three** algorithms despite later saying "four"; use FCFS, SJF and RR.

8.7 Common exam traps

- Giving code that *can* deadlock without showing an actual blocking interleaving.
- Claiming every graph cycle proves deadlock, without checking instance counts.
- Reversing request and allocation arrows.
- Treating E as free resources, or treating R_i as including resources already held.
- Comparing resource-vector totals instead of comparing every component.
- Updating A incorrectly: after marking a process, use $A + C_i$, and reconsider previously ineligible processes.
- Treating non-preemptive SJF as preemptive, or choosing a job that has not arrived.
- Retaining a completed process in an RR queue, or giving it a full quantum after it finishes.
- Confusing completion with turnaround, or first-start delay with total waiting.

8.8 Exam-like exercises

Allow approximately **60–75 minutes** for these three problems. Show intermediate states and concise justifications.

Exercise 1 — A three-process semaphore deadlock

Suggested time: 12–15 minutes.

Three exclusive resources are controlled by:

```
Semaphore L = 1
Semaphore M = 1
Semaphore N = 1
```

The processes execute:

```
P1:
    wait(L)
    wait(M)
    work()
    signal(M)
    signal(L)

P2:
    wait(M)
```

```
    wait(N)
    work()
    signal(N)
    signal(M)

P3:
    wait(N)
    wait(L)
    work()
    signal(L)
    signal(N)
```

Assume `work()` eventually finishes if reached. Processes retain acquired resources while blocked.

1. Give an explicit interleaving that produces deadlock. For each acquisition attempt, state whether it succeeds or blocks, and which resources are held.
2. Define deadlock, then identify all four Coffman conditions **in this execution**.
3. Explain why giving each blocked process another CPU turn cannot resolve the situation.
4. Repair the acquisition order using a single global resource order. Write the corrected acquisition and release operations for each process.
5. Would making all three semaphores fair/FIFO repair the original deadlock? Justify your answer.

Exercise 2 — Resource situation → graph → matrices → Banker

Suggested time: 22–27 minutes.

An image-processing service manages:

- three interchangeable camera units K;
- two interchangeable storage interfaces S;
- two interchangeable accelerator units G.

Every instance is exclusive and cannot be forcibly reclaimed. A process retains its current allocation while waiting. Once its complete additional request is satisfied, it can finish without further requests and releases all its resources.

The current situation is:

Process	Currently holds	Requests additionally
P_1	One camera and one accelerator	One storage interface

Process	Currently holds	Requests additionally
P_2	One storage interface	One camera and one accelerator
P_3	One camera	One storage interface
P_4	One camera and one storage interface	One accelerator

Use columns (K, S, G) and rows (P_1, P_2, P_3, P_4).

1. Draw the resource-allocation graph, including all instances and outstanding requests.
2. Identify at least one directed cycle. Explain what its existence lets you conclude **before** executing the algorithm.
3. Derive E, C, R, A. Check resource conservation separately for all three types.
4. Execute the course's Banker procedure. Whenever several processes are eligible, select the one with the **smallest index**.

   Record:

Step	Selected process	Its request	A before	A after

5. State whether all processes can complete. Give a completion sequence or the unmarked processes, and justify the conclusion.

Now consider an **independent variant of the original state**: all holdings remain unchanged, but P_4 additionally requests **one camera and one accelerator**, rather than only one accelerator.

6. Update the graph and R, then repeat the algorithm. Does the conclusion change?
7. Explain why the presence of a free resource instance is insufficient by itself to establish progress in this variant.

Exercise 3 — Scheduling with arrivals and queue boundaries

Suggested time: 25–30 minutes.

Five processes have the following arrival times and CPU bursts:

Process	Arrival a_i, ms	Burst b_i, ms
P_1	0	7
P_2	1	4
P_3	2	1

Process	Arrival a_i, ms	Burst b_i, ms
P_4	4	4
P_5	5	2

Use one CPU, no I/O and zero context-switch overhead.

Apply:

- FCFS;
- **non-preemptive** SJF;
- RR with $q = 2$ ms.

Conventions:

- For SJF ties, prefer earlier arrival, then lower index.
- In RR, arrivals join the queue when they occur.
- An arrival exactly at quantum expiry is enqueued **before** the unfinished running process is re-enqueued.
- A completed process is removed immediately.

1. Draw a complete Gantt diagram for each policy.
2. For each policy, calculate every process's:

$$f_i, \qquad T_i, \qquad W_i.$$

3. Calculate the average waiting time for each policy. Which is smallest for this workload?
4. For RR, give the ready queue immediately before selecting the next process at:

$$t = 2, \qquad t = 4, \qquad t = 5.$$

   Explain how the stated arrival convention affects these queues.
5. Explain whether P_2's arrival at $t = 1$ interrupts P_1 under SJF or RR. Then determine **only the SJF execution order** if all five processes instead arrive at $t = 0$.
6. If the RR quantum becomes 10 ms, which studied algorithm does its execution match for this workload? Explain. Also identify which of FCFS, SJF and RR can starve a ready process under a continuing stream of arrivals.

8.9 Solutions

These solutions use the stated resource assumptions, SJF tie-breaking rule, and RR convention that arrivals at quantum expiry are enqueued **before** the unfinished running process.

Exercise 1 — A three-process semaphore deadlock

Deadlocking execution

One valid interleaving is:

Step	Process	Operation	Result	Resources held afterwards
1	P_1	`wait(L)`	Succeeds	$P_1 : L$
2	P_2	`wait(M)`	Succeeds	$P_1 : L;\ P_2 : M$
3	P_3	`wait(N)`	Succeeds	$P_1 : L;\ P_2 : M;\ P_3 : N$
4	P_1	`wait(M)`	Blocks: P_2 holds M	Unchanged
5	P_2	`wait(N)`	Blocks: P_3 holds N	Unchanged
6	P_3	`wait(L)`	Blocks: P_1 holds L	Unchanged

The resulting dependencies are:

$$P_1 \text{ waits for } P_2, \qquad P_2 \text{ waits for } P_3, \qquad P_3 \text{ waits for } P_1.$$

All three are blocked before `work()` and before their release operations.

Other interleavings are equally valid if they establish these holdings and blocked requests.

Definition and Coffman conditions

Definition: A group of processes is deadlocked when every process in the group waits for an event that only another blocked process in that group can cause.

Here, those events are resource releases.

Condition	Explanation in this execution
Mutual exclusion	Each semaphore has one permit, so each resource can be held by only one process.
Hold and wait	Each process retains its first resource while waiting for its second.
No preemption	Resources are released through `signal`; they cannot be forcibly reclaimed under the assumptions.
Circular wait	$P_1 \to P_2 \to P_3 \to P_1$.

Why more CPU turns do not help

Each process is blocked at its second `wait`. It cannot execute the later `signal` operations until that `wait` succeeds.

Scheduling another process does not release the resource held by the suspended process. **CPU scheduling cannot break this resource dependency.**

Repair through a global acquisition order

Choose:

$$L < M < N.$$

Every process acquires its required resources in increasing order:

```
P1:
    wait(L)
    wait(M)
    work()
    signal(M)
    signal(L)

P2:
    wait(M)
    wait(N)
    work()
    signal(N)
    signal(M)

P3:
    wait(L)
    wait(N)
    work()
    signal(N)
    signal(L)
```

Only P_3's acquisition order needed changing.

A circular wait would now require resource ranks to increase around a cycle and return to the starting rank, which is impossible.

Releasing in reverse order is conventional. **Consistent acquisition ordering** is what prevents this deadlock.

Would FIFO semaphores repair it?

No.

FIFO determines which waiting process receives a permit **when one becomes available**. Here, no process can reach its release operations, so no permit becomes available.

Fairness does not eliminate this circular wait.

Exercise 2 — Resource situation → graph → matrices → Banker

Resource-allocation graph

One valid naming of the individual instances is:

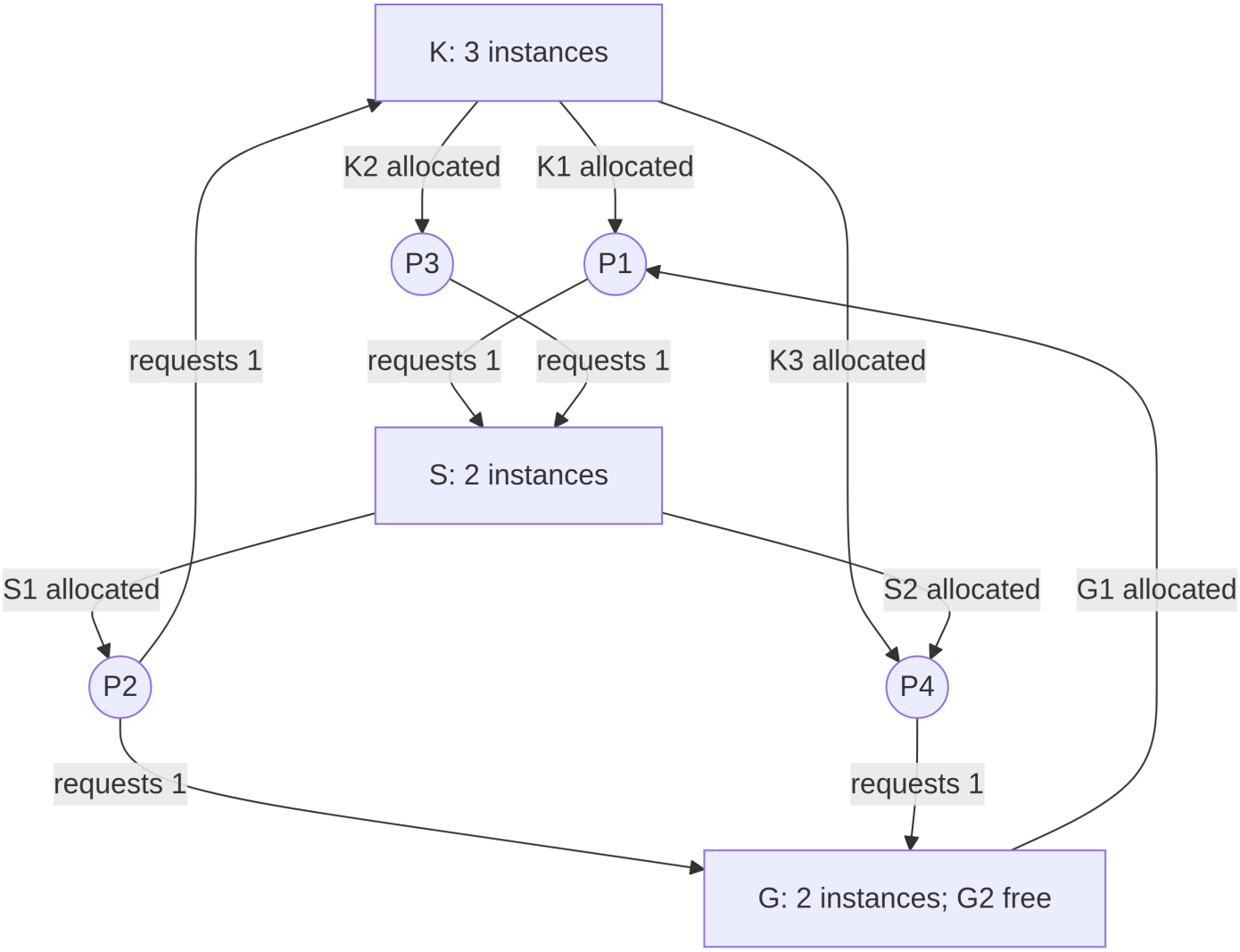
Type	Instance allocations
Cameras K	$K_1 \to P_1,\ K_2 \to P_3,\ K_3 \to P_4$
Storage interfaces S	$S_1 \to P_2,\ S_2 \to P_4$
Accelerators G	$G_1 \to P_1;\ G_2$ is free

The instance numbers are arbitrary; assigning different instance names gives an equivalent graph.

The requests are:

$$P_1 \to S, \qquad P_2 \to K, \qquad P_2 \to G, \qquad P_3 \to S, \qquad P_4 \to G.$$

This compact graph labels allocation edges with instance names:



Cycle and its interpretation

One directed cycle is:

$$\boxed{P_1 \to S \to P_2 \to K \to P_1}.$$

Another is:

$$P_1 \to S \to P_4 \to G \to P_1.$$

Because the resource types have multiple instances, **a cycle alone does not establish deadlock**. An interchangeable instance may become available through another process's completion.

Vectors and matrices

Use columns (K, S, G) and rows (P_1, P_2, P_3, P_4).

The total resource vector is:

$$\boxed{E = (3, 2, 2)}.$$

The allocation matrix is:

$$\boxed{C = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}}.$$

The request matrix is:

$$\boxed{R = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}.$$

The column sums of C are:

$$\sum_i C_i = (3, 2, 1).$$

Therefore:

$$\boxed{A = E - \sum_i C_i = (0, 0, 1)}.$$

Resource conservation holds separately:

Type	Total	Allocated	Free
K	3	3	0
S	2	2	0
G	2	1	1

Banker execution

Initially:

$$A = (0, 0, 1).$$

- P_1 is ineligible: it needs a storage interface.
- P_2 is ineligible: it needs a camera.
- P_3 is ineligible: it needs a storage interface.
- P_4 is eligible: its request is $(0, 0, 1)$.

Applying the smallest-eligible-index rule:

Step	Selected process	Request R_i	A before	Allocation released C_i	A after
1	P_4	$(0, 0, 1)$	$(0, 0, 1)$	$(1, 1, 0)$	$(1, 1, 1)$
2	P_1	$(0, 1, 0)$	$(1, 1, 1)$	$(1, 0, 1)$	$(2, 1, 2)$
3	P_2	$(1, 0, 1)$	$(2, 1, 2)$	$(0, 1, 0)$	$(2, 2, 2)$
4	P_3	$(0, 1, 0)$	$(2, 2, 2)$	$(1, 0, 0)$	$(3, 2, 2)$

After P_4 finishes, P_1, P_2, P_3 are all eligible. The specified rule selects P_1 first.

Remember:

$$A_{\text{after}} = A_{\text{before}} + C_i.$$

The requested resources are temporarily granted and subsequently returned; they do not produce an additional net increase.

Conclusion for the original state

All processes can complete:

$$\boxed{P_4,\ P_1,\ P_2,\ P_3}.$$

Thus:

$$\boxed{\text{No deadlock under the stated model.}}$$

The final availability vector equals E, as expected when every process has released its resources.

Other completion sequences can also be valid, but the question's smallest-index rule determines the sequence above.

Changed-request variant

Return to the **original allocations and initial availability**. Only P_4's request changes.

The new request matrix is:

$$\boxed{R' = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}}.$$

The graph gains an additional request edge:

$$P_4 \to K.$$

Its request edge to G remains.

Since E and C are unchanged:

$$A = (0, 0, 1).$$

Process	Request	Why it is ineligible
P_1	$(0, 1, 0)$	No storage interface is free.
P_2	$(1, 0, 1)$	No camera is free.

Process	Request	Why it is ineligible
P_3	$(0, 1, 0)$	No storage interface is free.
P_4	$(1, 0, 1)$	No camera is free.

No process can be marked. The algorithm stops immediately with:

$$\boxed{\text{Unmarked processes: } P_1, P_2, P_3, P_4}.$$

Therefore:

$$\boxed{\text{The variant is deadlocked under the stated model.}}$$

Why the free accelerator does not establish progress

The free accelerator satisfies only part of the requests of P_2 and P_4. Both also need a camera.

Meanwhile, P_1 and P_3 need storage interfaces held by P_2 and P_4.

No process can obtain its **complete request**, finish and release its existing allocation. Availability must be checked **component by component**, for the complete request vector.

Exercise 3 — Scheduling with arrivals and queue boundaries

All times below are in milliseconds.

Use:

$$T_i = f_i - a_i, \qquad W_i = T_i - b_i.$$

Gantt diagrams

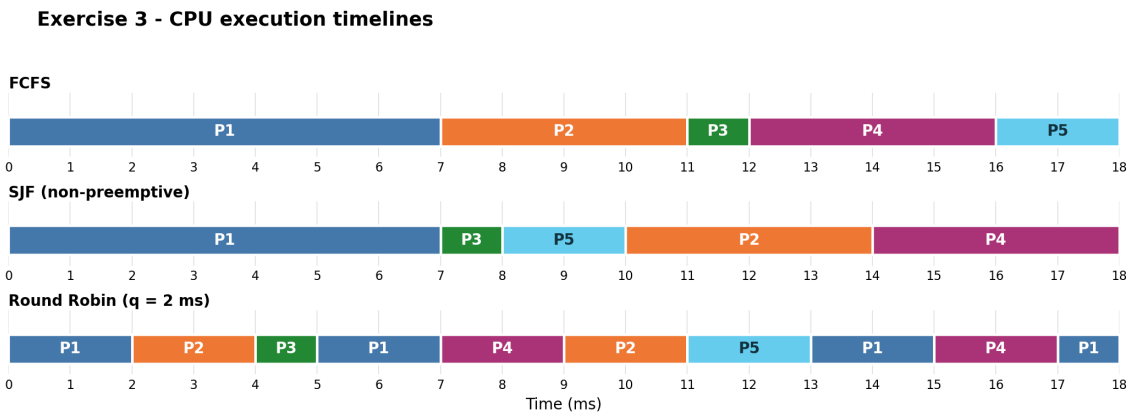


Figure 8.1: gantt diagrams

FCFS

The arrival order is:

$$P_1,\ P_2,\ P_3,\ P_4,\ P_5.$$

Interval	Running process
0–7	P_1
7–11	P_2
11–12	P_3
12–16	P_4
16–18	P_5

The resulting times are:

Process	Arrival a_i	Burst b_i	Completion f_i	Turnaround T_i	Waiting W_i
P_1	0	7	7	7	0
P_2	1	4	11	10	6
P_3	2	1	12	10	9
P_4	4	4	16	12	8
P_5	5	2	18	13	11

Hence:

$$\boxed{\overline{W}_{\text{FCFS}} = \frac{0+6+9+8+11}{5} = \frac{34}{5} = 6.8 \text{ ms}}.$$

Non-preemptive SJF

At $t = 0$, only P_1 has arrived. It runs until $t = 7$, because SJF is non-preemptive.

At $t = 7$, all other processes are ready. Their burst lengths are:

$$P_3 : 1, \qquad P_5 : 2, \qquad P_2 : 4, \qquad P_4 : 4.$$

P_2 precedes P_4 because it arrived earlier.

The execution order is:

$$\boxed{P_1,\ P_3,\ P_5,\ P_2,\ P_4}.$$

Interval	Running process
0–7	P_1
7–8	P_3
8–10	P_5
10–14	P_2
14–18	P_4

Process	Arrival a_i	Burst b_i	Completion f_i	Turnaround T_i	Waiting W_i
P_1	0	7	7	7	0
P_2	1	4	14	13	9
P_3	2	1	8	6	5
P_4	4	4	18	14	10
P_5	5	2	10	5	3

Thus:

$$\boxed{\overline{W}_{\text{SJF}} = \frac{0+9+5+10+3}{5} = \frac{27}{5} = 5.4 \text{ ms}}.$$

Round Robin with $q = 2$ ms

Queue fronts are shown on the **left**. Each queue below is recorded after the interval's arrivals, completion or re-enqueue have been handled.

Interval	Running process	Its remaining burst afterwards	Ready queue afterwards
0–2	P_1	5	P_2, P_3, P_1
2–4	P_2	2	P_3, P_1, P_4, P_2
4–5	P_3	0	P_1, P_4, P_2, P_5
5–7	P_1	3	P_4, P_2, P_5, P_1
7–9	P_4	2	P_2, P_5, P_1, P_4
9–11	P_2	0	P_5, P_1, P_4
11–13	P_5	0	P_1, P_4
13–15	P_1	1	P_4, P_1
15–17	P_4	0	P_1
17–18	P_1	0	Empty

Notice that P_3 needs only one millisecond. Its interval ends at $t = 5$, so the next process starts immediately.

Process	Arrival a_i	Burst b_i	Completion f_i	Turnaround T_i	Waiting W_i
P_1	0	7	18	18	11
P_2	1	4	11	10	6
P_3	2	1	5	3	2
P_4	4	4	17	13	9
P_5	5	2	13	8	6

For example:

$$W_1 = 18 - 0 - 7 = 11 \text{ ms}.$$

Although P_1 starts immediately, it waits between later slices.

The average is:

$$\boxed{\overline{W}_{\text{RR}} = \frac{11 + 6 + 2 + 9 + 6}{5} = \frac{34}{5} = 6.8 \text{ ms}}.$$

Comparison of average waiting times

Policy	Average waiting time
FCFS	6.8 ms
Non-preemptive SJF	5.4 **ms**
RR, $q = 2$ ms	6.8 ms

SJF gives the smallest average waiting time for this workload.

FCFS and RR happen to have equal averages, despite different individual waiting times. For example, P_3 waits 9 ms under FCFS and only 2 ms under RR.

All schedules end at:

$$7 + 4 + 1 + 4 + 2 = \boxed{18 \text{ ms}},$$

because the CPU is continuously busy and overhead is ignored.

Requested RR queue snapshots

These queues are recorded **immediately before selecting the next process**, after all events at the stated time have been handled.

Time	Ready queue, front first	Next process
$t = 2$	$\boxed{P_2, P_3, P_1}$	P_2
$t = 4$	$\boxed{P_3, P_1, P_4, P_2}$	P_3
$t = 5$	$\boxed{P_1, P_4, P_2, P_5}$	P_1

Explanation:

- **At $t = 2$:** P_2 has been waiting since $t = 1$. Newly arriving P_3 is enqueued before unfinished P_1 is re-enqueued.
- **At $t = 4$:** P_3, P_1 are already waiting. Newly arriving P_4 is appended before unfinished P_2.
- **At $t = 5$:** P_3 finishes and is removed. Newly arriving P_5 joins behind P_1, P_4, P_2.

Does P_2's arrival interrupt P_1?

No, under either policy.

- **Non-preemptive SJF:** P_1 continues until its burst finishes at $t = 7$.
- **RR:** the arrival does not interrupt the current slice. P_1 continues until quantum expiry at $t = 2$.

SJF order if all processes arrive at zero

Sort by burst length, breaking the tie between P_2 and P_4 by lower index:

$$\boxed{P_3,\ P_5,\ P_2,\ P_4,\ P_1}.$$

Their bursts are:

$$1,\ 2,\ 4,\ 4,\ 7.$$

RR with a quantum of 10 ms

Every burst is at most 7 ms, so every process finishes during its first CPU allocation.

RR therefore matches **FCFS** for this workload:

$$\boxed{P_1,\ P_2,\ P_3,\ P_4,\ P_5}.$$

Starvation under continuing arrivals

Policy	Starvation possible?	Reason
FCFS	**No**, under the usual finite-burst assumptions	New arrivals join behind already waiting processes.
SJF	**Yes**	A continuing stream of short jobs can indefinitely postpone a long ready job.
RR	**No**, under the usual FIFO and positive-quantum assumptions	A ready process repeatedly receives a turn.

For this finite set of five processes, all three policies eventually finish every process.

Suggested self-marking guide

This is an **unofficial 60-point scheme** you can use for comparison:

Exercise	Suggested points	Allocation
Exercise 1	**12**	Trace 3; definition and conditions 5; CPU-turn explanation 1; repair 2; FIFO explanation 1
Exercise 2	**20**	Graph 4; cycle interpretation 1; vectors/matrices 4; Banker trace 4; original conclusion 1; variant 4; free-resource explanation 2
Exercise 3	**28**	Gantt diagrams 8; completion/turnaround/waiting tables 9; averages 2; queues 3; arrival behaviour and simultaneous-arrival SJF 3; large quantum and starvation 3

For **Folgefehler**, identify the original mistake and still award method credit for later steps calculated correctly from your earlier result. A consistently propagated arithmetic error should receive less deduction than using the wrong algorithm or resource-update rule.